



Ben Gurion University of the Negev

Faculty of Engineering Science

School of Electrical and Computer Engineering

Dept. of Electrical and Computer Engineering

Fourth Year Engineering Project

Final Report

Reliable High-Speed Networking on FPGA Platforms (UDP, TCP, and UDP++ Protocols)

Project number:	p-2026-142
Students:	Oshrath Elijah, Yair Tayri
Supervisors:	Prof. Ron Dvora & Eyal Lampel
Sponsors:	ELTA Systems Ltd.
Submitting date:	26, July, 2026

Table of Contents

1. Abstract.....	5
1.1 English Abstract.....	5
1.2 Hebrew Abstract: תקציר.....	6
2. List of Abbreviations.....	7
3. Introduction.....	10
3.1 Motivation and Problem Statement.....	10
3.2 Alternative Approaches.....	10
3.3 Selected Approach.....	11
3.4 Report Organization.....	11
4. General System Architecture and Test Environment.....	13
4.1 System Level Architecture.....	13
4.2 Test and Measurement Environment.....	14
4.3 Toolchain Version Update.....	18
4.4 Specification Compliance Summary.....	18
5. TCP Performance Server.....	19
5.1 Final System Specification.....	19
5.2 Changes from Preliminary Design.....	19
5.3 Hardware Platform Configuration.....	20
5.4 Memory Architecture: DDR as a Feasibility Condition.....	22
5.5 lwIP Stack Selection and Performance Tuning.....	22
5.6 TCP Performance Results.....	23
6. UDP Echo Server.....	25
6.1 Final System Specification.....	25
6.2 Changes from Preliminary Design.....	26
6.3 MicroBlaze Subsystem and Memory Allocation.....	27
6.4 Clock Architecture.....	27

6.5 UDP Offload Engine: IP Packaging and Integration.....	28
6.6 AXI DMA and BRAM Buffering Architecture.....	31
6.7 Custom IP Development: tuser_builder and Stream Splitter.....	32
6.8 Software Flow and On Chip Debug Infrastructure.....	33
6.9 Data Path Verification: The 0x42 Packet Walkthrough.....	38
6.10 UDP Performance Results.....	42
6.11 Resource Utilization Analysis.....	46
7. UDP++ Reliability Layer.....	50
7.1 Initial Design Requirements.....	52
7.2 Reliability Concept and Header Design.....	52
7.3 Sequence/NAK Architecture and Register Map.....	52
7.4 Retransmission Logic and TX History Buffer.....	54
7.5 Inline Integration and Dynamic TUSER Metadata.....	55
7.6 Software Control Plane.....	58
7.7 UDP++ Comparative Results.....	59
7.7.1 TX Path and Header Injection (Test Case 1).....	59
7.7.2 NAK Handling and Selective Retransmission (Test Case 2).....	66
7.7.3 TX History Window: Hit versus Miss (Test Case 3).....	68
7.7.4 Out of Order Tolerance (Test Case 4).....	72
7.7.5 RX Path and Header Stripping (Test Case 5).....	73
7.7.6 RX Side Loss Detection and NAK Generation (Test Case 6).....	79
7.7.7 Resource, Timing, and Power Cost.....	83
7.7.8 Latency and Throughput.....	86
8. Summary of Results.....	91
9. Problems and Solutions.....	95
9.1 PHY Initialization Failures (Electrical Domain).....	95
9.1.1 I/O Bank Voltage Mismatch: "PHY setup failure"	95

9.1.2 PHY Latched in Reset at Power Up.....	95
9.2 Memory and Linker Constraints (Software Domain).....	96
9.2.1 TCP Application Footprint vs. On Chip Memory.....	96
9.3 Clocking and Synchronization (Timing Domain).....	96
9.3.1 SGMII Reference Clock Mismatch: 625 MHz Board Preset vs. 125 MHz Design.....	96
9.3.2 Auto Negotiation Fragility on the TCP Platform.....	97
9.4 Integration Defects (Logic Domain).....	97
9.4.1 Unconnected DMA Clock Inputs After Connection Automation.....	97
9.4.2 Reset Polarity Inconsistency Across the UOE Boundary.....	98
9.4.3 Mid Transmission Metadata Corruption Risk: TVALID/TLAST Coordination....	98
9.5 External Constraint: Laboratory Access Disruption.....	99
10. Conclusions and Recommendations.....	100
10.1 Achievement of Project Goals.....	100
10.2 Comparison: Plan vs. Execution.....	101
10.3 Advantages and Disadvantages of the Delivered Architecture.....	103
10.4 Recommendations and Future Work.....	104
11. References.....	106
12. Appendices.....	107
12.1 Appendix A: Source Code.....	107
12.1.1 UDP_Main.c: Structured Code Walkthrough.....	107
12.1.2 udp_pp.c: UDP++ Control Software Walkthrough.....	109
12.1.3 tuser_builder_v1_0.vhd: Dynamic TUSER Length Injection.....	110
12.1.4 udp_pp VHDL Source Tree: Architectural Walkthrough.....	110
12.2 Appendix B: Management Tasks Table.....	113
12.3 Appendix C: Linker Script and Memory Map Details.....	114
12.4 Appendix D: Grade Recommendation Form.....	116

1. Abstract

1.1 English Abstract

Reliable High-Speed Networking on FPGA Platforms (UDP, TCP, and UDP++ Protocols)

Students: Elijah Oshrath, Tayri Yair

elijahos@post.bgu.ac.il, tayriy@post.bgu.ac.il

Advisers: Dvora Ron, Lampel Eyal

Background: High speed Ethernet is fundamental in modern embedded systems. UDP delivers throughput without reliability, whereas TCP guarantees delivery at significant resource and latency cost.

Aim: The project implemented and evaluated a primarily software based TCP server and a UDP server whose data path is implemented mainly in hardware, on FPGA platforms. The UDP++ reliability layer was subsequently implemented, and its performance was compared against both servers.

Methods: A TCP performance server was implemented on a custom Zynq 7000 board, utilizing the ARM Cortex A9 Processing System, external DDR3 memory, and an optimized lwIP stack. A UDP echo server was implemented mostly in Programmable Logic on the VCU118 (UltraScale+) platform, integrating a MicroBlaze processor, a custom UDP Offload Engine (UOE), AXI DMA, and a Tri Mode Ethernet MAC with SGMII PHY. Verification combined Wireshark captures, dedicated stress testing software, and on chip ILA monitoring. A 64 KB internal BRAM buffer was allocated, to which the DMA writes the received payload and from which it reads the data for echo transmission.

Findings: The TCP server sustained ~940 Mbps over a 300 second run. The UDP data path achieved ~850 Mbps average throughput under sustained stress (~1% loss) and zero packet loss in latency testing. The UDP++ reliability layer achieved a throughput of ~700 Mbps for 512 byte payloads, with a median RTT of ~507 μ s. The NAK based selective retransmission mechanism was implemented in hardware and validated in system tests.

Conclusion: The project results showed that high rate communication servers can be implemented on FPGA platforms through a combination of software and hardware. Among the implementations tested, the TCP server achieved the highest throughput, but the difference also stems from the systems being implemented in different ways and on different boards, so it should not be concluded that TCP is generally faster than UDP. The UDP++ layer improved communication reliability at the cost of some reduction in throughput, an increase in RTT, and additional FPGA resource usage. The choice between TCP, UDP, and UDP++ therefore depends on the application requirements in terms of throughput, reliability, latency, and hardware resources.

Keywords: FPGA, Zynq, MicroBlaze, Ethernet, UDP, TCP, UDP++, AXI Stream, DMA, Hardware Offload.

1.2 Hebrew Abstract: תקציר

מימוש תקשורת רשת אמינה במהירות גבוהה על פלטפורמות FPGA באמצעות פרוטוקולי TCP, ++UDP, UDP

שמות הסטודנטים: אושרת אליזה, יאיר טיירי

elijahos@post.bgu.ac.il, tayriy@post.bgu.ac.il

שמות המנחים: פרופ' רון דבורא, אייל למפל

רקע: תקשורת Ethernet במהירות גבוהה היא מרכיב יסודי במערכות Embedded מודרניות. UDP מספק קצב העברה גבוה ללא אמינות, בעוד TCP מבטיח אמינות במחיר משמעותי של משאבים והשהיה. **מטרה:** בפרויקט מומשו ונבדקו שרת TCP המבוסס בעיקר על תוכנה ושרת UDP בעל נתיב נתונים הממומש בעיקר בחומרה, על גבי פלטפורמות FPGA. בהמשך מומשה שכבת האמינות ++UDP, וביצועיה הושוו לביצועי שני השרתים.

שיטות: שרת ביצועי TCP מומש על לוח Zynq 7000 מותאם, תוך שימוש במעבד ARM Cortex A9 שב Processing System, זיכרון DDR3 חיצוני ומחסנית lwIP שעברה אופטימיזציה. שרת UDP Echo מומש רובו בלוגיקה הניתנת לתכנות (PL) על פלטפורמת (UltraScale+) VCU118, בשילוב מעבד MicroBlaze, מנוע UDP Offload Engine (UOE) מותאם, בקר AXI DMA ובקר Tri Mode Ethernet MAC עם שכבה פיזית בתקן SGMII. האימות שילב לכידות Wireshark, תוכנת בדיקות עומס ייעודית וניטור פנימי באמצעות ILA. הוקצה BRAM פנימי בגודל 64 KB, שאליו ה-DMA כותב את ה-payload המתקבל וממנו הוא קורא את הנתונים לצורך החזרת האות (echo).

ממצאים: שרת ה-TCP שמר על קצב של כ 940 Mbps לאורך ריצה של 300 שניות. נתיב הנתונים של ה-UDP השיג קצב ממוצע של כ 850 Mbps ואפס אובדן חבילות בבדיקת ההשהיה. תחת עומס רציף נצפה אובדן של כ 1%. שכבת ה-++UDP השיגה קצב העברה של כ 700 Mbps עבור Payload בגודל 512 בתים, עם זמן RTT חיצוני של כ 507 מיקרו שניות. מנגנון השידור החוזר הסלקטיבי מבוסס ה-NAK מומש בחומרה ואומת בבדיקות המערכת.

מסקנה: תוצאות הפרויקט הראו כי ניתן לממש שרתי תקשורת בקצבים גבוהים על גבי פלטפורמות FPGA באמצעות שילוב של תוכנה וחומרה. במימושים שנבדקו, שרת ה-TCP השיג את התפוקה הגבוהה ביותר, אך ההבדל נבע גם מכך שהמערכות מומשו בדרכים שונות ועל גבי לוחות שונים, ולכן אין להסיק מכך ש-TCP מהיר יותר מ-UDP באופן כללי. שכבת ה-++UDP שיפרה את אמינות התקשורת, במחיר של ירידה מסוימת בתפוקה, עלייה בזמן ה-RTT ושימוש נוסף במשאבי ה-FPGA. לכן הבחירה בין TCP, UDP ו-++UDP תלויה בדרישות היישום מבחינת תפוקה, אמינות, השהיה ומשאבי חומרה.

מילות מפתח: FPGA, Zynq, MicroBlaze, Ethernet, UDP, TCP, UDP++, AXI Stream, DMA, עיבוד חומרה.

2. List of Abbreviations

Abbreviation	Meaning
ACK	Acknowledgment
ARQ	Automatic Repeat request
AXI	Advanced eXtensible Interface (ARM AMBA)
AXI4 Lite	Memory mapped AXI interface for control and status registers
AXI4 Stream	Address less streaming interface using valid+ready handshaking
BRAM	Block Random Access Memory (FPGA on chip memory)
BSP	Board Support Package
CPU	Central Processing Unit
DDR / DDR3	Double Data Rate synchronous DRAM
DHCP	Dynamic Host Configuration Protocol
DMA	Direct Memory Access
DUT	Device Under Test
EMIO	Extended Multiplexed I/O (Zynq PS to PL routing)
FCS	Frame Check Sequence
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
GMII	Gigabit Media Independent Interface
HDL	Hardware Description Language
ILA	Integrated Logic Analyzer
INTC	Interrupt Controller
IP	Intellectual Property (core) / Internet Protocol (per context)
JTAG	Joint Test Action Group (debug/programming interface)
LMB	Local Memory Bus (MicroBlaze)

LUT	Look Up Table
LVC MOS	Low Voltage CMOS I/O standard
LVDS	Low Voltage Differential Signaling
lwIP	Lightweight IP (open source TCP/IP stack)
MAC	Media Access Control
MDIO	Management Data Input/Output
MDM	MicroBlaze Debug Module
MIO	Multiplexed I/O (Zynq PS)
MM2S	Memory Mapped to Stream (DMA read channel)
NAK	Negative Acknowledgment
PCS/PMA	Physical Coding Sublayer / Physical Medium Attachment
PDR	Preliminary Design Review
PHY	Physical layer transceiver
PL	Programmable Logic
PS	Processing System (Zynq hard processor subsystem)
RETX	Retransmission
RGMII	Reduced Gigabit Media Independent Interface
RTT	Round Trip Time
RX / TX	Receive / Transmit
S2MM	Stream to Memory Mapped (DMA write channel)
SGMII	Serial Gigabit Media Independent Interface
SLR	Super Logic Region (UltraScale+ die segment)
SoC	System on Chip
TCP	Transmission Control Protocol
TDATA	AXI4 Stream Data Signal
TEMAC	Tri Mode Ethernet Media Access Controller

TLAST	AXI4 Stream End of Packet Signal
TREADY	AXI4 Stream Receiver Ready Signal
TUSER	AXI4 Stream user defined sideband signal
TVALID	AXI4 Stream Data Valid Signal
UART	Universal Asynchronous Receiver Transmitter
UDP	User Datagram Protocol
UDP++	Custom UDP based reliability layer (this project)
UOE	UDP Offload Engine
VHDL	VHSIC Hardware Description Language
XDC	Xilinx Design Constraints
XSA	Xilinx Support Archive (exported hardware platform)

Table 2.1: List of abbreviations.

3. Introduction

3.1 Motivation and Problem Statement

Modern embedded systems, radar signal processing, real time video streaming, and distributed avionics among them, depend on sustained Gigabit class Ethernet communication. At the transport layer, system designers face a fundamental tradeoff between the two dominant protocols. UDP (RFC 768) is connectionless and stateless: its minimal overhead permits transmission at near line rate, but it provides no delivery guarantee, no ordering, and no recovery from packet loss. TCP (RFC 9293) provides a reliable, ordered byte stream through connection management, sequence numbering, acknowledgments, and retransmission. These mechanisms, however, impose per connection state machines, large retransmission buffers, and RTT dependent timers, resulting in reduced throughput, nondeterministic latency, and substantial resource utilization [1].

For latency sensitive embedded applications, neither extreme is satisfactory. The motivation for this project was to quantify this tradeoff on real FPGA hardware and to establish the infrastructure for a third option: UDP++, a lightweight reliability layer that augments UDP with sequence numbers, session identification, checksum validation, detection of missing or corrupted packets, and NAK based selective retransmission [3], aiming to improve reliability while preserving much of UDP's performance.

3.2 Alternative Approaches

Two principal implementation strategies were considered for realizing Gigabit class transport protocols on FPGA platforms:

Software protocol stack on an embedded processor (PS based). In this approach, a hard processor (e.g., the ARM Cortex A9 in the Zynq 7000 Processing System) executes a lightweight TCP/IP stack such as lwIP, with the Ethernet MAC realized as a hard peripheral. Development effort is generally lower than for a custom HDL implementation and the full protocol suite is available. However, each packet requires protocol processing by the software stack running on the processor, making throughput and latency dependent on processor performance, memory bandwidth, interrupt handling, and software stack configuration.

Hardware offload in Programmable Logic (PL based). Here the packet data path, parsing, header processing, and framing, is implemented directly in FPGA fabric and coupled to

memory via DMA, removing the CPU from packet parsing and byte wise protocol processing [3]. The processor is used mainly for configuration, initialization, DMA control, and interrupt handling. This approach allows packet processing to be performed directly in hardware, but requires greater design and verification effort.

3.3 Selected Approach

The project used a PS based software implementation for TCP and a PL based hardware data path for UDP and UDP++, with control software running on a MicroBlaze processor implemented in the PL. This enabled a comparison between the different implementations and provided the hardware data path required for integrating UDP++.

The **TCP baseline** was implemented as a PS based lwIP application on a custom Zynq 7000 board. Implementing a complete TCP protocol stack in HDL would require substantial design and verification effort [1]. With the mature lwIP stack, after systematic tuning (64 KB TCP windows, enlarged pbuf pools, fixed 1000 Mbps link speed, and relocation of code and buffers to external DDR3), the TCP server achieved approximately 940 Mbps.

The **UDP baseline** was implemented as a PL based design on the VCU118 (UltraScale+) platform [6]. This decision was driven directly by the bottleneck identified in the PS centric architecture: in a processor mediated data path, sustained per packet CPU involvement constrains achievable throughput and introduces jitter, precisely the costs UDP is chosen to avoid. Migrating the UDP data path into Programmable Logic, built around a custom UDP Offload Engine (UOE), a MicroBlaze control processor, and AXI DMA transfers into on chip buffering, removed the processor from packet parsing and byte wise data processing, while retaining it for DMA control and interrupt handling, and provided a hardware data path suitable for integrating UDP++.

This dual platform structure carries a further architectural rationale: UDP++ is defined as an AXI Stream HDL module, and only a PL resident data path allows it to be inserted directly between the AXI DMA and the UOE in both the transmit and receive paths. The VCU118 baseline therefore serves as the direct integration substrate for the reliability layer.

3.4 Report Organization

The report is organized as follows. Chapter 4 presents the general system architecture and the shared test and measurement environment. Chapters 5, 6, and 7 are each dedicated to one protocol, the TCP performance server, the UDP echo server, and the UDP++ reliability layer

respectively, and cover each protocol from specification through implementation to measured results. Chapter 8 consolidates and compares the results of the three transports. Chapter 9 documents problems and resolutions. Chapter 10 evaluates the final results against the initial project requirements and proposes future work. References and appendices complete the report.

4. General System Architecture and Test Environment

The delivered system comprises two independent Gigabit Ethernet server implementations, each realizing a distinct architectural strategy, together with a unified host side measurement environment. The specification below reflects the system as built and tested.

The preliminary design (pre-2026-142) [5] specified a single hardware platform, an Avnet PicoZed Zynq 7000 SOM [7], hosting both the TCP and UDP implementations, with data buffered in external DDR3. During implementation, the TCP and UDP implementations were developed on different hardware platforms. These changes, documented per procedure requirements [4], were made to address technical requirements encountered during implementation and did not change the project goals.

4.1 System Level Architecture

The project comprises two complete, independently verified server systems sharing a common host side measurement environment. Figure 4.1 presents the top level architecture of both platforms.

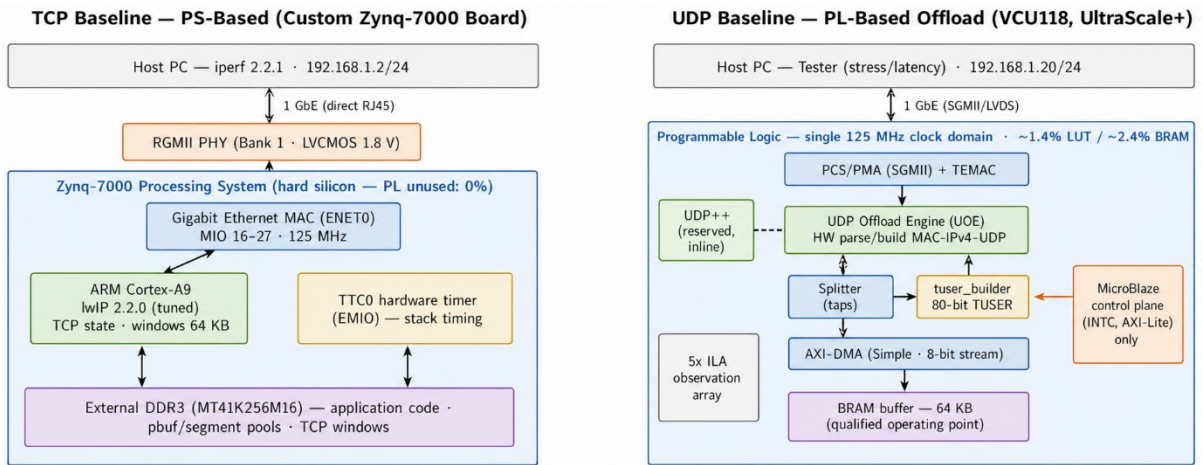


Figure 4.1: PS based TCP architecture (left) and PL based UDP architecture (right). TCP runs through ARM/lwIP, while UDP packet data is processed in hardware, with MicroBlaze limited to DMA control.

TCP data path (PS based). An inbound frame enters through the on board RGMII PHY and is received by the hard Gigabit Ethernet MAC (ENET0) embedded in the Zynq 7000 Processing System. The MAC transfers frames via its built in DMA into packet buffers (pbufs) residing in external DDR3. The lwIP stack, executing on the ARM Cortex A9, performs IP/TCP protocol processing, connection state, sequencing, acknowledgment generation, and window management, entirely in software, and the performance application echoes the payload back through the same path. Every packet therefore traverses the processor. The engineering effort in

this design concentrates on removing every avoidable bottleneck around that fixed constraint (Section 5).

UDP data path (PL based). An inbound frame enters through the SGMII/LVDS interface into the PCS/PMA core, which recovers the parallel GMII stream for the Tri Mode Ethernet MAC (TEMAC) [9]. The TEMAC strips the Ethernet framing and streams the packet, IP and UDP headers included, over AXI4 Stream into the UDP Offload Engine (UOE). The UOE parses and validates the IPv4/UDP headers in hardware and forwards only the clean payload to the AXI DMA, which writes it into on chip BRAM and raises s2mm_introut. The MicroBlaze, which never touches packet data, services the interrupt and commands the DMA to stream the payload back. On transmit, the Splitter taps the DMA stream, the tuser_builder supplies the 80 bit TUSER routing vector (addresses, ports, length), and the UOE reencapsulates the payload with fresh UDP/IPv4/MAC headers toward the TEMAC and PCS/PMA. The per packet path is hardware timed at 125 MHz. The processor is used mainly for configuration, DMA control and interrupt handling.

This architectural contrast, a processor inside the packet data path versus a processor removed from it, is the central engineering theme of the project and the direct foundation for the UDP++ layer (Section 7), which inserts directly on the AXI Stream path between the AXI DMA and the UOE.

4.2 Test and Measurement Environment

Parameter	Specification
Connectivity	Direct RJ45 Cat 5e link, host NIC at static IP (192.168.1.2 / 192.168.1.20 per setup), subnet 255.255.255.0
Host side test software	iperf 2.2.1 for TCP throughput measurements. Adviser supplied Tester application for UDP and UDP++ throughput, packet integrity, latency, and retransmission tests
Protocol verification	Wireshark was used for packet inspection. A basic Python socket script was used for an initial functional test, transmitting a 1,024 byte payload filled with the value 0x42 and verifying that the data path operated correctly. All throughput, latency, packet loss, and detailed packet monitoring tests were

	performed using the adviser supplied Tester application.
Programming/debug	Xilinx JTAG (bitstream + application), Vivado Hardware Manager (ILA), serial terminal

Table 4.1: Test and measurement environment.

All acceptance tests employed a deliberately minimal two node topology: the device under test (DUT) connected to a host measurement PC over a direct Cat 5e RJ45 link, with static addressing on a private /24 subnet (TCP setup: board 192.168.1.10, host 192.168.1.2. UDP setup: board 192.168.1.10, host 192.168.1.20). The direct link eliminates switch queuing and unrelated network traffic, reducing external network influences on the measurements. However, host side software, the network interface, and the operating system may still affect the measured throughput, latency, and packet loss.

The measurement instruments were layered so that each result is independently corroborated:

Layer	Instrument	Purpose
End to end performance	iperf 2.2.1 (TCP), adviser supplied Tester application (UDP: latency/RTT, stress)	Throughput, loss, and stability quantification
Protocol correctness	Wireshark on the host NIC	Independent, DUT external validation of frame structure, headers, and payload integrity
Initial functional verification	Python socket script, 1,024 B of constant 0x42 ('B')	Verification of basic packet transmission, echo operation, and payload integrity (Section 6.9)
On chip signal monitoring	Five core ILA array (Section 6.8), Vivado Hardware Manager	Cycle accurate observation inside the FPGA at every data path boundary
Console/status	Serial terminal (TCP), JTAG UART via MDM (UDP)	Link speed confirmation, application state

Table 4.2: Layered measurement instrumentation.

Test sequencing followed a fixed discipline for every configuration: (1) Program FPGA enabled in the Vitis run configuration, guaranteeing the bitstream on silicon matches the software about to run. (2) link establishment confirmed (terminal message "Link speed for phy address 0: 1000" on the TCP platform. PHY lock ILA capture on the UDP platform). (3) functional verification with deterministic traffic. (4) performance and stress measurement.

The deterministic payload was chosen for a specific reason: the Tester generates random payloads, correct for stress statistics, but opaque for structural verification, since header bytes and payload bytes are visually indistinguishable in a capture. The 0x42 pattern makes the header/payload seam self evident in both Wireshark hex dumps and ILA waveforms, so structural verification (Section 6.9) preceded and derisked the stress campaign (Sections 6.10 and 6.11).

Definition of latency. Throughout this chapter, latency means per packet round trip time (RTT). The Tester emits a burst of 1,000 probe packets at a fixed 1,000 μ s interval. Each probe is individually timestamped at transmission and matched to its own echo on return, and the live chart plots one RTT sample per probe. The summary statistics (p50, p99) are therefore percentiles of the per packet RTT population, not the completion time of the burst as a whole. The alternative aggregate definition ("time until all packets return") was considered and rejected: it scales with burst length, conflates delay with throughput, and hides the distribution tails that matter to a latency sensitive application. Per packet RTT is the quantity a real application experiences for each datagram, it matches the instrument's native semantics, and its percentiles are robust to isolated outliers.

Plausibility for a standalone point to point bench. With the DUT connected to the host by a single cable, no switches, routers, or competing traffic, wire and hardware contributions to RTT are nearly negligible: serialization of a ~ 1 kB frame in both directions accounts for roughly 17 μ s, and the FPGA's own turnaround is microsecond scale (hardware timestamped interpacket pacing of ~ 10 μ s is demonstrated in Section 6.10). The measured RTT of several hundred microseconds is therefore dominated almost entirely by the host: the Windows socket stack, NIC interrupt moderation, and the measurement process's own scheduling. A p50 in the 400 to 500 μ s range with millisecond class p99 spikes is exactly the signature expected of a PC to FPGA direct link, and the deterministic PL data path contributes no additional jitter discernible at the available measurement resolution, a conclusion corroborated at signal level by the ILA captures of Section 6.9.

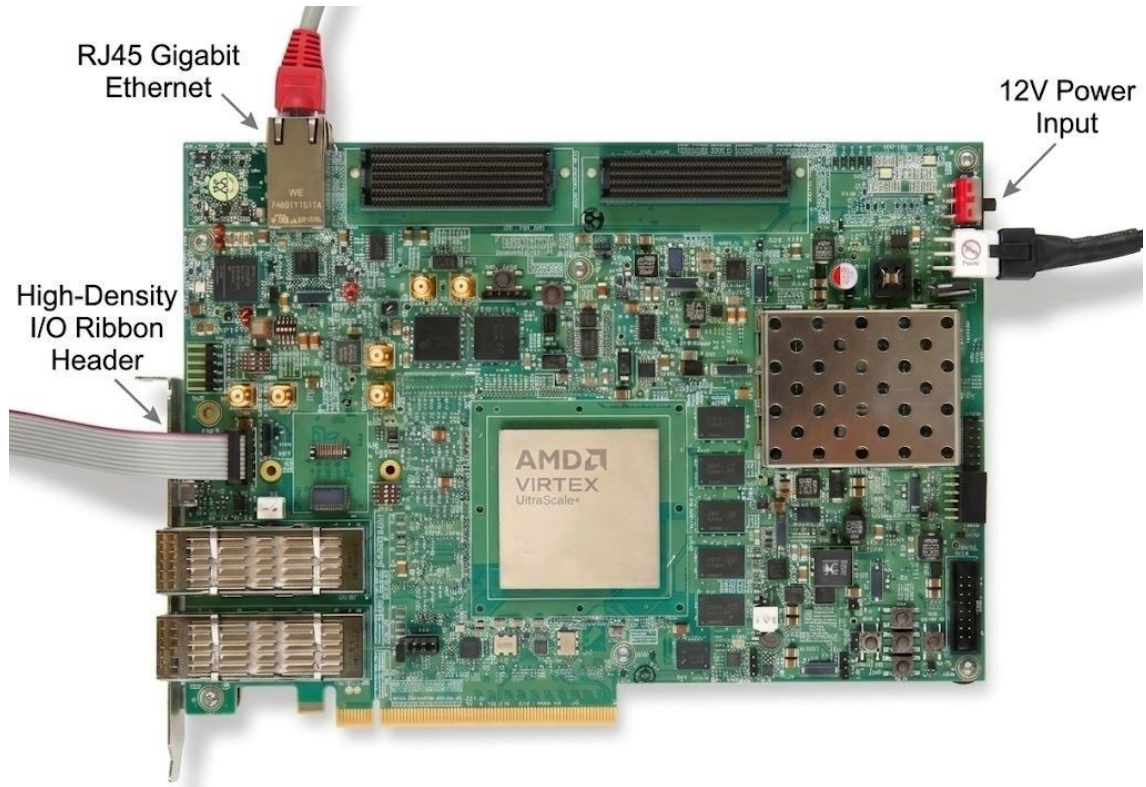


Figure 4.2(a): Physical test bench, VCU118 with 12 V supply, JTAG programming cable, and direct RJ45 link.

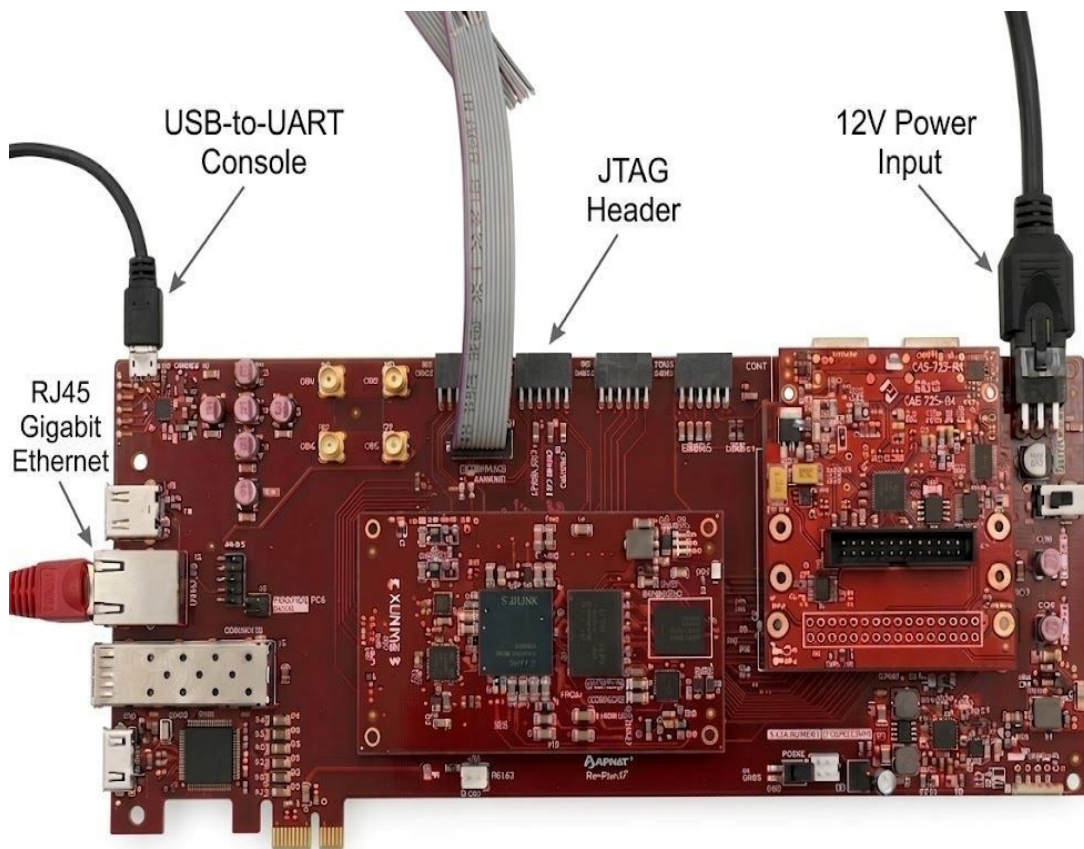


Figure 4.2(b): TCP platform bench connection, board, programming cable, and Ethernet link to the host.

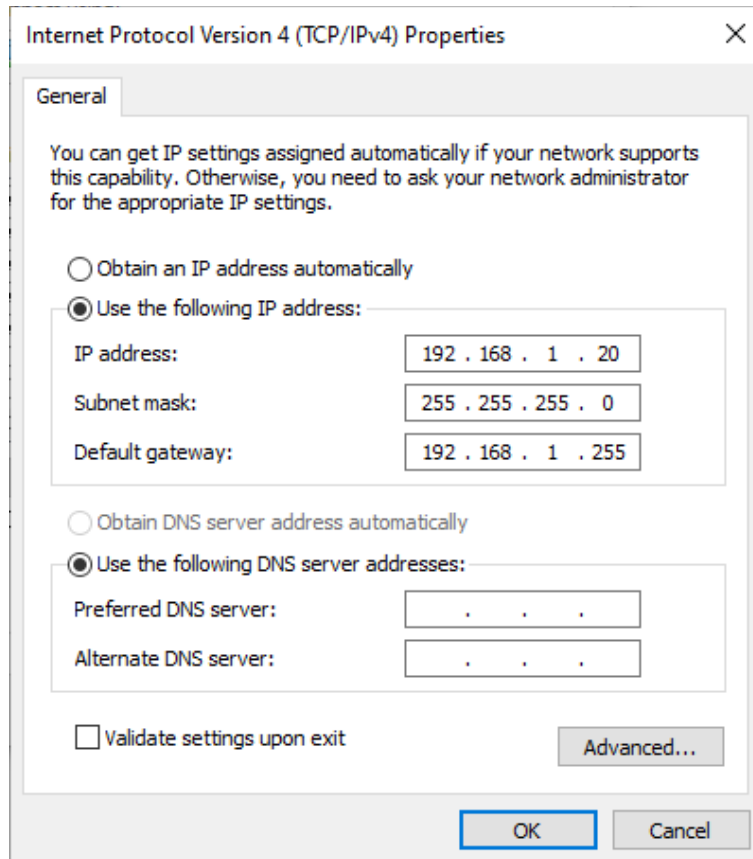


Figure 4.2(c): Host static IP configuration (192.168.1.20/24).

4.3 Toolchain Version Update

Vivado/Vitis were updated from the planned 2023.2+ to 2024.2 (Vitis Unified IDE), aligning the project with the current industrial environment. The udp_pp IP core, packaged earlier under Vivado 2023.2, was integrated unchanged into the 2024.2 flow (Appendix 12.1.4). The lwIP library version in use is 2.2.0 (lwip220 BSP).

4.4 Specification Compliance Summary

The headline throughput requirement (≥ 900 Mbps on a 1 GbE link) was met by the TCP implementation (~ 940 Mbps) and approached by the UDP implementation (~ 850 Mbps average under stress, $\sim 94\%$ of target). The gap analysis and its engineering implications are discussed in Section 10.1. All remaining preliminary specifications, 1 GbE physical layer, AXI Stream internal interconnect, DMA based memory access, reproducible build flow, and the measurement framework, were implemented as planned.

5. TCP Performance Server

5.1 Final System Specification

Parameter	Specification
Platform	Custom Zynq 7000 board, XC7Z020-CLG400-1 [7]
Processing	ARM Cortex A9 (Processing System), standalone (bare metal)
Network interface	ENET0 hard MAC, RGMII PHY via MIO pins 16–27, Bank 1 at LVCMOS 1.8 V
Link speed	Fixed 1000 Mbps (ENET0 reference clock 125 MHz)
External memory	DDR3, MT41K256M16 RE-125, application code and network buffers mapped to DDR
Protocol stack	lwIP 2.2.0, tcp_wnd and tcp_snd_buf = 65,535 B, memp_n_tcp_seg = memp_n_pbuf = 1024, DHCP disabled, static IP 192.168.1.10
Timer	TTC0 via EMIO, dedicated hardware clock for lwIP timing
Application footprint	3,361,542 B (~3.36 MB), necessitating DDR execution
Toolchain	Vivado / Vitis Unified IDE 2024.2
Measured performance	~940 Mbps sustained (iperf 2.2.1, 2 MB window, 300 s continuous run)

Table 5.1: Final specification, TCP performance server.

5.2 Changes from Preliminary Design

The TCP server was realized on a custom XC7Z020 board rather than the PicoZed SOM. The migration required board specific hardware adaptation with no architectural change: MIO pin remapping (ENET0 to MIO 16–27, dual UARTs), I/O bank voltage assignment (Bank 1 lowered to LVCMOS 1.8 V, mandatory for the RGMII PHY interface, whose violation produces a "PHY setup failure" and total loss of communication), DDR controller configuration for the on board MT41K256M16 RE-125 device, and a dedicated XDC constraint binding the PHY reset to pin V13.

The memory architecture decision, however, was quantitative and platform independent. The compiled TCP application measured 3,361,542 B against ~600 KB of available on chip BRAM, a $5.6\times$ deficit. Sustaining 1,000 Mbps TCP further requires 64 KB windows and 1024 entry pbuf/segment pools resident in memory simultaneously. Execution from external DDR3 was therefore a feasibility condition. This confirmed the preliminary design's DDR based buffering assumption and quantified its necessity.

5.3 Hardware Platform Configuration

The hardware design consists of a single ZYNQ7 Processing System block. No custom logic occupies the PL. The design effort lies in configuring the PS hard peripherals to match the custom board precisely [1], [7]. The physical platform, a PicoZed class Zynq 7000 module on its carrier, is photographed in Figure 4.2(b) alongside the full test bench.

MIO mapping and I/O bank voltages. The board routes its peripherals through the Multiplexed I/O (MIO) fabric: Quad SPI flash on MIO 1–6, Ethernet (ENET0) on MIO 16–27, and two UARTs on MIO 42–43 and 48–49. The critical configuration is electrical: ENET0's pins reside in MIO Bank 1, and the RGMII standard used by the on board PHY mandates 1.8 V signaling. Bank 1 was therefore assigned LVCMOS 1.8 V (Bank 0 remaining at 3.3 V for the QSPI). Operating Bank 1 at the 3.3 V default produces a "PHY setup failure" at initialization and total loss of link, a failure mode encountered and resolved during bring up (Section 9).

Dedicated network timer. TTC0 was enabled and routed through EMIO, providing lwIP with a hardware timer independent of the CPU's private timers. This isolates protocol timing (retransmission timeouts, delayed ACK timers) from application induced timing contention.

Gigabit clocking. The ENET0 peripheral clock was configured for 1000 Mbps operation, yielding the required 125 MHz reference. This is the PS side counterpart of the PL clocking chain built explicitly on the VCU118.

DDR controller. The DDR configuration was bound to the exact on board device, MT41K256M16 RE-125. Remaining timing parameters derive automatically from the part selection. An XDC constraint file completes the board adaptation: PHY reset bound to package pin V13 (LVCMOS18), without which the PHY may remain latched in reset at power up, and bitstream compression enabled to shorten programming cycles.

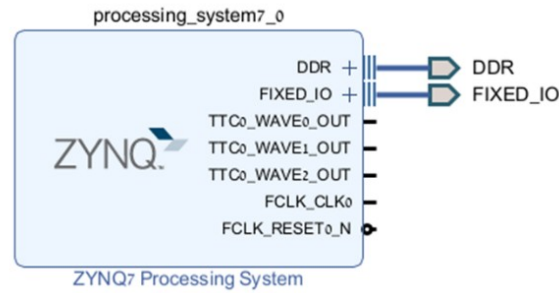


Figure 5.1: Final PS only block design (Zynq TCP platform)

Post implementation reports show zero PL resource consumption, all instantiated functions (CPU, MAC, DDR controller, timers) are hard silicon. This both minimizes power and leaves the entire fabric free for future hardware accelerators, UDP++ included.

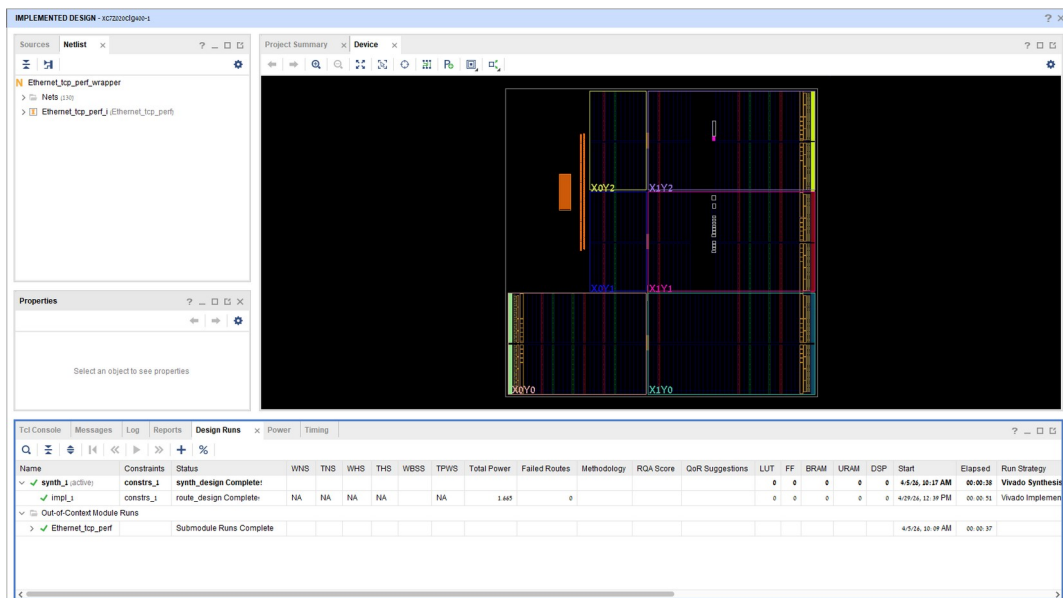


Figure 5.2(a): TCP platform utilization, 0% PL usage

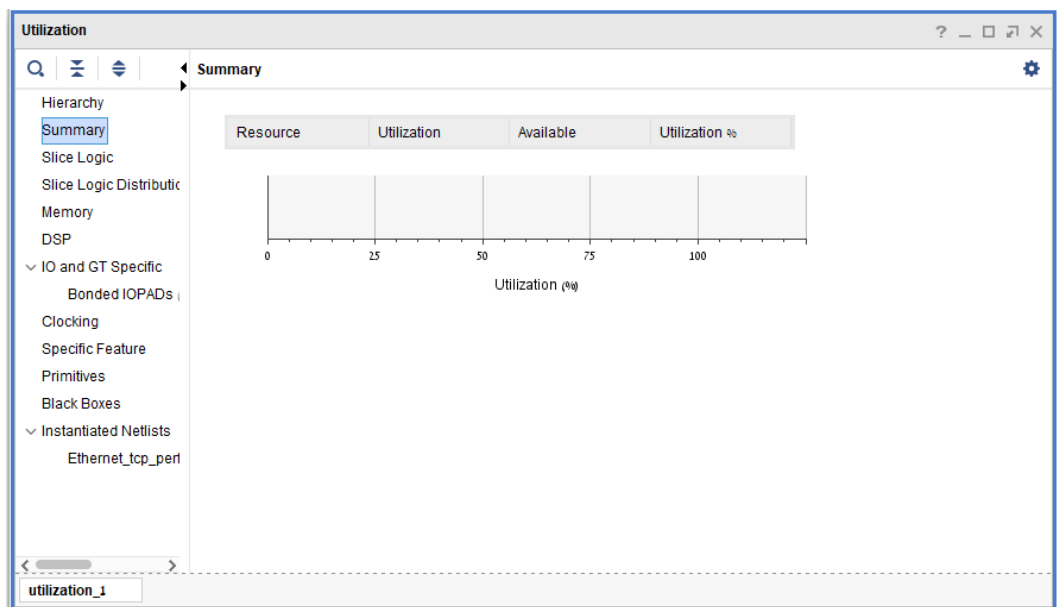


Figure 5.2(b): TCP platform floorplan, all functions in hard silicon

5.4 Memory Architecture: DDR as a Feasibility Condition

Placement of code and buffers was resolved quantitatively. The compiled application measures 3,361,542 B (~3.36 MB, per the Vitis build log), against ~600 KB of total on chip BRAM, a 5.6× deficit before a single packet buffer is allocated. Sustaining 1,000 Mbps TCP additionally demands 64 KB send/receive windows and 1024 entry pbuf and segment pools held in memory concurrently. Mapping the executable and the lwIP heap to external DDR3 was therefore a feasibility condition. The tradeoff is the higher access latency of external memory, acceptable in a windowed, throughput oriented protocol, and the exact inverse of the conclusion reached for the UDP path (Section 6.6), where transient single packet buffering favors low latency BRAM.

5.5 lwIP Stack Selection and Performance Tuning

lwIP 2.2.0 (lwip220 BSP) was selected over an HDL TCP implementation on complexity grounds: a hardware TCP core requires per connection state machines, retransmission buffering, and RTT timer logic whose verification effort is disproportionate for a baseline whose purpose is measurement [1]. The stack was, however, systematically tuned. Defaults do not approach line rate:

- **tcp_wnd = tcp_snd_buf = 65,535 B**, the maximum standard (nonscaled) TCP window. Window size bounds in flight data per RTT. Submaximal windows starve a Gigabit link.
- **memp_n_tcp_seg = memp_n_pbuf = 1024**, enlarged segment/buffer pools, sized so that pool exhaustion never throttles the 940 Mbps stream.
- **temac_phy_link_speed = CONFIG_LINKSPEED1000** (replacing auto detect), fixing the negotiated speed eliminates an observed failure class in which line noise during auto negotiation left the link down or degraded (Section 9).
- **DHCP disabled**, static addressing (board 192.168.1.10, host 192.168.1.2/24), removing address acquisition latency and nondeterminism from a two node test network.
- **xiltimer bound to ps7_ttc_0**, attaching the stack's sleep/timeout services to the dedicated TTC0 hardware timer.

The application layer is the Vitis lwIP TCP Perf Server template, unmodified except for static addressing, deliberately so, keeping the baseline a measurement of platform and stack rather than of custom application code.

5.6 TCP Performance Results

The TCP acceptance test ran iperf 2.2.1 against the Zynq server with the command: `iperf-2.2.1-win64.exe -c 192.168.1.10 -i 5 -t 300 -w 2M`.

Parameter selection was deliberate: **-t 300** subjects the system to five minutes of continuous saturation, a stability test that exposes buffer pool exhaustion, timer drift, or memory leak failure modes that short runs mask. **-w 2M** sets a 2 MB client window ensuring the host never becomes the bottleneck, so the measurement bounds the DUT. **-i 5** provides 60 interval samples across the run for rate stability inspection.

Result: ~940 Mbps sustained across the full 300 second run, with the server reporting "TCP test passed Successfully" at completion. Two aspects of this figure warrant analysis:

- **Proximity to theoretical maximum.** After Ethernet, IP, and TCP header overhead, the practical goodput ceiling of a 1 GbE link is ~941–949 Mbps. The measured 940 Mbps therefore represents effective saturation, the software stack, DDR based buffering, and 64 KB window tuning (Section 5.5) left no meaningful throughput unclaimed.
- **Stability as the primary finding.** Rate consistency across 60 consecutive 5 second intervals demonstrates that the 1024 entry pbuf/segment pools and the TTC0 based timing remained in equilibrium under sustained load, the specific failure modes the tuning targeted did not manifest. The final iperf summary line (Figure 5.3(a)) confirms a total transfer of 32.8 GBytes across the 300.02 second run at an overall average of 939 Mbps, with no degradation toward the end of the test.

The specification target of ≥ 900 Mbps was met with margin.

```

Administrator: Command Prompt
Microsoft Windows [Version 10.0.19044.2728]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>d:

D:\>iperf-2.2.1-win64.exe -c 192.168.1.10 -i 5 -t 300 -w 2M
-----
Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 2.00 MByte
-----
[ 1] local 192.168.1.2 port 58315 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.00-5.00 sec  561 MBytes    942 Mbits/sec
[ 1] 5.00-10.00 sec 560 MBytes    940 Mbits/sec
[ 1] 10.00-15.00 sec 560 MBytes    939 Mbits/sec
[ 1] 15.00-20.00 sec 560 MBytes    940 Mbits/sec
[ 1] 20.00-25.00 sec 560 MBytes    940 Mbits/sec
[ 1] 25.00-30.00 sec 560 MBytes    940 Mbits/sec
[ 1] 30.00-35.00 sec 560 MBytes    940 Mbits/sec
[ 1] 35.00-40.00 sec 560 MBytes    940 Mbits/sec
[ 1] 40.00-45.00 sec 560 MBytes    940 Mbits/sec
[ 1] 45.00-50.00 sec 560 MBytes    939 Mbits/sec
[ 1] 50.00-55.00 sec 560 MBytes    940 Mbits/sec
[ 1] 55.00-60.00 sec 560 MBytes    940 Mbits/sec
[ 1] 60.00-65.00 sec 560 MBytes    940 Mbits/sec
[ 1] 65.00-70.00 sec 560 MBytes    940 Mbits/sec
[ 1] 70.00-75.00 sec 560 MBytes    939 Mbits/sec
[ 1] 75.00-80.00 sec 560 MBytes    940 Mbits/sec
[ 1] 80.00-85.00 sec 560 MBytes    940 Mbits/sec
[ 1] 85.00-90.00 sec 560 MBytes    940 Mbits/sec
[ 1] 90.00-95.00 sec 560 MBytes    940 Mbits/sec
[ 1] 95.00-100.00 sec 560 MBytes    940 Mbits/sec
[ 1] 100.00-105.00 sec 560 MBytes    940 Mbits/sec
[ 1] 105.00-110.00 sec 560 MBytes    940 Mbits/sec
[ 1] 110.00-115.00 sec 560 MBytes    940 Mbits/sec
[ 1] 115.00-120.00 sec 560 MBytes    939 Mbits/sec
[ 1] 120.00-125.00 sec 560 MBytes    940 Mbits/sec
[ 1] 125.00-130.00 sec 560 MBytes    940 Mbits/sec
[ 1] 130.00-135.00 sec 559 MBytes    937 Mbits/sec
[ 1] 135.00-140.00 sec 560 MBytes    939 Mbits/sec
[ 1] 140.00-145.00 sec 560 MBytes    940 Mbits/sec
[ 1] 145.00-150.00 sec 560 MBytes    939 Mbits/sec
[ 1] 150.00-155.00 sec 560 MBytes    940 Mbits/sec
[ 1] 155.00-160.00 sec 560 MBytes    940 Mbits/sec
[ 1] 160.00-165.00 sec 560 MBytes    940 Mbits/sec
[ 1] 165.00-170.00 sec 560 MBytes    940 Mbits/sec
[ 1] 170.00-175.00 sec 560 MBytes    940 Mbits/sec
[ 1] 180.00-185.00 sec 560 MBytes    940 Mbits/sec
[ 1] 185.00-190.00 sec 560 MBytes    940 Mbits/sec
[ 1] 190.00-195.00 sec 560 MBytes    939 Mbits/sec
[ 1] 195.00-200.00 sec 560 MBytes    940 Mbits/sec
[ 1] 200.00-205.00 sec 560 MBytes    940 Mbits/sec
[ 1] 205.00-210.00 sec 559 MBytes    937 Mbits/sec
[ 1] 210.00-215.00 sec 560 MBytes    940 Mbits/sec
[ 1] 215.00-220.00 sec 560 MBytes    939 Mbits/sec
[ 1] 220.00-225.00 sec 560 MBytes    939 Mbits/sec
[ 1] 225.00-230.00 sec 560 MBytes    939 Mbits/sec
[ 1] 230.00-235.00 sec 560 MBytes    940 Mbits/sec
[ 1] 235.00-240.00 sec 560 MBytes    939 Mbits/sec
[ 1] 240.00-245.00 sec 560 MBytes    939 Mbits/sec
[ 1] 245.00-250.00 sec 560 MBytes    940 Mbits/sec
[ 1] 250.00-255.00 sec 560 MBytes    940 Mbits/sec
[ 1] 255.00-260.00 sec 559 MBytes    938 Mbits/sec
[ 1] 260.00-265.00 sec 560 MBytes    940 Mbits/sec
[ 1] 265.00-270.00 sec 560 MBytes    940 Mbits/sec
[ 1] 270.00-275.00 sec 560 MBytes    940 Mbits/sec
[ 1] 275.00-280.00 sec 560 MBytes    940 Mbits/sec
[ 1] 280.00-285.00 sec 560 MBytes    940 Mbits/sec
[ 1] 285.00-290.00 sec 560 MBytes    940 Mbits/sec
[ 1] 290.00-295.00 sec 560 MBytes    939 Mbits/sec
[ 1] 295.00-300.00 sec 560 MBytes    939 Mbits/sec
[ 1] 0.00-300.02 sec 32.8 GBytes    939 Mbits/sec

```

Figure 5.3(a): iperf 300 s run, initial intervals and final summary showing 939 Mbps average

```

COM10 - Tera Term VT
File Edit Setup Control Window Help

-----lwIP RAW Mode TCP Server Application-----
Using default Speed from design
link speed for phy address 0: 1000
Configuring default IP 192.168.1.10
Board IP:      192.168.1.10
Netmask :      255.255.255.0
Gateway :      192.168.1.1

TCP server listening on port 5001
On Host: Run $iperf -c 192.168.1.10 -i 5 -t 300 -w 2M

[ 1] local 192.168.1.10 port 5001 connected with 192.168.1.2 port 58315
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0- 0.0 sec  32.8 GBytes    0.00 bits/sec
TCP test passed Successfully

```

Figure 5.3(b): Server side confirmation, "TCP test passed Successfully"

6. UDP Echo Server

6.1 Final System Specification

Parameter	Specification
Platform	AMD/Xilinx VCU118 Evaluation Board, Virtex UltraScale+ [6], with networking FMC module [8]
Processing	MicroBlaze soft processor, 128 KB local memory (LMB BRAM), standalone
Data path clock	125 MHz system wide, derived from the on board 300 MHz differential oscillator via Clocking Wizard
Network interface	Tri Mode Ethernet MAC (internal PHY mode) + 1G PCS/PMA core, SGMII over LVDS, 125 MHz LVDS reference clock, auto negotiation enabled
UDP processing	Custom UDP Offload Engine (UOE), VHDL 2008, packaged as AXI4 Stream IP, hardware parsing and generation of MAC/IPv4/UDP headers
Packet routing metadata	Custom tuser_builder IP: five AXI4 Lite registers assembling an 80 bit TUSER vector (IP addresses, ports, length), latched at start of packet
DMA	AXI DMA, Simple mode (Scatter Gather disabled), 14 bit buffer length register (≤ 16 KB transfers), 8 bit stream width matched to MAC byte rate
Packet buffering	AXI BRAM Controller, a 64 KB BRAM buffer used for packet storage between the DMA receive and transmit paths
Debug infrastructure	Five ILA cores: PHY status (Native, 16 bit vector probe) and four AXI Stream monitors (RX/TX at MAC and UOE boundaries, TX_UDP with 80 bit TUSER probe), JTAG UART via MDM for console I/O
Resource utilization	$\sim 1.36\%$ LUTs, $\sim 2.43\%$ BRAM of VCU118 capacity, logic concentrated in SLR1

Measured performance	~850 Mbps average (TX 860 / RX 851 Mbps, ~1% loss under continuous stress), 1000/1000 packets received in latency testing at 64 KB buffering
----------------------	--

Table 6.1: Final specification, UDP echo server.

6.2 Changes from Preliminary Design

The preliminary design placed the UDP baseline on the same Zynq platform, However, according to the platform specification, the Ethernet interface available to the Programmable Logic supported only 100 Mbps and therefore could not meet the project target of at least 900 Mbps over a 1 G link. The UDP and UDP++ implementations were consequently moved to the VCU118 [6], which provides a 1 Gb capable network interface accessible from the PL. On the VCU118, the packet data path was implemented in Programmable Logic, while the MicroBlaze handled system configuration, DMA control, and interrupt handling.

- **Processing:** hard ARM Cortex A9 replaced by a MicroBlaze soft core (128 KB local memory), confined strictly to the control plane, DMA configuration, programming the tuser_builder registers used to generate the 80 bit TUSER metadata, and interrupt service.
- **Network stack:** the lwIP software stack replaced by the hardware UOE, performing UDP/IPv4/MAC parsing and framing at line rate. The processor does not perform byte wise packet parsing or payload processing; it is used mainly for DMA control, transfer management, and interrupt handling.
- **PHY architecture:** hard RGMII PHY replaced by a PL instantiated MAC + PCS/PMA pair over SGMII/LVDS [8], granting full design control of the 125 MHz clocking chain.

Two additional quantitative consequences followed. First, unlike the TCP implementation, which used DDR memory, the UDP echo implementation used a 64 KB on chip BRAM buffer for temporary packet storage between the DMA receive and transmit paths. In a separate latency test, all 1,000 probe packets were received, while approximately 1% packet loss was measured during the sustained stress test described in Section 6.10. Second, the migration enabled the UDP++ AXI4 Stream HDL module to be inserted directly between the AXI DMA and the UOE. The reliability processing was therefore implemented in hardware, while the MicroBlaze remained responsible for configuration, DMA control, and interrupt handling. Implementing

the same structure in the original PS mediated topology would have required a substantial architectural redesign.

6.3 MicroBlaze Subsystem and Memory Allocation

The control processor is a MicroBlaze soft core instantiated in fabric [2], [6]. Two MicroBlaze subsystem settings were configured:

- **Local memory = 128 KB.** The MicroBlaze executes the application from LMB-attached on chip BRAM. The local memory was configured to 128 KB to accommodate the application, approximately 23 KB of code and data according to the final linker report, together with the stack, heap, and driver structures. This allocation keeps the application in on chip memory and provides sufficient space for execution.
- **Interrupt controller enabled (AXI INTC).** The echo application is interrupt driven: DMA completion interrupts (mm2s_introut, s2mm_introut) are concatenated into the INTC and delivered to the processor, eliminating the need to poll for DMA completion.

The linker script (lscript.ld) maps all sections, .text, .data, .bss, stack, heap, explicitly to lmb_bram, so the application code and allocated data reside in the MicroBlaze local memory.

6.4 Clock Architecture

Gigabit Ethernet fixes the byte rate clock: $1,000 \text{ Mbps} \div 8 \text{ bits} = 125 \text{ MHz}$. The design derives this frequency from the VCU118's 300 MHz differential system oscillator through a Clocking Wizard (differential input, clk_out1 = 125 MHz) and, critically, distributes it as the single clock for the entire data path: MicroBlaze interconnect, TEMAC (including its AXI4 Lite management port, retimed from the 100 MHz default to 125 MHz), DMA, BRAM controller, UOE, and all ILA cores.

Three synchronization mechanisms complete the architecture:

- **Internal clocking of the MAC:** the PCS/PMA's internally generated clk125_out drives the TEMAC's gtx_clk directly [9], guaranteeing phase alignment between the PHY facing and logic facing domains without an additional external oscillator.
- **Locked gated resets:** the Clocking Wizard's locked output feeds the dcm_locked input of all three Processor System Reset blocks, holding every subsystem in reset until the MMCM has stabilized, eliminating the power up race between logic and clock.

- **Reset polarity management:** the PCS/PMA and the UOE require active high reset while the AXI infrastructure is active low. Utility Vector Logic NOT gates perform the inversions, and the UOE's reset interfaces were explicitly declared ACTIVE_HIGH during IP packaging so Vivado would not insert spurious inverters.

A board configuration override proved essential: associating the PCS/PMA with the VCU118 board preset silently selects a 625 MHz SGMII reference. The core was reconfigured (DIFFCLK = Custom, LVDS reference = 125 MHz) to match the design's clock tree, without which the PHY never achieves lock (Section 9).

6.5 UDP Offload Engine: IP Packaging and Integration

The UOE is the system's central protocol block: a VHDL core performing hardware based parsing, validation, and generation of MAC/IPv4/UDP headers. Integrating it as a first class Vivado IP, rather than loose source files, required a disciplined packaging flow (Create and Package New IP):

- **Language standard enforcement.** The core employs VHDL 2008 constructs. The file type property must be fixed inside the IP metadata or it silently reverts on repackaging. This was enforced via the Tcl sequence: `set_property file_type {VHDL 2008} [get_files *.vhd]`. `Ipx::merge_project_changes files [ipx::current_core]`. `Ipx::save_core [ipx::current_core]`.
- **Interface inference.** Discrete port groups were bound to standard bus abstractions, `axis_rtl` for the MAC side, UDP side, and RAW data channels. `aximm_rtl` for the AXI4 Lite control interface (4 KB address window in the MicroBlaze memory map), and single bit inference for clocks and resets, with reset POLARITY explicitly parameterized ACTIVE_HIGH (`G_ACTIVE_RST = 1`) to match the core's internal logic.
- **System integration.** `CLK_UOE` was driven by the 125 MHz system clock, while `CLK_TX` and `CLK_RX` were driven by the TEMAC-generated `tx_mac_aclk` and `rx_mac_aclk` clocks, respectively. A dedicated Processor System Reset block supplies its active high `peripheral_reset`. `M_MAC_TX` / `S_MAC_RX` connect to the TEMAC's AXI Stream transmit/receive ports, and an AND gate reduction of the PHY lock/status signals drives `PHY_LAYER_RDY`, holding the engine idle until the physical link is fully established. Unused external/RAW TVALID inputs are tied to constant zero, a deliberate noise prevention measure flagged (benignly) by design validation.

With packaging complete, the UOE was integrated into the block design, with its interfaces connected to the corresponding 125 MHz system and MAC side clocks. The figure below shows the final VCU118 block design as implemented, validated, and taken through bitstream generation. The zoom in views that follow decompose it into its functional tiers: the first pair presents the control subsystem and the data path core. The second, the AXI peripheral tier and the MAC/PCS PMA physical tier. The third, the PHY status aggregation logic that derives the PHY_LAYER_RDY qualifier.

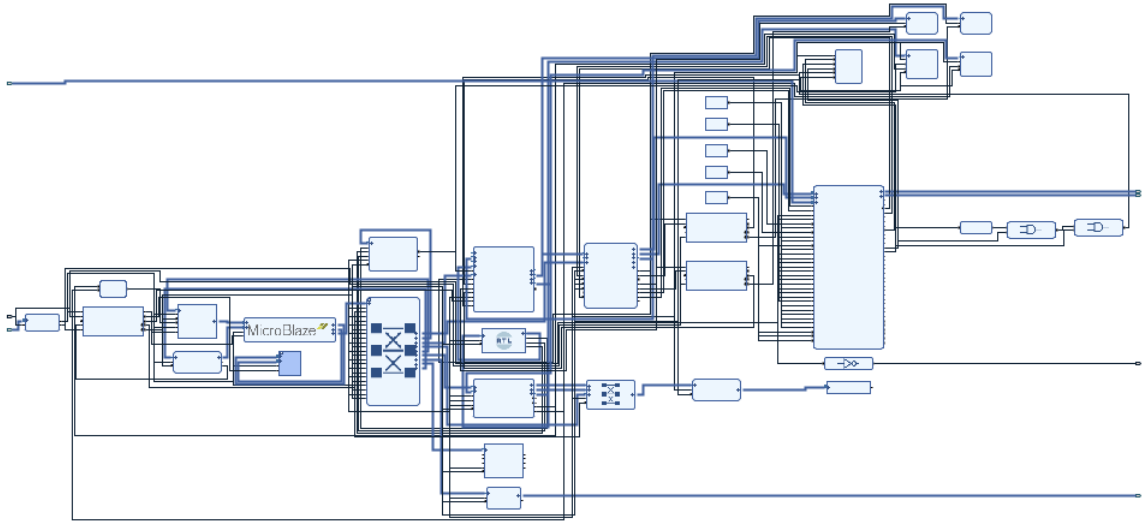


Figure 6.1: Final VCU118 block design (zoom out)

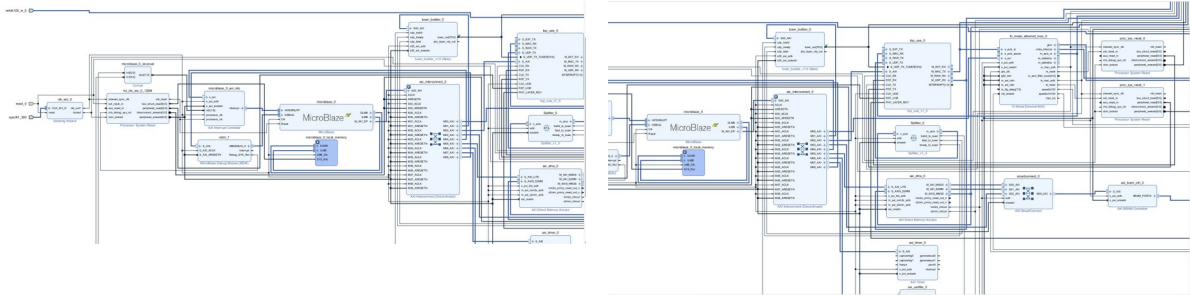


Figure 6.2: Zoom in: control subsystem (left) and data path core (right)

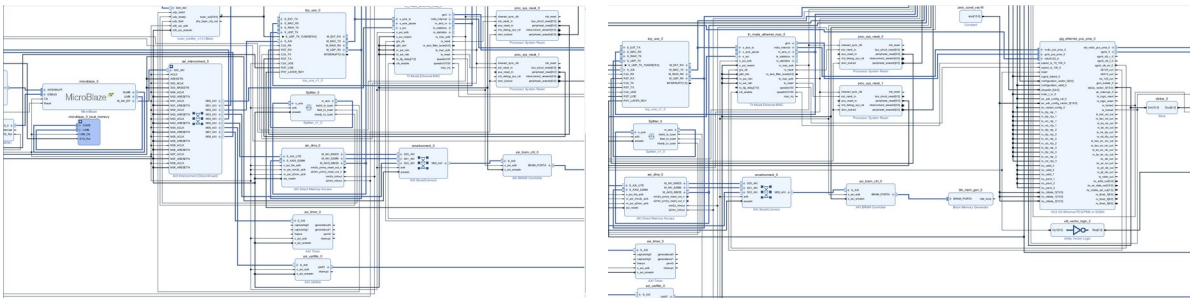


Figure 6.3: Zoom in: AXI peripheral tier (left) and MAC/PCS PMA physical tier (right)

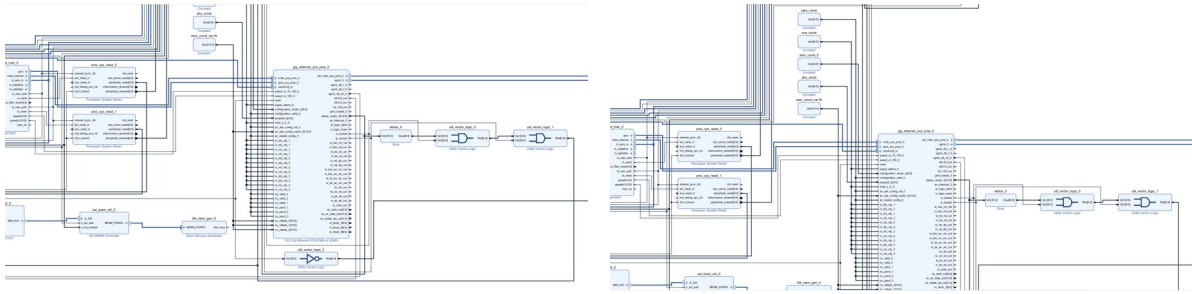


Figure 6.4: Zoom in: PHY status aggregation logic deriving *PHY_LAYER_RDY*

Each module carries a single, well bounded responsibility. The MicroBlaze is used mainly for system control: it configures the PHY over MDIO, programs the tuser_builder routing registers, controls the DMA, and services completion interrupts. It does not perform byte wise packet parsing or payload processing in the normal echo path. The AXI INTC funnels the DMA interrupts into the processor. The MDM provides the JTAG UART console without consuming a physical UART. The Clocking Wizard derives the 125 MHz byte rate clock from the board's 300 MHz differential input, and the Processor System Reset tree distributes locked gated, polarity correct resets. On the network side, the PCS/PMA core terminates the SGMII/LVDS serial link and recovers the parallel GMII byte stream. The TEMAC performs Ethernet framing, preamble and FCS handling. The UOE parses and generates the MAC, IPv4, and UDP headers directly in hardware. The AXI DMA converts between the streaming and memory mapped domains, with SmartConnect arbitrating its two masters into the AXI BRAM Controller and the 64 KB packet buffer. The tuser_builder assembles the 80 bit TUSER routing metadata, and the Splitter replicates stream control signals so that metadata latching is race free. Five ILA cores complete the design as its built in observability infrastructure.

Anatomy of a received packet (RX). A frame arrives on the SGMII/LVDS pair and is deserialized by the PCS/PMA into a GMII byte stream at 125 MHz. The TEMAC validates the FCS, strips the Ethernet preamble and framing, and forwards the frame as an 8 bit AXI Stream to the UOE. The UOE receive engine parses the MAC, IPv4, and UDP headers on the fly, validates addressing and checksums, strips the entire 42 byte header stack, and emits a payload only stream whose TLAST marks the exact payload boundary. The AXI DMA S2MM channel writes the payload through SmartConnect into BRAM and raises s2mm_introut. The interrupt handler marks the frame ready for the echo loop. From wire to memory, no processor instruction touches the data path.

Anatomy of a transmitted packet (TX). Software rearms the MM2S channel with the buffer address and length. The DMA reads the payload from BRAM and streams it toward the UOE through the Splitter, whose replicated TVALID/TLAST/TREADY taps inform the

tuser_builder of packet boundaries. At start of packet the tuser_builder latches the 80 bit TUSER vector, destination MAC, IP, ports, and length, and holds it stable for the frame's duration. The UOE transmit engine consumes payload and metadata, generates fresh MAC/IPv4/UDP headers with computed checksums, and hands the completed frame to the TEMAC, which appends Ethernet framing and FCS before the PCS/PMA serializes it back onto the wire. The mm2s_introut interrupt closes the transaction and the handler rearms reception.

6.6 AXI DMA and BRAM Buffering Architecture

The AXI DMA is the design's protocol bridge between its two on chip interconnect paradigms: the packet path is streaming (AXI4 Stream, address less, unidirectional, backpressure governed by TVALID/TREADY), while memory is address mapped (AXI4 Memory Mapped). The DMA translates between them in both directions, S2MM (stream to memory) writes received payloads into the buffer, MM2S (memory to stream) reads them back for transmission, without consuming a single MicroBlaze cycle per byte.

Three configuration decisions define the DMA's role in this design:

- **Simple mode (Scatter Gather disabled).** SG mode chains buffer descriptors in memory for autonomous multibuffer transfers, capability the echo application does not need, since each transaction is a single contiguous packet. Disabling SG collapses the driver model to direct register programming (source/destination address, length, start), radically simplifying the bare metal software and shortening the debug loop: every DMA transaction is fully described by a handful of register writes visible in the debugger.
- **14 bit buffer length register.** Sized for transfers of up to approximately 16 KB, well above the 1,024-byte test payload and the maximum packet size used in this project.
- **8 bit stream width.** Matched exactly to the TEMAC's byte per cycle rate at 125 MHz. Width conversion logic between the DMA and the MAC side stream is thereby eliminated. The entire AXI4 Stream packet path moves one byte per clock.

Memory side, the DMA's M_AXI_MM2S and M_AXI_S2MM master ports converge, together with the processor's data port, on an AXI SmartConnect (three slave inputs), which arbitrates access into an AXI BRAM Controller (single interface mode, narrow bursts disabled). The choice of on chip BRAM over the board's DDR4 is the mirror image of the TCP decision in Section 5.4: an echo path holds each packet only transiently, so capacity is secondary to access

latency, and BRAM offers single digit cycle access with no controller queuing. The buffer is provisioned at 64 KB, sized to absorb traffic microbursts with ample margin. The ~1% residual loss observed under sustained stress originates in the echo service rate rather than in buffer capacity (Section 6.10).

Interrupt plumbing closes the control loop: mm2s_introut and s2mm_introut are concatenated (Concat block) into the AXI INTC, so the MicroBlaze is notified precisely at transfer completion, allowing the MicroBlaze to manage each transfer through DMA configuration and completion interrupts.

Design validation during this stage exposed two integration findings worth recording: the connection automation left the DMA's m_axi_mm2s_aclk / m_axi_s2mm_aclk inputs unconnected (resolved by manual tie in to the 125 MHz tree), and S_AXIS_S2MM reported as dangling, an expected condition at that stage, as the receive stream connects to the UOE only later in the flow.

6.7 Custom IP Development: tuser_builder and Stream Splitter

Two in house IP blocks complete the transmit path. Their necessity stems from a fundamental property of AXI4 Stream: the payload channel carries no addressing. The UOE constructs outgoing UDP/IPv4/MAC headers in hardware, but the destination IP address, source and destination UDP ports, and payload length must arrive on the stream's TUSER sideband, synchronized to the packet they describe. Bridging processor owned configuration into a hardware timed sideband is precisely the gap these two blocks fill.

tuser_builder, software to hardware metadata bridge. The block was generated from the Vitis AXI4 Lite slave peripheral template (32 bit, 4 registers) and extended in VHDL to five registers: the address decoder was widened (OPT_MEM_ADDR_BITS = 2), slv_reg4 added with full write/read decode paths and reset initialization, giving the MicroBlaze register level access to all routing fields plus PHY readiness status. On the hardware side, the relevant register fields are assembled into a single 80-bit tuser_reg_out vector containing the destination IP address, source and destination UDP ports, and payload length in fixed field positions. The vector is wired directly to the UOE's S_UDP_TX_TUSER input.

The block's critical feature is its **latching mechanism**: on detection of start of packet on the transmit stream, the current register contents are frozen for the duration of that packet. Without it, a processor write landing mid transmission would corrupt the in flight header, a race between

the 125 MHz data path and asynchronous software timing that cannot be excluded by convention alone. The latch makes this class of intermittent corruption impossible by construction.

Splitter, control signal replication for synchronization and observability. The transmit stream's control signals (TVALID, TLAST, TREADY) are point to point by protocol definition, yet the tuser_builder needs sight of them to time its latch. The Splitter, a compact VHDL module inserted between the DMA's MM2S output and the UOE's payload input, passes the full AXI Stream through unmodified while exporting replicated copies of the handshake signals on dedicated to_tuser outputs, three of which drive the tuser_builder's packet boundary detection. The same tap points serve the ILA instrumentation (Section 6.8), giving the debug infrastructure a protocol clean observation point on the DMA → UOE boundary without loading the functional path. The block is purely combinational on the data route, zero added latency, zero protocol alteration.

The two blocks implement the design's division of labor: the processor writes the routing information once per flow, the hardware supplies the metadata for every transmitted packet, and the 80 bit interface between them is race free and fully observable.

6.8 Software Flow and On Chip Debug Infrastructure

Vitis application architecture. The software layer follows the platform/application model of Vitis Unified IDE 2024.2: the exported design_1_wrapper.xsa generates a standalone (bare metal) platform on microblaze_0, over which the UDP_app application component is built from the Empty Application template with a single source file, UDP_Main.c (Appendix 12.1).

The application's control flow is deliberately minimal: (1) **Initialization**, configure MAC/IP identity (static 192.168.1.x addressing, matching the host's 192.168.1.20/24), program the tuser_builder registers over AXI4 Lite, initialize the DMA (Simple mode), and register the two DMA interrupt handlers with the INTC. (2) **In loopback mode, an interrupt assisted polling loop:** s2mm_introut signals a received payload in BRAM. The s2mm_introut interrupt sets the RxDone flag after reception completes. The software polls this flag, copies the received payload from the RX BRAM region to the TX BRAM region, and then starts the MM2S transfer. The mm2s_introut interrupt sets the TxDone flag when the DMA transfer completes.

The software uses DMA completion interrupts together with polling. The S2MM interrupt service routine acknowledges the interrupt and sets the RxDone flag. The main loop polls this

flag, copies the received payload from the RX BRAM region to the TX BRAM region using processor read and write operations, and then starts the MM2S transfer. The MM2S interrupt service routine acknowledges the interrupt and sets the TxDone flag, which is also polled before the program continues. Therefore, the DMA transfers data between the streaming interfaces and BRAM, while the MicroBlaze performs transfer control and copies the payload between the RX and TX buffer regions.

All peripherals are reached through the Address Editor assigned base addresses, partitioned into two networks: the data/control space (DMA, Timer, UOE, BRAM controller, UART, tuser_builder, each with a dedicated base address and range) and the instruction space (LMB local memory), with overlap free mapping verified before bitstream generation.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/axi_dma_0					
/axi_dma_0/Data_MM2S (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	64K	0xC000_FFFF
/axi_dma_0/Data_S2MM (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	64K	0xC000_FFFF
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	64K	0xC000_FFFF
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0x41E0_0000	64K	0x41E0_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/mdm_1/S_AXI	S_AXI	Reg	0x4140_0000	4K	0x4140_0FFF
/microblaze_0_axi_intc/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	128K	0x1_FFFF
/top_uoe_0/S_AXI	S_AXI	reg0	0x2_0000	16K	0x2_3FFF
/tri_mode_ethernet_mac_0/s_axi	s_axi	Reg	0x44A0_0000	64K	0x44A0_FFFF
/tuser_builder_0/S00_AXI	S00_AXI	S00_AXI_reg	0x44A1_0000	64K	0x44A1_FFFF
Network 1					
/microblaze_0					
/microblaze_0/Instruction (32 address bits : 4G)					

Figure 6.5: Vivado Address Editor, final address map

Console I/O was routed through the MicroBlaze Debug Module's JTAG UART: rather than routing printf through the AXI UARTLite, the MicroBlaze Debug Module (MDM) was reconfigured with Enable JTAG UART, exposing an S_AXI port (Interconnect expanded to nine masters, M08_AXI) and redirecting the BSP's stdin/stdout to mdm_1. Diagnostics thus flow over the existing JTAG cable at high speed, one less physical dependency during lab bring up, and the UARTLite remains available as an independent channel.

PHY register configuration over MDIO (TI DP83867). The physical layer transceiver on the networking module is a Texas Instruments DP83867 Gigabit Ethernet PHY [8], [10], and its

operating mode is not left to hardware defaults: the Vitis application configures it explicitly at initialization through the MDIO management channel. MDIO transactions are issued through the TEMAC's management interface [9] (MDC/MDIO pair routed to the PHY), giving the software read/write access to the PHY's register file at its strapped MDIO address. Two register classes are involved:

- **Standard register configuration.** The application writes the value 0x1140 to the Basic Mode Control Register, BMCR (address 0x0000), to configure the PHY operating mode [10]. After the configuration writes are completed, the software repeatedly reads register 0x05 and waits until bit 14 is asserted before continuing, confirming that the link is ready (Section 6.5).
- **PHY specific register configuration.** The application writes configuration values directly to PHY register addresses 0xD3, 0x10, 0x14, and 0x32 through the TEMAC MDIO interface. These writes configure the SGMII clock and operating mode, enable SGMII operation, apply the required CFG2 setting, and disable the RGMII interface [10].

The configuration sequence explicitly programs the PHY for the required SGMII and Gigabit operating mode before UDP traffic begins. After the configuration writes, the software polls register 0x05 until bit 14 is asserted, while the `phy_enable` ILA is used to observe the PHY lock and readiness signals (Section 6.5).

ILA instrumentation array: design for debuggability. High speed FPGA logic is opaque at runtime. Its internal signals are physically unobservable without dedicated instrumentation. The design therefore treats observability as an architectural requirement, embedding five Integrated Logic Analyzer cores that partition the debug problem along its natural boundaries, one state question and four data flow questions:

ILA	Monitoring type	Observation point	Engineering question answered
<code>phy_enable</code>	Native, 5 probes (probe0 = 16 bit)	PCS/PMA status_vector, AND gate outputs,	Was the Ethernet link established and were the PHY

		PHY_LAYER_RDY, peripheral_aresetn	lock and reset readiness signals asserted before any traffic?
RX_MAC	Interface (AXI Stream aware)	TEMAC m_axis_rx → UOE	Does the raw frame (headers included) enter the system intact?
RX_UDP	MIX	UOE → DMA S_AXIS_S2MM	Did the UOE strip headers correctly, leaving clean payload + TLAST?
TX_UDP	MIX, probe0 = 80 bit	DMA MM2S → UOE, + TUSER vector	Is the payload leaving memory with the correct routing metadata attached?
TX_MAC	Interface	UOE M_MAC_TX → TEMAC s_axis_tx	Was the outgoing packet re encapsulated correctly before the PHY?

Table 6.2: The five core ILA observation array.

The type assignments follow the nature of each signal class. **Native** probing suits the PHY monitor because link status is represented by discrete state signals, lock bits, a 16 bit status vector, a reset line, with no bus protocol to decode. Individual signal capture with a wide vector probe is the correct abstraction. **Interface** monitoring (System ILA) suits the two MAC boundary points because AXI Stream is a protocol: the analyzer decodes TVALID/TREADY/TLAST/TDATA relationships automatically and renders transactions, packets, rather than raw wires. The two UOE boundary monitors use **MIX** mode because their observation is inherently hybrid: an AXI4 Stream channel together with additional native probes, including the 80 bit TUSER vector on the TX side.

All five cores clock from the same 125 MHz tree as the monitored logic, mandatory for cycle accurate capture, and probe4 of the PHY monitor samples peripheral_aresetn, making reset release itself an observable event.

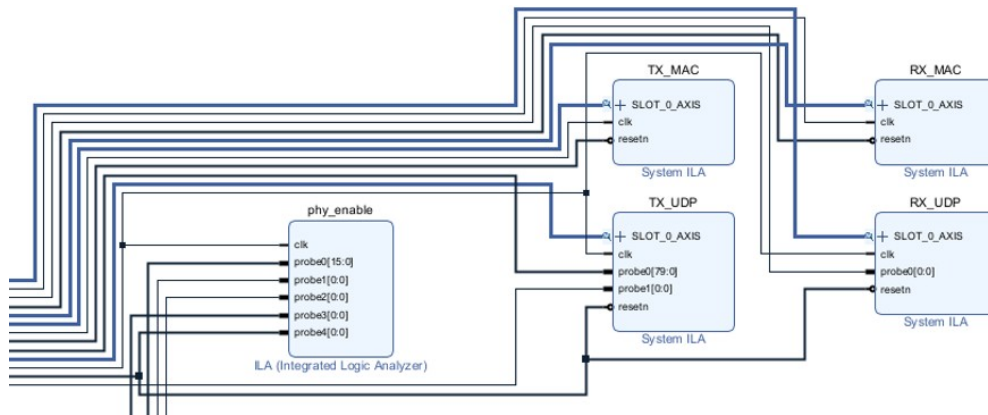


Figure 6.6: ILA in the block design

Operationally, the array is trigger driven from the Vivado Hardware Manager: the bitstream is programmed together with its .ltx probe file, TVALID rising edge (or interrupt line high) conditions arm the capture, and the system waits in "Waiting for Trigger" until real traffic arrives. The full verification campaign built on this array, the four station packet walkthrough with the deterministic 0x42 payload, interrupt timing capture, and behavior under Tester generated stress, constitutes measured evidence and is reported in Section 6.9.

6.9 Data Path Verification: The 0x42 Packet Walkthrough

Before any performance claim, the UDP data path was verified end to end at signal level: a single 1,024 byte packet of constant 0x42 was transmitted from the Python script to port 5001 and traced through four ILA observation stations spanning the complete receive process transmit cycle. Capture was armed on TVALID rising edge, so each station recorded precisely the packet of interest.

Precondition, physical link. The phy_enable ILA confirmed the TX/RX lock signals, the aggregate PHY_LAYER_RDY at logic '1', and the release of reset before traffic. No data path observation is meaningful without this gate.

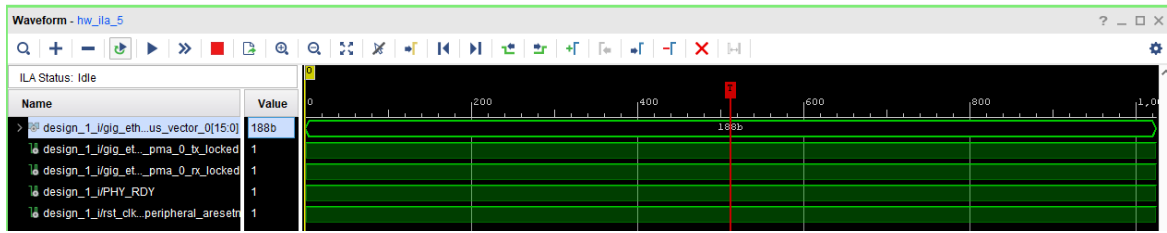


Figure 6.7: PHY precondition, phy_enable ILA

Station 1, RX_MAC (frame ingress). With Ethernet framing stripped by the TEMAC, the capture shows TVALID asserting and the packet's leading header bytes (MAC/IP/UDP headers) streaming on TDATA toward the UOE. A zoomed out capture spans the full transfer window, TVALID held high continuously, terminating with the 0x42 payload region adjacent to the end of frame, establishing that the frame entered the system whole and uninterrupted.

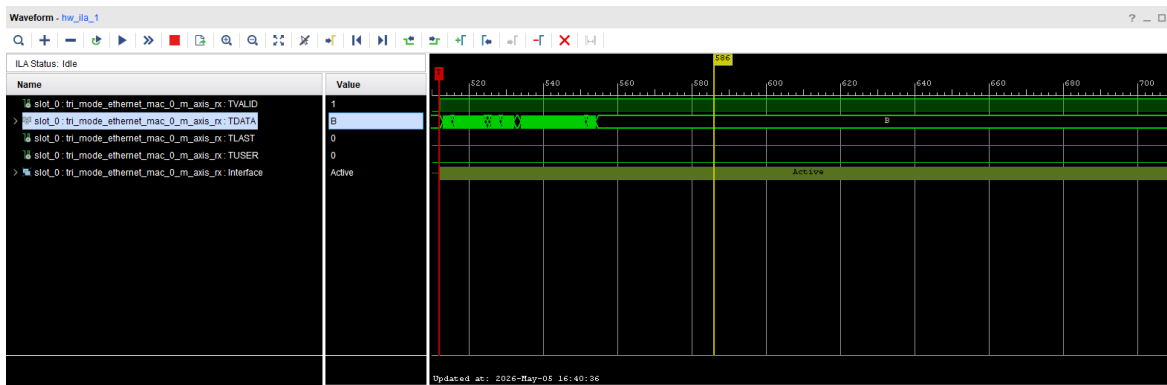


Figure 6.8: Station 1, RX_MAC, frame ingress

Station 2, RX_UDP (header extraction). At the UOE's output toward the DMA, the waveform shows exactly what correct offload looks like: all protocol headers absent, clean 0x42 payload on TDATA, and TLAST asserted coincident with the final payload cycle. The delta between Stations 1 and 2 is the UOE's parsing function, observed directly in hardware.



Figure 6.9: RX header parse at the UOE ingress (two zoom views)

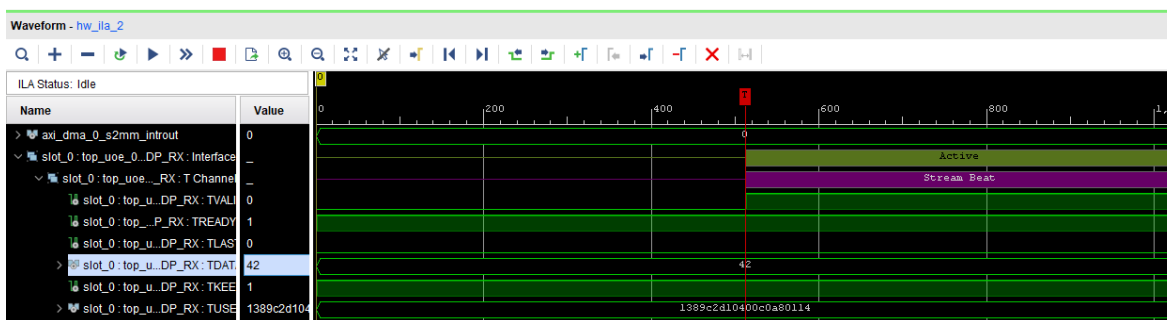


Figure 6.10: Station 2, RX_UDP, only payload

Interrupt integrity (RX). s2mm_introut captured at logic ‘1’ upon DMA write completion, the hardware to software handoff occurring at the correct instant, causing the interrupt service routine to acknowledge the interrupt and set the RxDone flag.

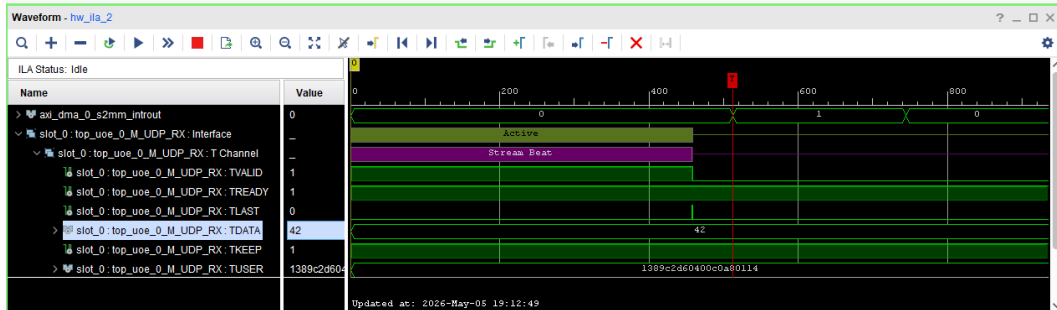


Figure 6.11: RX interrupt, s2mm_introut rising

Station 3, TX_UDP (memory egress). The DMA’s MM2S stream shows TVALID and TREADY asserted while 0x42 payload data is transferred from DMA toward the UOE. The 80-bit TUSER probe remains stable during the transfer and confirms the programmed routing metadata: destination IP address, source and destination UDP ports, and payload length.



Figure 6.12: Station 3, TX_UDP, DMA to UOE AXI4 Stream transfer with TUSER metadata

Station 4, TX_MAC (reencapsulation). At the UOE’s output toward the TEMAC, the packet reappears wrapped in freshly generated UDP/IPv4/MAC headers with source/destination fields exchanged, final data cycle qualified by TLAST. The mm2s_introut interrupt indicated completion of the DMA MM2S transfer..

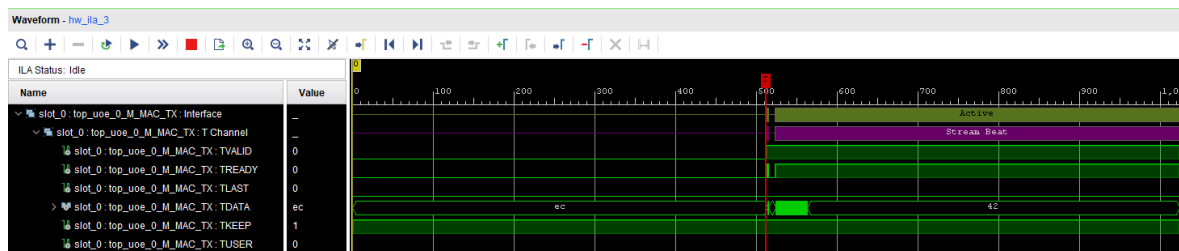


Figure 6.13: Station 4, TX_MAC, header and payload (zoom out)

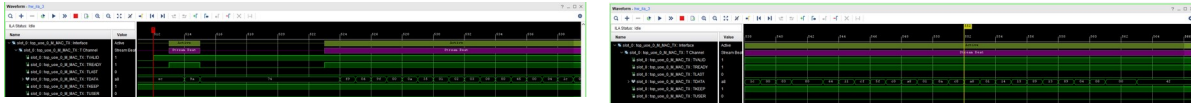


Figure 6.14: TX header generation at the UOE egress (two zoom views)

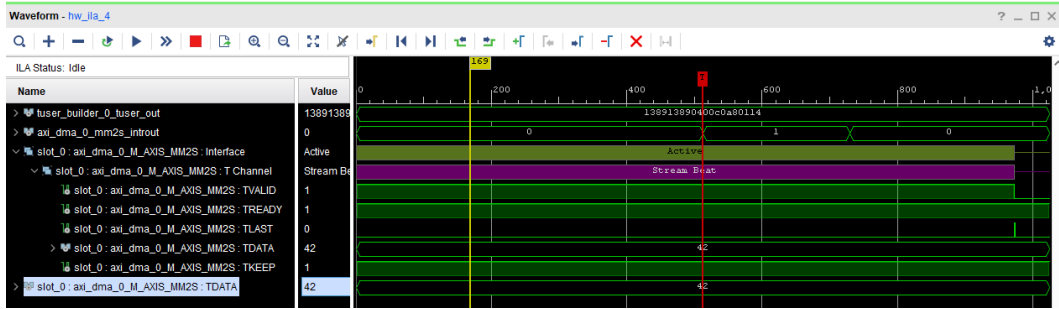


Figure 6.15: TX completion, mm2s_introut rising

Independent corroboration. The Wireshark capture of the returned packet provides DUT external proof: the hex dump shows the standard header block terminating exactly where 1,024 bytes of 0x42 ("BBBB...") begin, byte for byte identical to the transmitted payload, zero corruption across the complete FPGA traversal.

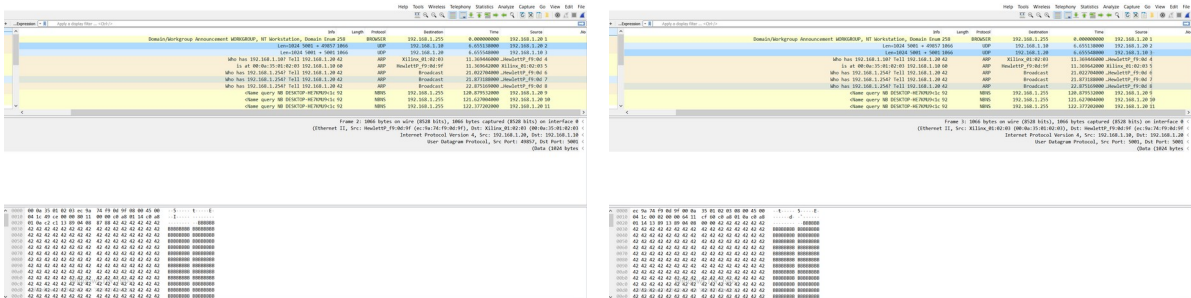


Figure 6.16: Wireshark corroboration, request (left) and echo (right)

Every subsequent quantitative result in this chapter rests on a data path whose correctness was demonstrated station by station.

ILA corroboration under stress. The same instrumentation array was rearmored under Tester load with 1,024 byte packets: RX captures show TVALID held high through entire payloads with clean TLAST framing at packet boundaries. TX captures show TVALID/TREADY jointly asserted in sustained transfer, no backpressure stalls in the observation windows. Interrupt triggered captures confirm s2mm_introut firing per completed reception. One instrumentation limit is noted for methodological completeness: with 1,024 sample ILA depth and 1,024 byte packets, a single capture window cannot contain both packet boundaries, a constraint of the capture depth, with no bearing on the data path.

6.10 UDP Performance Results

UDP performance was measured with the adviser supplied Tester under continuous stress in the 64 KB buffer configuration, preceded by loopback integrity testing at 1,024 B payload.

Integrity under load: among the packets successfully returned during the loopback integrity test, no bit mismatches were detected between the transmitted and echoed payloads, extending the single packet integrity proof of Section 6.9 to sustained traffic. This confirmed payload integrity under sustained traffic, while the separate stress test measured approximately 1% packet loss.

Throughput under continuous stress:

Metric	Measured value
Average throughput	~850 Mbps
TX rate	860 Mbps
RX rate	851 Mbps
Packet loss	~1%, stable

Table 6.3: UDP stress test results (64 KB buffer configuration).

Analysis of these figures:

- ~85% of line rate through a byte wide path. The architectural rate ceiling of the data path is set by its $8 \text{ bit} \times 125 \text{ MHz}$ stream width, 1,000 Mbps raw, before interpacket gaps, header cycles, and DMA transaction turnaround. The 512 byte run sustained approximately 850 Mbps steady state RX throughput and approximately 809 Mbps over the complete run. The UOE performs the protocol processing, the DMA handles the streaming transfers, and the MicroBlaze manages the DMA transactions and copies the payload between the RX and TX BRAM regions.
- The approximately 1% loss remained stable across the run, with TX consistently exceeding RX by about 9 Mbps. This is consistent with a receive-side service-rate limitation, although the measurement does not by itself identify the exact loss mechanism. No payload corruption was detected among successfully returned packets. Because UDP does not retransmit lost packets, the measured loss forms the baseline that the UDP++ layer (Section 7.7) is designed to address.
- **Against specification:** ~850 Mbps approaches but does not meet the ≥ 900 Mbps target (~94%). The gap, its architectural sources, and the identified paths to close it (stream widening, transmit pipelining) are analyzed in Section 10.1.

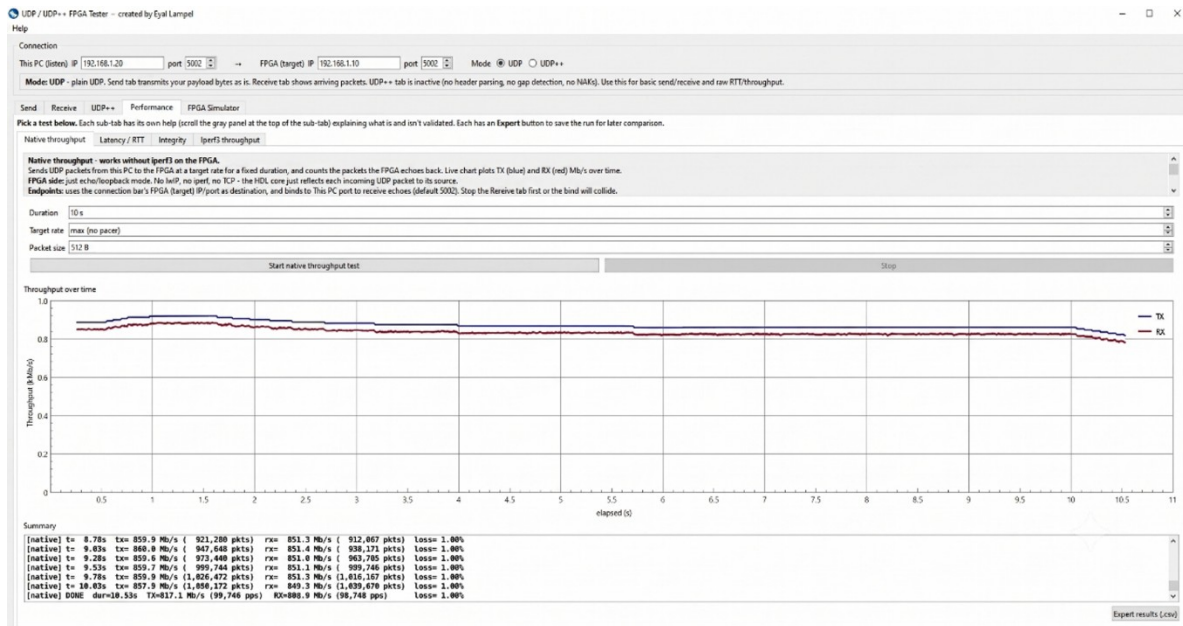


Figure 6.17: UDP baseline native throughput at 512 B: approximately 850 Mbps steady state RX throughput and 809 Mbps whole run average.

Latency (per packet RTT). Under the definition of Section 4.2, the baseline UDP data path, with its provisioned 64 KB buffer, returned all 1,000 probes (512 B payload) with $p_{50} = 413.1 \mu s$ and $p_{99} = 3,391.2 \mu s$. For this measurement the Vitis application ran its loopback branch ("For loop back" in UDP_Main.c): reception is armed, the S2MM completion interrupt sets the RxDone flag, and the software polls this flag, copies the received payload to the TX buffer, and starts the MM2S transfer. The MM2S completion interrupt then sets the TxDone flag, which is polled before reception is rearmed. The RTT chart shows a mostly stable baseline with occasional millisecond-scale spikes. The measurement represents the complete host to FPGA to host round trip and does not isolate the FPGA only processing time. The Wireshark capture confirms the strict request/echo alternation of the probe stream on the wire.

```

579 //For LoopBack
580
581 while(1){
582
583     Status = prepare_receive();
584     if (Status != XST_SUCCESS) {
585         return XST_FAILURE;
586     }
587     Status = wait_for_rx_done();
588     if (Status != XST_SUCCESS) {
589         return XST_FAILURE;
590     }
591
592     Status = prepare_echo_transmit();
593     if (Status != XST_SUCCESS) {
594         return XST_FAILURE;
595     }
596
597     Status = wait_for_tx_done();
598     if (Status != XST_SUCCESS) {
599         return XST_FAILURE;
600     }
601
602     xil_printf("END UDP Protocole...\r\n");
603 }
604
605 while(1){
606 }
607 return 0;
608 }

```

Figure 6.18: Loopback mode code, the for loopback branch in UDP_Main.c (Appendix 12.1.1)

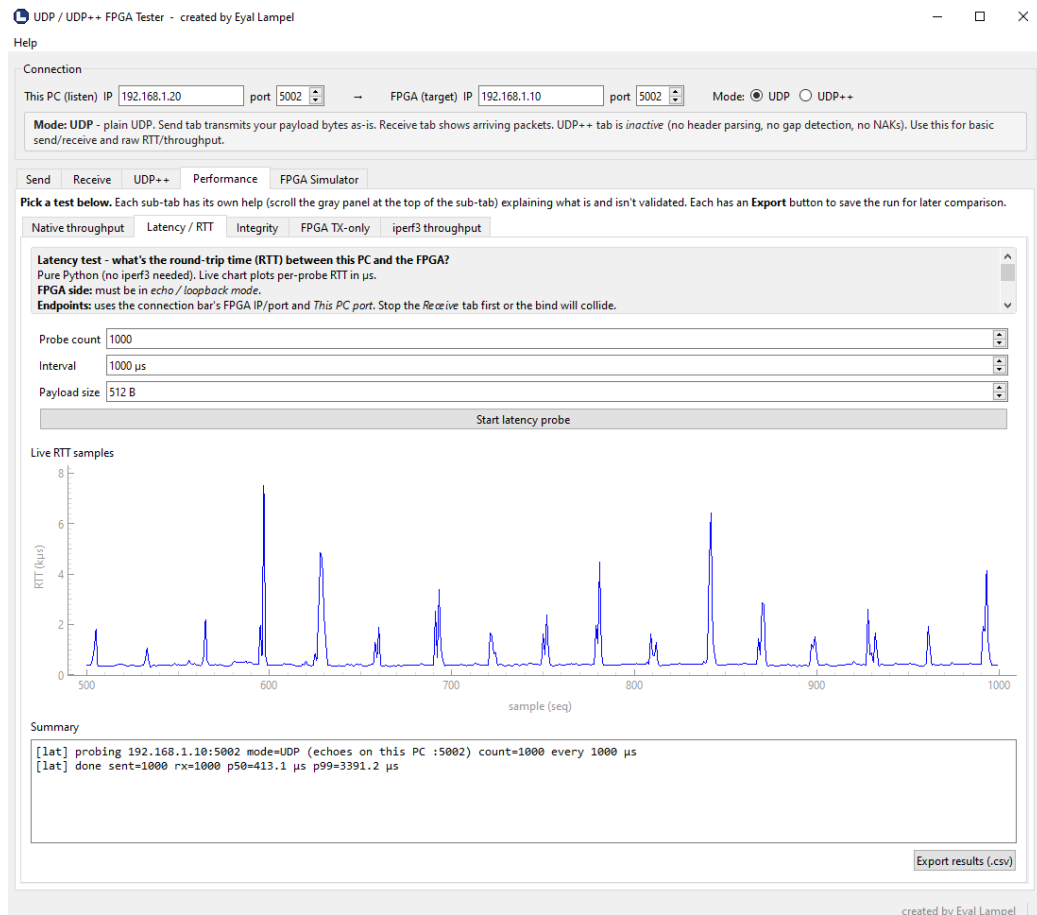


Figure 6.19: UDP baseline latency, per packet RTT (512 B)

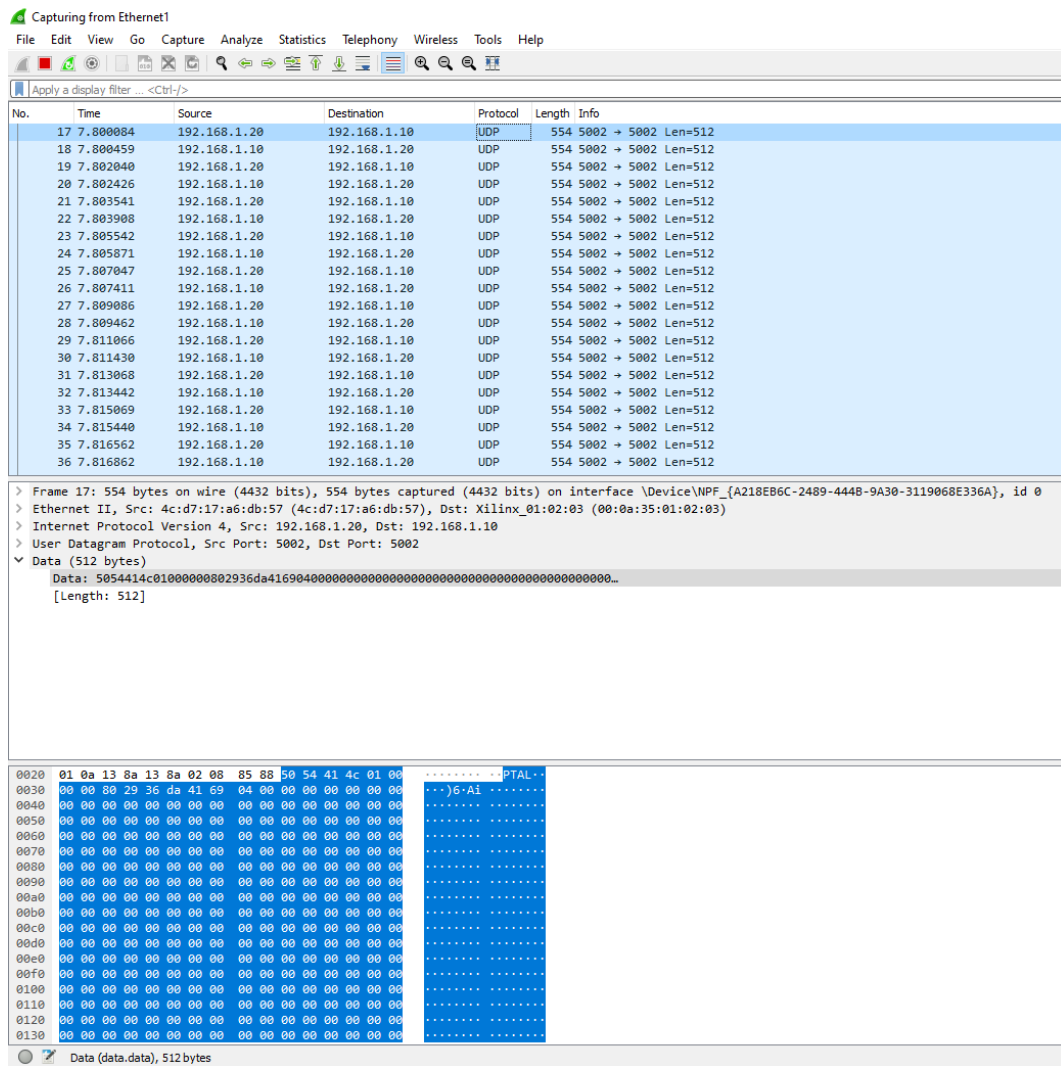


Figure 6.20: Latency run, request/echo alternation on the wire

Transmit-side packet format (FPGA TX-only). In the application's "For TX only" variant, the MicroBlaze transmits packets without an echo path. The first eight payload bytes contain a software incremented sequence number and the lower 32 bits of the AXI Timer counter. Since the timer operates at 125 MHz, one tick represents 8 ns. The remaining payload bytes contain the predefined test pattern. The Wireshark capture confirms the 1,024 byte UDP payload and the placement of the sequence number and timestamp at the beginning of the payload. This capture is presented to verify the transmitted packet format and is not used as a throughput measurement. For the throughput test, the application continuously transmitted 512 byte UDP payloads in an infinite loop, while the adviser supplied Tester measured the resulting TX and RX throughput.

```

374 // for TX only
375 Status = send_packet();
376 if (Status != XST_SUCCESS) {
377     return XST_FAILURE;
378 }

```

Figure 6.21: TX only mode code, the for TX only branch in UDP_Main.c (Appendix 12.1.1)

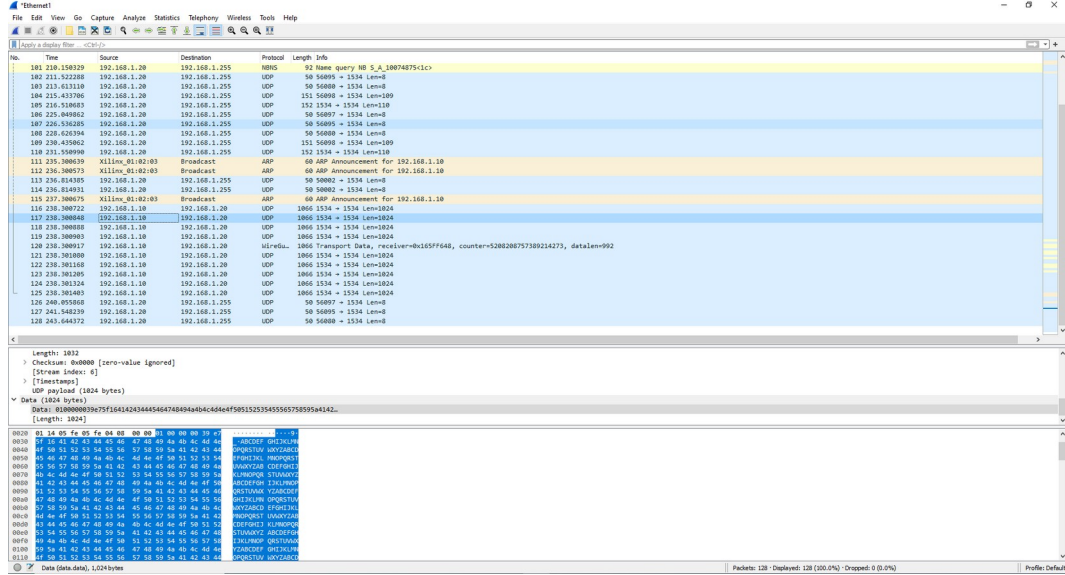


Figure 6.22: TX only stream, sequence number and hardware timestamp

6.11 Resource Utilization Analysis

Post implementation reports for the complete VCU118 design, MicroBlaze subsystem, TEMAC + PCS/PMA, UOE, DMA, SmartConnect, both custom IPs, and all five ILA cores, show:

Resource	Utilization
LUTs	~1.36%
BRAM	~2.43%
Dominant logic consumers	UOE core, AXI SmartConnect
Dominant BRAM consumers	MicroBlaze local memory, packet buffer, MAC infrastructure
Primitive profile	FDRE registers, LUT6 dominated combinational logic
Placement	Concentrated in a single SLR (SLR1)

Table 6.4: VCU118 resource utilization summary.

The analysis yields three engineering observations. First, **headroom**: at ~1.4% logic utilization, the platform readily absorbs the UDP++ layer, including its sequence tracking, NAK generation, and retransmission window buffering (Section 7.7.7), plus substantial future acceleration, without approaching resource pressure. The preliminary design’s stated risk of BRAM exhaustion for reordering windows [5] is retired on this platform. Second, **the utilization profile validates the architecture**: logic concentrates in exactly the blocks doing protocol work (UOE) and interconnect arbitration (SmartConnect), while memory concentrates in the processor and buffering, no unexplained consumers. Third, **timing robustness by construction**: single SLR placement avoids inter SLR crossings, directly supporting closure of the 125 MHz single clock design. The post route noise report additionally confirms all I/O standards (LVCMOS18, LVDS) within electrical margins, with no signal integrity violations.

On the TCP platform the corresponding result is trivial but meaningful: 0% PL utilization (Section 5.3), the entire protocol function executes in hard silicon, and the fabric remains fully available.

Timing, power, and software footprint complete the cost picture. The post route timing summary met all analyzed constraints at 125 MHz, with WNS = +1.131 ns for setup and +0.011 ns for hold. The positive slack values confirm timing closure, although the hold margin is small. The Vitis build log quantifies the software economy of the offload architecture: the complete echo application occupies 20,352 bytes (text + data + bss), comfortably inside the 128 KB LMB memory with room for the UDP++ control plane.

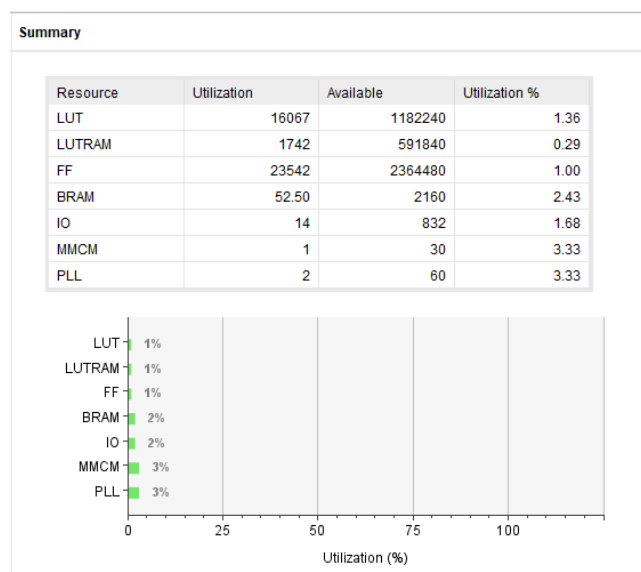


Figure 6.23: VCU118 utilization summary

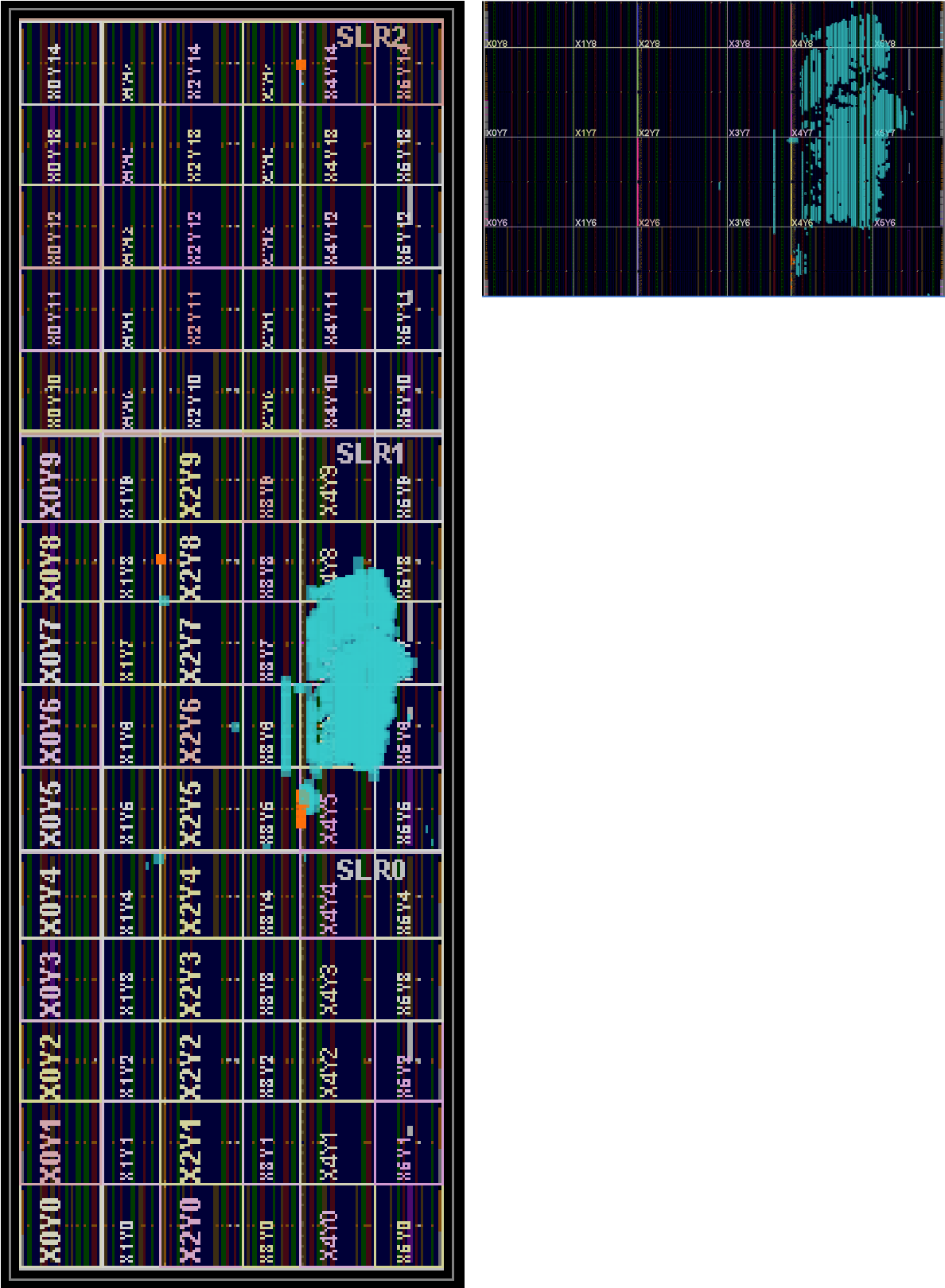


Figure 6.24: Implemented device views (xcvu9p)

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.131 ns	Worst Hold Slack (WHS): 0.011 ns	Worst Pulse Width Slack (WPWS): 0.005 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 63050	Total Number of Endpoints: 63012	Total Number of Endpoints: 26449
All user specified timing constraints are met.		

Figure 6.25: Timing summary, all constraints met

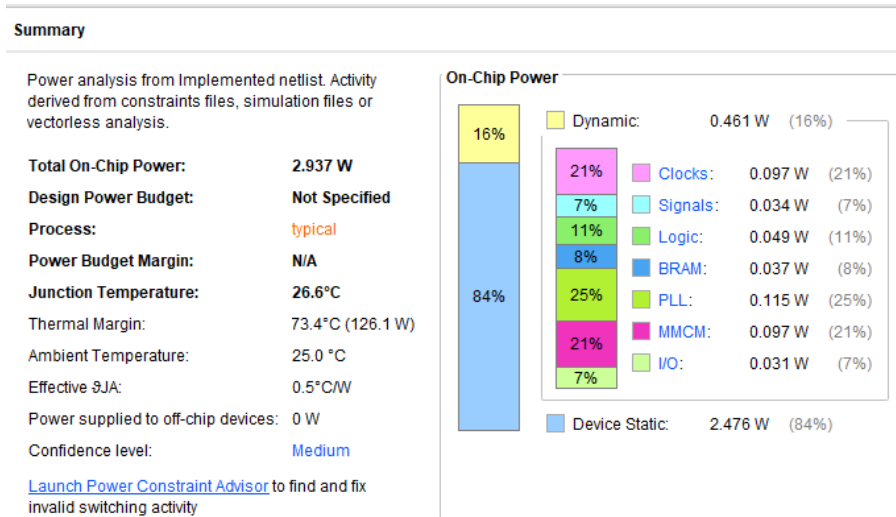


Figure 6.26: Power report, UDP baseline

```

OUTPUT x PROBLEMS 3 x DEBUG CONSOLE x VITIS LOG x C:\WINDOWS\SYSTEM32\CMD.EXE UDP_App:build
Vitis Messages UDP_Platform UDP_App:build x UDP_App:clean
296 |
297 [2/2] cmd.exe /C "cd . && F:\Xilinx\Vitis\2024.2\gnu\microblaze\nt\bin\mb-gcc.exe -O2 -mlittle-endian -mxl-soft-mul
298 text data bss dec hex filename
299
300 14272 608 5472 20352 4E80 UDP_App.elf
301
302 Build Finished successfully
303 -----
304 [5/6/2026, 7:56:24 AM]: Build for UDP_App:build with id '6991f3ea-a04e-45b4-bcae-63a4eadac0b2' ended.
305

```

Figure 6.27: Vitis build log, application footprint

7. UDP++ Reliability Layer

The UDP++ layer adds selective, NAK based retransmission on top of the hardware UDP baseline of Section 6. The reliability functions operate in hardware on the 125 MHz AXI4 Stream data path, while the MicroBlaze performs configuration, monitoring, and DMA control. The layer comprises three elements: a custom AXI IP core (udp_pp, v1.0) inserted inline on the AXI Stream paths between the AXI DMA and the UOE. A modified tuser_builder supporting variable frame lengths, and a Vitis C control plane. The udp_pp core is memory mapped over AXI4 Lite at base address 0x44A2_0000 (Appendix 12.3).

The baseline architecture was explicitly prepared for this insertion: the module reuses the TUSER metadata convention established by the tuser_builder and inherits ILA observation points at all four of its stream boundaries (TX_DMA_TO_UDP_PP, TX_UDP_PP_TO_UOE, RX_UOE_TO_UDP_PP, RX_UDP_PP_TO_DMA), which serve as the verification anchors of Section 7.7.

The layer was implemented against three supervisor provided conceptual diagrams, reproduced below as the specification contract. The first defines the UDP++ packet formats: a 16 byte DATA header containing an opcode, session identifier, 16 bit window field, 32 bit sequence number, 32 bit acknowledgment bitmap, Fletcher 16 checksum, and two reserved bytes, together with a separate 8 byte NAK frame. For DATA packets, the checksum covers header bytes 0-11 and the payload. The second fixes the header's position, at the head of the UDP payload, inside an otherwise standard Ethernet/IPv4/UDP frame. This in payload placement is the specification's most consequential decision: every switch, NIC, operating system, and capture tool on the path sees plain UDP, so the reliability layer deploys over unmodified infrastructure. The third diagram defines the AXI4 Lite register map through which the processor configures the layer and observes its behavior, control bits, live state (next sequence, oldest history entry), and diagnostic counters for received NAKs, history misses, retransmissions, and checksum failures.

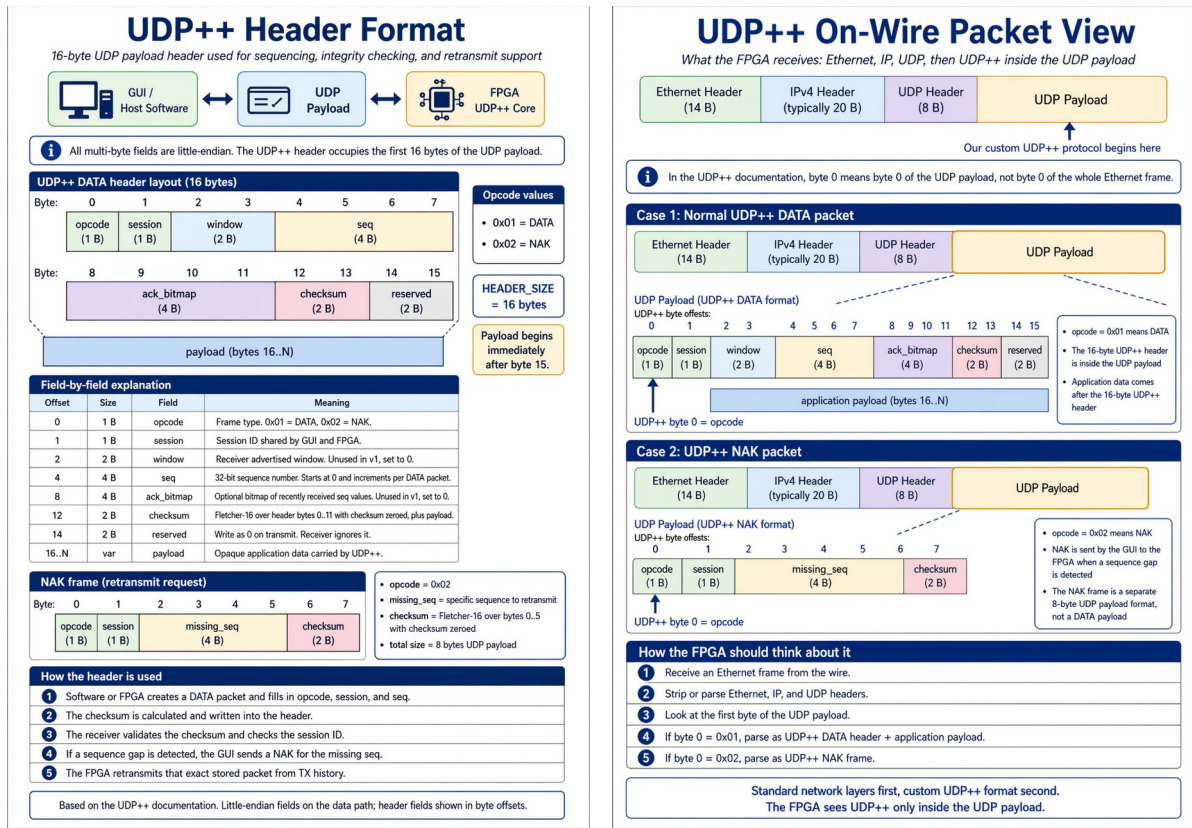


Figure 7.1: Supervisor specification: UDP++ header format (left) and frame placement (right)

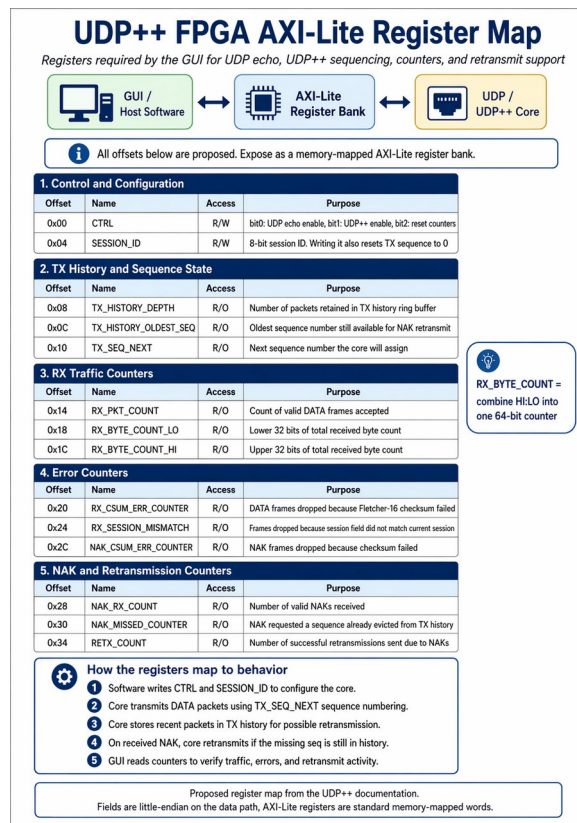


Figure 7.2: Supervisor specification: AXI4 Lite register map

The division of labor implied by these diagrams shaped the entire implementation. Sequence numbering, Fletcher 16 checksum computation, gap detection, NAK generation, and retransmission are implemented in programmable logic on the 125 MHz AXI4 Stream path. The MicroBlaze configures and monitors the UDP++ core and also manages the DMA transfers and completion interrupts. The subsections that follow develop the implemented microarchitecture from this specification.

7.1 Initial Design Requirements

The UDP++ layer was specified ahead of implementation by three supervisor provided conceptual diagrams: the 16 byte DATA header and the separate 8 byte NAK frame format, the placement of that header inside a standard Ethernet/IPv4/UDP frame, and the AXI4 Lite register map exposing control, state, and counter registers to the processor. These diagrams defined the main functional requirements used for implementation and testing. They are reproduced and analyzed at the opening of Section 7, where the implemented architecture is developed directly from them.

7.2 Reliability Concept and Header Design

UDP++ uses two packet formats. Each DATA frame prepends a 16 byte header to the application payload, containing an opcode, a session identifier, a 32 bit sequence number, window and acknowledgment fields, and a Fletcher 16 checksum. Data frames carry opcode 0x01. With the 1,024 byte test payload, the resulting UDP payload is 1,040 bytes, consisting of the 16 byte UDP++ header and the application payload. Control frames (RETRANSMIT_REQ, hereafter NAK) carry opcode 0x02 and occupy 8 bytes only, holding the requested sequence number and a checksum. The sequence number increments after each newly transmitted DATA frame. retransmissions reuse the original sequence number. The receiver validates the checksum and session identifier and tracks sequence continuity.

Reliability is receiver driven. No positive acknowledgment is sent per frame. A retransmission is requested when a missing or corrupted DATA frame is detected. The return channel overhead is therefore an 8 byte NAK frame per detected recovery event rather than a per packet acknowledgment stream, reducing control overhead while enabling selective retransmission.

7.3 Sequence/NAK Architecture and Register Map

The transmit side maintains a running sequence counter (TX_SEQ_NEXT) and a retransmission window bounded from below by TX_HISTORY_OLDEST_SEQ. The receive side maintains packet and byte counters together with dedicated error counters for checksum

failures and session mismatches. The NAK path is instrumented separately: valid received NAKs, NAKs failing checksum, NAKs outside the history window (NAK_MISSED), and executed retransmissions (RETX). All counters are exposed to the MicroBlaze through the AXI4 Lite register file of Table 7.1 and constitute the primary evidence source of Section 7.7. Gap handling supports packet reordering by buffering valid out of order DATA frames within the receive window. When a future sequence number first reveals a gap, the receiver generates a NAK for the expected missing sequence. If the missing frame subsequently arrives, the expected sequence advances across any consecutive frames already stored in the buffer, so no new retransmission request is issued for those buffered frames. (Section 7.7.4).

Offset	Register	Function
0x00	CTRL	Control: bit 0 UDP_EN, bit 1 UDP_PP_EN, bit 2 RESET_COUNTERS
0x04	SESSION_ID	Active session identifier
0x08	TX_HISTORY_DEPTH	History buffer depth
0x0C	TX_HISTORY_OLDEST_SEQ	Lower bound of the retransmission window
0x10	TX_SEQ_NEXT	Next transmit sequence number
0x14	RX_PKT_COUNT	Valid DATA frames accepted
0x18	RX_BYTE_COUNT_LO	Payload bytes delivered to the DMA (low word)
0x1C	RX_BYTE_COUNT_HI	Payload bytes delivered to the DMA (high word)
0x20	RX_CSUM_ERR_COUNTER	Receive checksum failures
0x24	RX_SESSION_MISMATCH	Session identifier mismatches
0x28	NAK_RX_COUNT	Valid NAK frames received
0x2C	NAK_CSUM_ERR_COUNTER	NAKs failing checksum validation
0x30	NAK_MISSED_COUNTER	NAKs outside the history

		window
0x34	RETX_COUNT	Executed retransmissions

Table 7.1: UDP++ AXI4 Lite register map (base address 0x44A2_0000).

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/axi_dma_0					
/axi_dma_0/Data_MMzS (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem	0xC000_0000	64K	0xC000_FFFF
/axi_dma_0/Data_SzMM (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem	0xC000_0000	64K	0xC000_FFFF
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem	0xC000_0000	64K	0xC000_FFFF
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0x41E0_0000	64K	0x41E0_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/mdm_1/S_AXI	S_AXI	Reg	0x4140_0000	4K	0x4140_FFFF
/microblaze_0_axi_intc/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	128K	0x1_FFFF
/top_uoe_0/S_AXI	S_AXI	reg0	0x2_0000	16K	0x2_3FFF
/tri_mode_ethernet_mac_0/s_axi	s_axi	Reg	0x44A0_0000	64K	0x44A0_FFFF
/tuser_builder_0/S00_AXI	S00_AXI	S00_AXI_reg	0x44A1_0000	64K	0x44A1_FFFF
/udp_pp_0/S00_AXI	S00_AXI	S00_AXI_reg	0x44A2_0000	64K	0x44A2_FFFF
Network 1					
/microblaze_0					
/microblaze_0/Instruction (32 address bits : 4G)					
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	128K	0x1_FFFF

Figure 7.3: Address Editor, udp_pp_0 at 0x44A2_0000

7.4 Retransmission Logic and TX History Buffer

Every new DATA frame is stored in the TX history buffer before being transmitted from that buffer. Upon a valid NAK whose sequence lies inside the window (TX_HISTORY_OLDEST_SEQ, TX_SEQ_NEXT), the frame is retrieved from the buffer and retransmitted, incrementing RETX (history hit). A NAK whose sequence is outside the valid history window, either older than TX_HISTORY_OLDEST_SEQ or not yet transmitted, increments NAK_MISSED and is discarded. The link continues uninterrupted (history miss). The buffer depth creates a direct memory resource versus recovery window tradeoff (Section 7.7.7). In the 40 frame acceptance run the window lower bound settled at sequence 24, i.e., the last 16 transmitted frames remained recoverable.

In hardware, the transmit engine is a 16-state finite-state machine that multiplexes three mutually exclusive transmit operations onto the single M_AXIS_TX interface: transmission of a new DATA frame, retransmission of a stored DATA frame, and generation of an 8-byte NAK frame. For a new DATA frame, the payload arriving from the AXI DMA is first stored in the TX

history buffer while the Fletcher 16 checksum is calculated. The completed 16 byte header is then written into the buffer, after which the entire frame is transmitted from the stored copy. TX_SEQ_NEXT increments only after transmission of a new DATA frame, while retransmissions reuse the original sequence number. A retransmission request or an RX side NAK generation request arriving while the transmit engine is busy is stored as pending and processed when the engine returns to the idle state.

The NAK handling chain separates frame validation, history window checking, and replay execution. The dedicated NAK checker validates the opcode, session identifier, and Fletcher 16 checksum. A valid NAK is then passed to the retransmission controller, which checks whether its sequence lies within the interval (TX_HISTORY_OLDEST_SEQ, TX_SEQ_NEXT). An out of window request, either older than the retained history or referring to a sequence not yet transmitted, increments NAK_MISSED and is discarded. An in window request is converted into a replay command and history buffer slot for the transmit FSM.

The receive engine uses two cooperating state machines: an RX parser and a DMA sender. The parser distinguishes 16 byte DATA headers from 8 byte NAK frames. For DATA frames, it validates the session identifier and Fletcher-16 checksum, strips the UDP++ header, and stores valid payloads according to their sequence numbers. When a future sequence reveals a missing frame, the receiver immediately requests a NAK for the expected sequence. Valid out-of-order frames within the receive window remain buffered, and when the missing frame arrives, sequence tracking advances across any consecutive frames already stored. The DMA sender releases the application payloads toward the DMA strictly in sequence order.

7.5 Inline Integration and Dynamic TUSER Metadata

The udp_pp core exposes symmetric stream interfaces, shown below: S_AXIS_TX/M_AXIS_TX on the transmit path between the AXI DMA and the UOE, where the header is injected, and S_AXIS_RX/M_AXIS_RX on the receive path between the UOE and the AXI DMA, where the header is validated and stripped so that the DMA writes clean payload to memory.

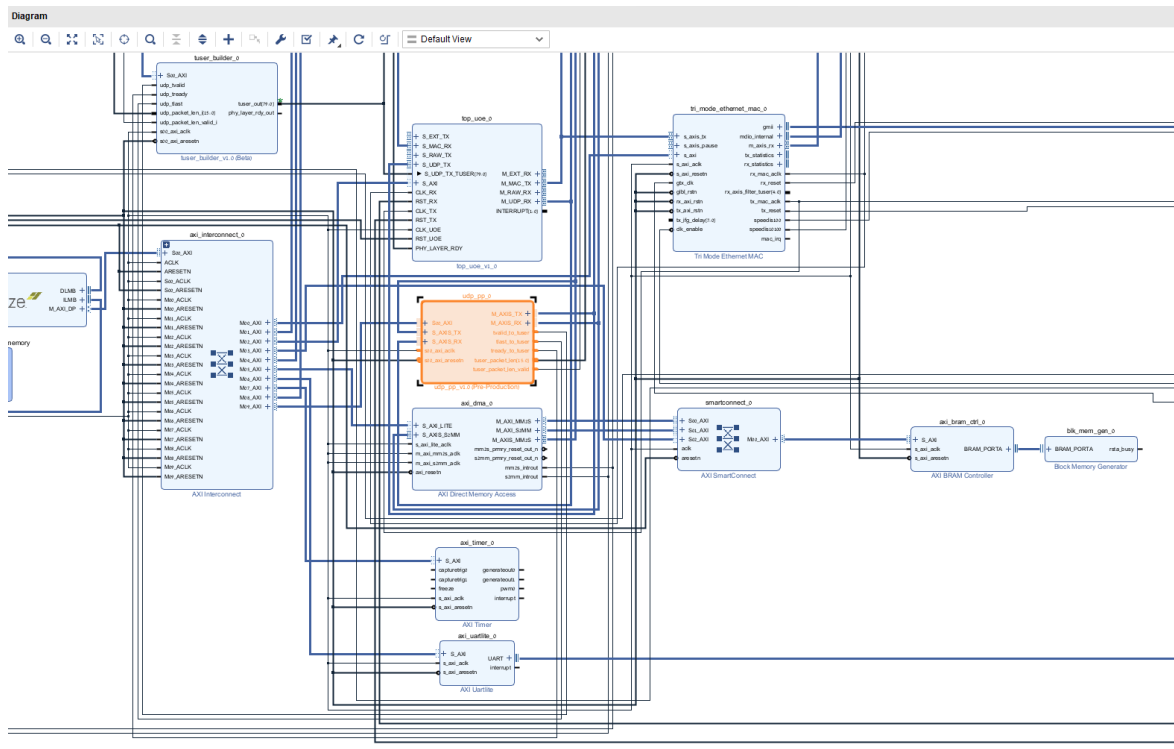


Figure 7.4: *udp_pp v1.0 inline in the block design (zoom in)*

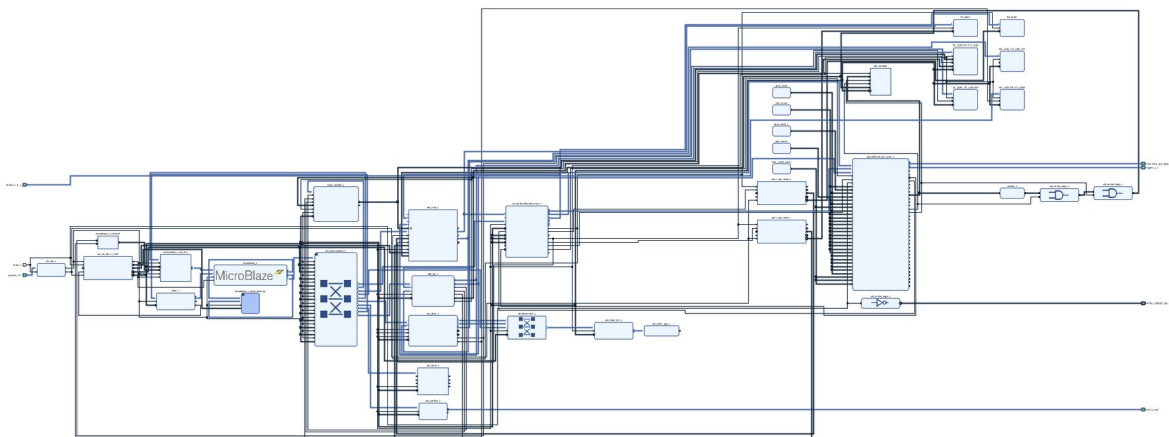


Figure 7.5: *Final block design with the UDP++ layer (zoom out)*

Because the frame length is no longer constant (8 byte NAKs versus 1,040 byte data frames), the `tuser_builder` top level was extended with two inputs driven by `udp_pp`, `udp_packet_len_i[15:0]` and `udp_packet_len_valid_i`, together with mirrors of the stream handshake (`tvalid`, `tready`, `tlast`). When the length input is valid, bits [47:32] of the 80 bit TUSER vector (the length field) are overridden with the per frame value. An `in_packet` tracker latches the selected TUSER word at the first beat of each frame so that the metadata remains stable for the frame duration. The AXI4 Lite slave submodule of the `tuser_builder` is unchanged, preserving backward compatibility with the UDP baseline (Appendix 12.1.3).

The original tuser_builder assembled a static 80 bit TUSER vector from software programmed destination IP, payload length, source port, and destination port fields, while a fifth AXI4 Lite register controlled the PHY ready signal. The selected TUSER value is latched at the first accepted beat of each packet. UDP++ breaks the static length assumption underlying that design: the same interface must now carry 1,040 byte data frames, 8 byte NAK frames, and history replays, back to back. The block's top level was therefore extended with two inputs driven directly by the udp_pp core, `udp_packet_len_i[15:0]` and `udp_packet_len_valid_i`. When the valid qualifier is asserted, the hardware substitutes the length field of the assembled TUSER vector (bits 47:32) with the per frame value supplied by the reliability layer. When it is not, the block falls back to the software programmed register, preserving exact baseline behavior when UDP++ is disabled.

Critically, the substitution preserves the block's race safety contract. The composed vector, whichever length source it carries, is still latched exactly once per packet, on the start of packet event derived from the `udp_pp` transmit output handshake signals, and held frozen until TLAST. A software register update or a length change for the next frame therefore cannot corrupt a frame already in flight. The UOE consequently receives, for every frame, a TUSER whose length field equals the number of UDP payload bytes in that frame, allowing it to generate the UDP length and checksum fields correctly for dynamically sized traffic. TLAST is generated separately by the `udp_pp` transmit engine.

Verification infrastructure update, ILA capture depth. Ahead of the UDP++ acceptance campaign, all System ILA cores (e.g., at the `TX_UDP_PP_TO_UOE` boundary) were recustomized: Sample Data Depth raised from 1,024 to 16,384 samples (MIX monitor type, three probes, capture control enabled). This removes the limit recorded in the UDP stress campaign (Section 6.10), where a 1,024 sample window could not contain both boundaries of a 1,024 byte packet. A 16,384 sample window captures a complete UDP++ frame contiguously, from the first injected header byte to the TLAST beat. The cost, e.g., 48.5 BRAM slices at this station, is accepted as a verification phase tradeoff and reflected in Section 7.7.7.

7.6 Software Control Plane

The control application binds to the hardware through the generated `xparameters.h` base addresses. `config_udp_mode` writes the session identifier, writes the requested mode together with `CTRL.RESET_COUNTERS`, waits briefly, and then writes the requested mode again without the reset bit, starting the selected test with the requested session and cleared diagnostic counters. The loopback and latency paths repeatedly arm the receive DMA, wait for the `RxDone` flag set by the S2MM interrupt handler, copy the received payload from the RX buffer to the TX buffer, start the transmit DMA, and wait for the `TxDone` flag set by the MM2S interrupt handler. The UDP++ TX tests repeatedly submit payloads to the DMA, while the UDP++ header and sequence number are generated in hardware. In the dedicated UDP++ throughput test, software

additionally refreshes a four byte timestamp before each packet. The RX only path re arms the receive DMA after every received packet, while the hardware gap tracker autonomously generates NAKs when required.

Observability follows the same philosophy as the baseline: `print_udp_pp_tx_status` and `print_udp_pp_rx_status` read the live hardware counters, `TX_SEQ_NEXT`, `OLDEST`, `NAK_RX`, `NAK_MISSED`, `RETX`, `NAK_CSUM_ERR` on the transmit side. `RX_PKT` and the byte counters on the receive side, and report them over the JTAG UART console. The software does not parse UDP++ headers or make reliability decisions and receives no per packet interrupts beyond DMA completion. In loopback mode, it copies the received payload from the RX buffer to the TX buffer, while all reliability actions visible in the acceptance tests, including header injection, gap detection, NAK emission, and selective replay, are executed by hardware. The console output serves as an audit trail showing how the hardware counters advanced.

7.7 UDP++ Comparative Results

The UDP++ layer was accepted against the fixed baseline axes of Sections 5.6, 6.10, and 6.11 using four complementary instrument classes: the adviser supplied Tester in UDP++ mode (header parsing, Fletcher 16 validation, live gap tracking, and automatic `RETRANSMIT_REQ` generation), Wireshark on the direct 1 GbE link, ILA arrays at the four `udp_pp` stream boundaries, and the hardware counters of Table 7.1 read over UART. Six test cases cover both protocol directions and the principal failure modes.

7.7.1 TX Path and Header Injection (Test Case 1)

Eight data frames were transmitted from the FPGA to the host. The Tester reports expected: 8, received: 8, missing: 0, with an incrementing sequence and a valid running checksum per frame, and its raw view shows the 16 byte header preceding each payload. Wireshark independently confirms eight UDP packets, each carrying a 1,040 byte UDP payload. On chip, the DMA side ILA shows a payload only stream ending with `TLAST`, while the `MM2S` interrupt indicates completion of the DMA transfer. The `UOE` side ILA shows the same payload preceded by the injected header, confirming that the header is generated in hardware between the two observation points. A zoomed out capture of three consecutive frames confirms sustained back to back operation, and the hardware counters close the loop: `TX_SEQ_NEXT`=8 with all NAK and retransmission counters at zero.

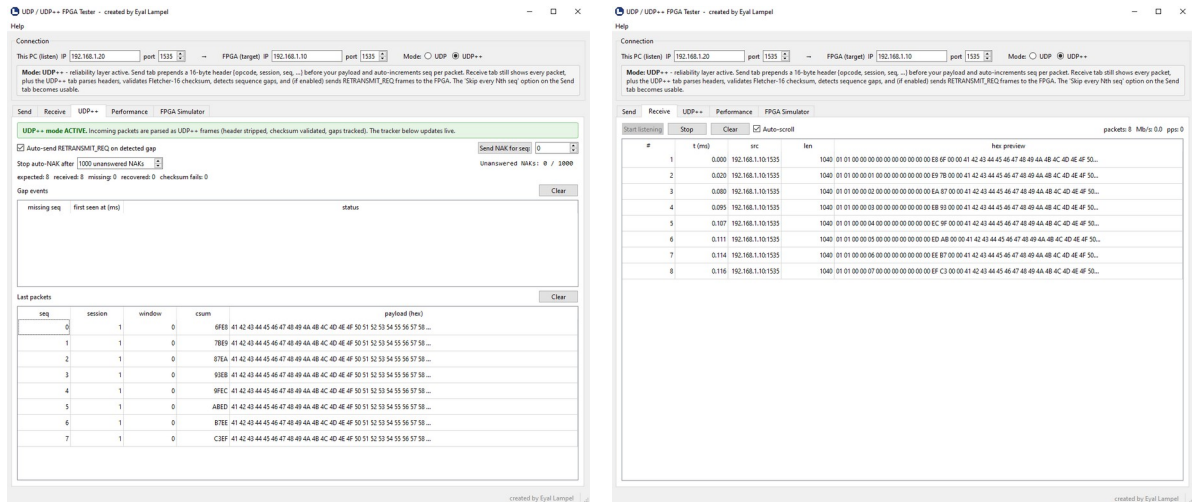
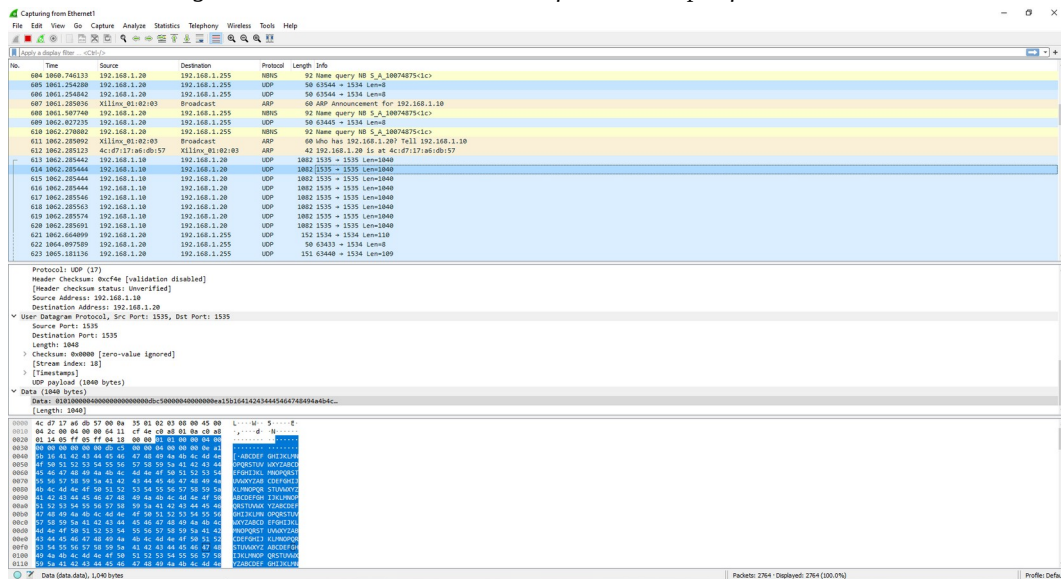


Figure 7.7: Tester UDP++ tab, 8/8 frames with per frame checksum



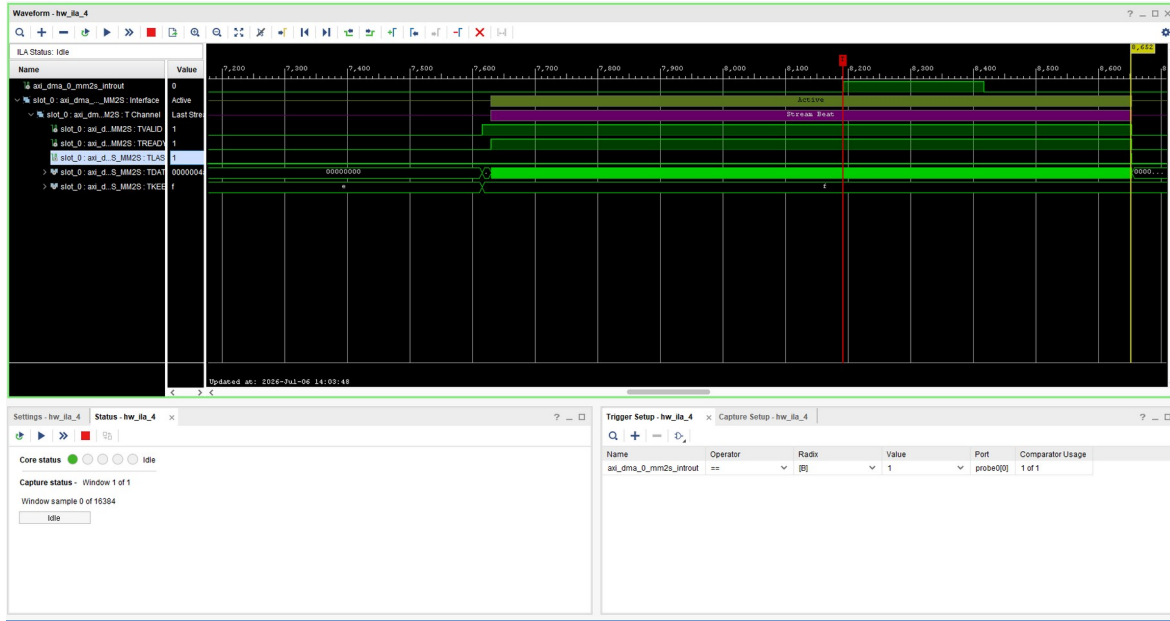


Figure 7.10: TX_DMA_TO_UDP_PP, TLAST at the payload boundary

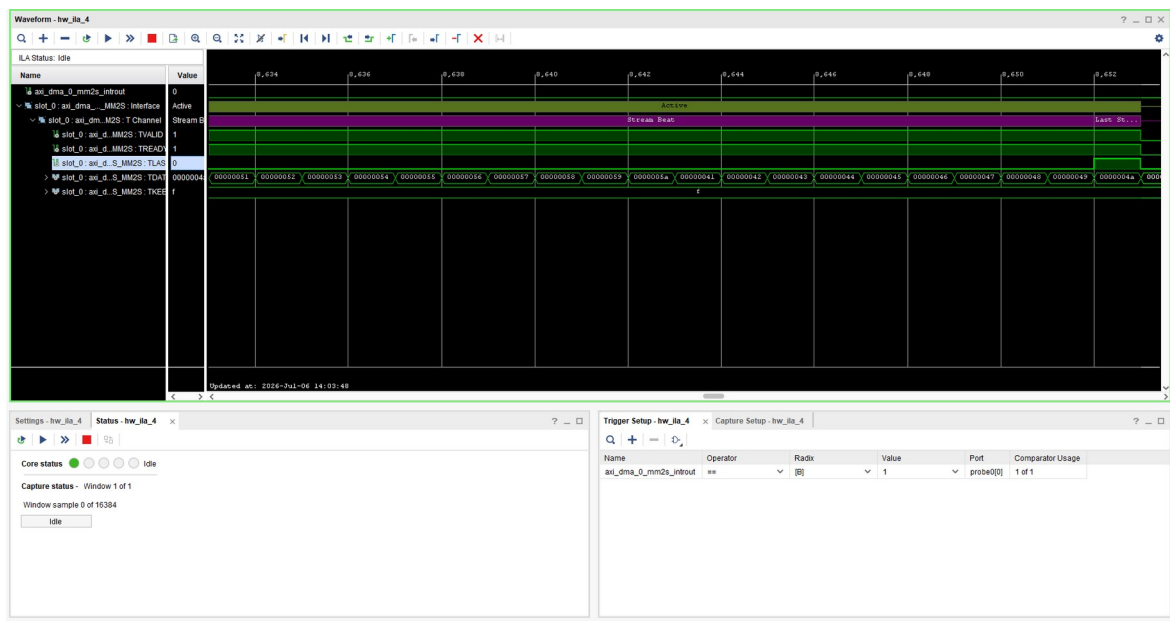


Figure 7.11: TX_DMA_TO_UDP_PP, final payload beats

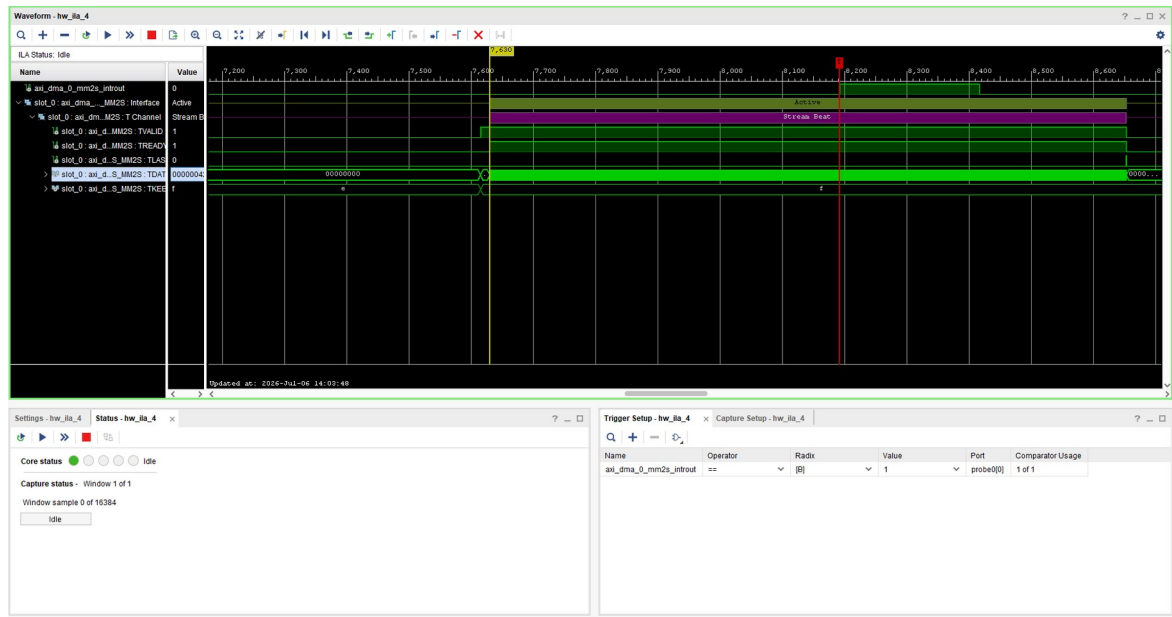


Figure 7.12: TX_DMA_TO_UDP_PP, mm2s_introut rising

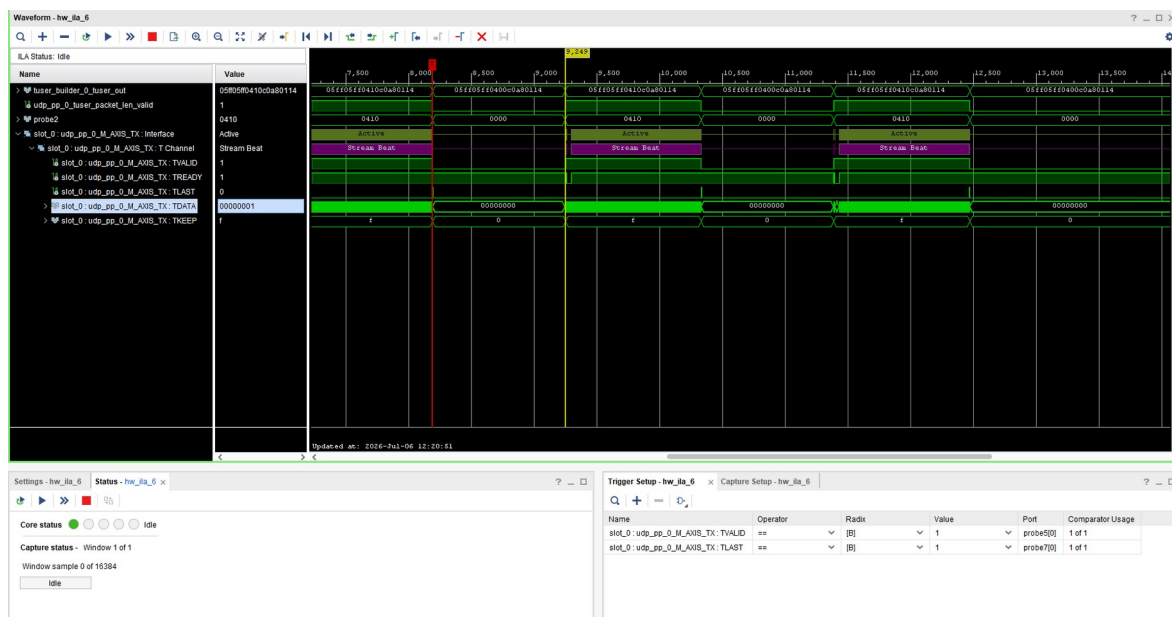


Figure 7.13: TX_UDP_PP_TO_UOE, three consecutive frames

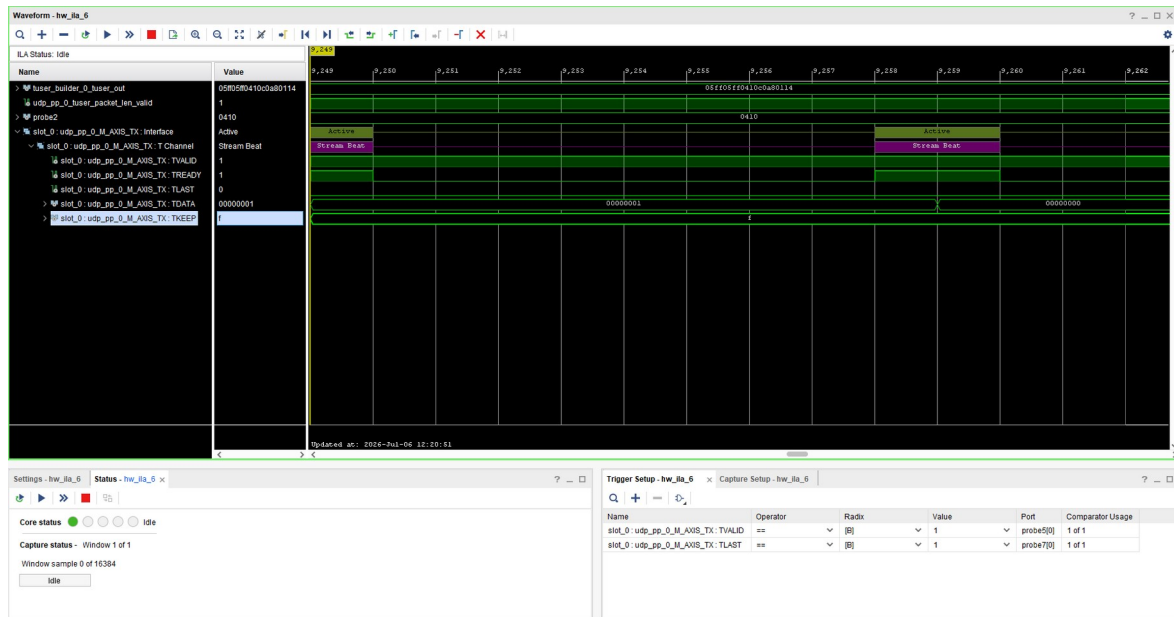


Figure 7.14: TX_UDP_PP_TO_UOE, packet header, sequence 1, opcode+session

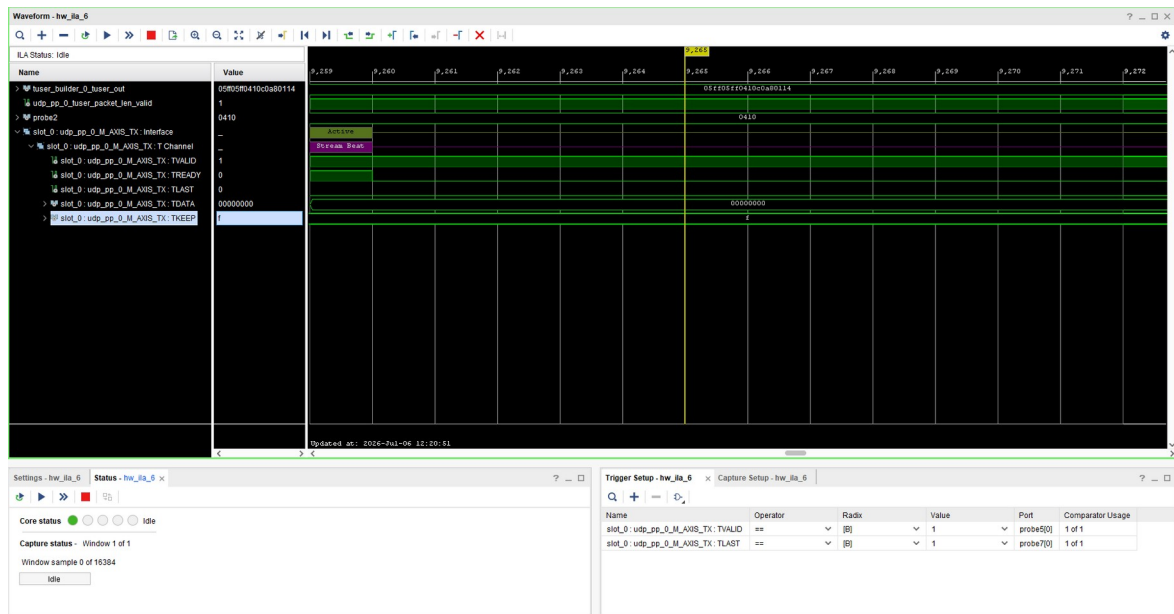


Figure 7.15: Packet header (zoom), window field

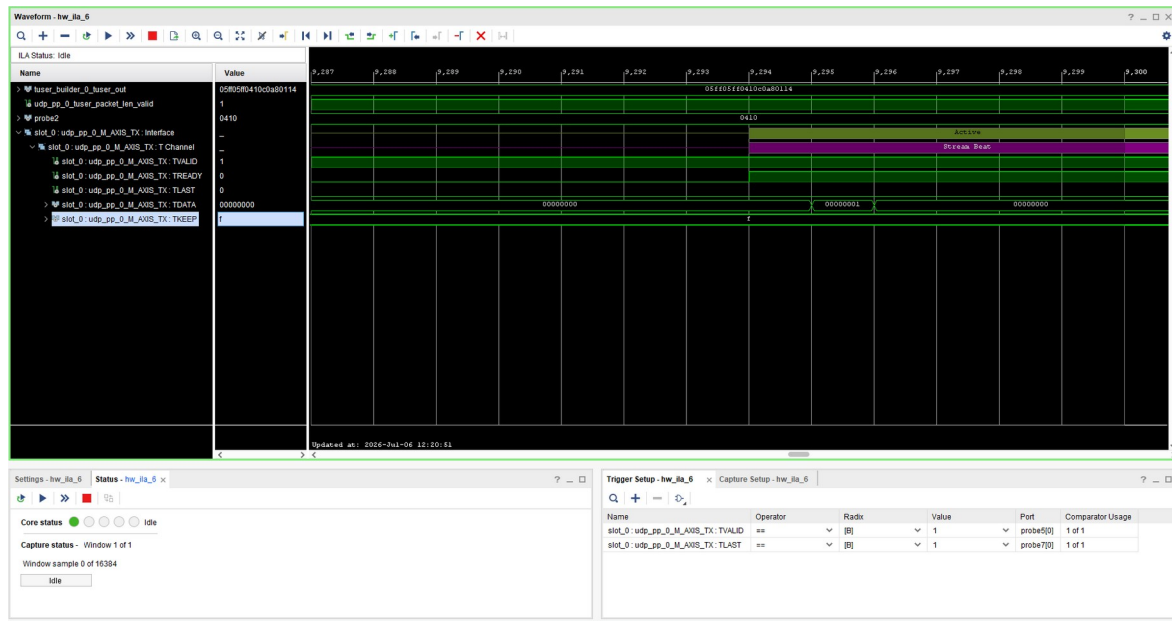


Figure 7.16: Packet header (zoom), seq+ack_bitmap

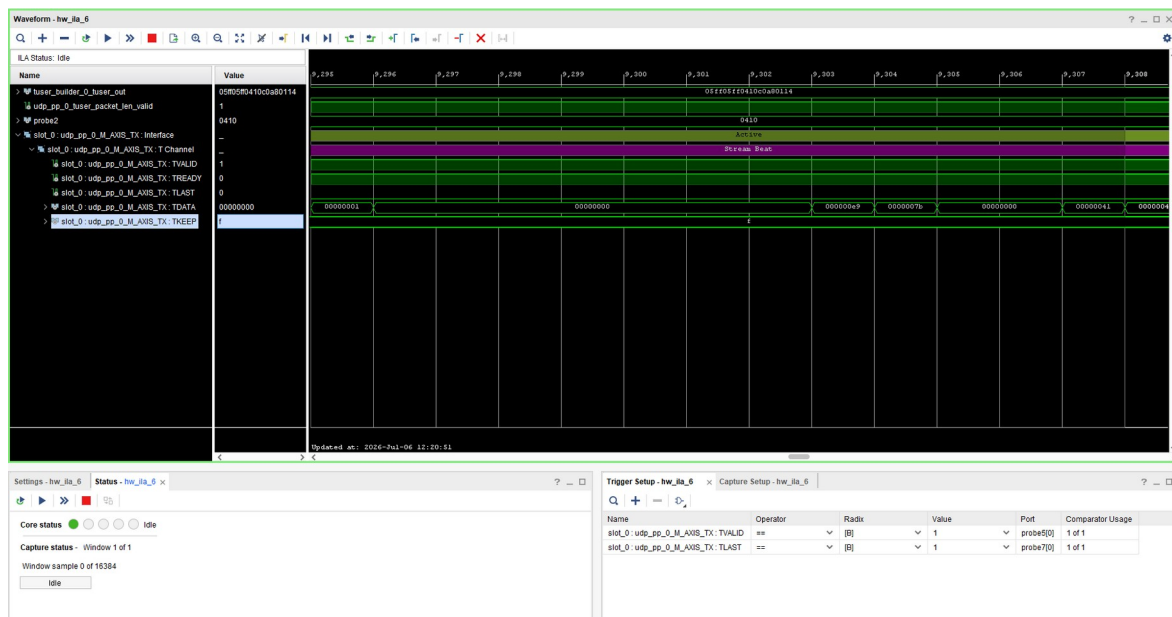


Figure 7.17: Packet header (zoom), ack_bitmap+checksum+reserved

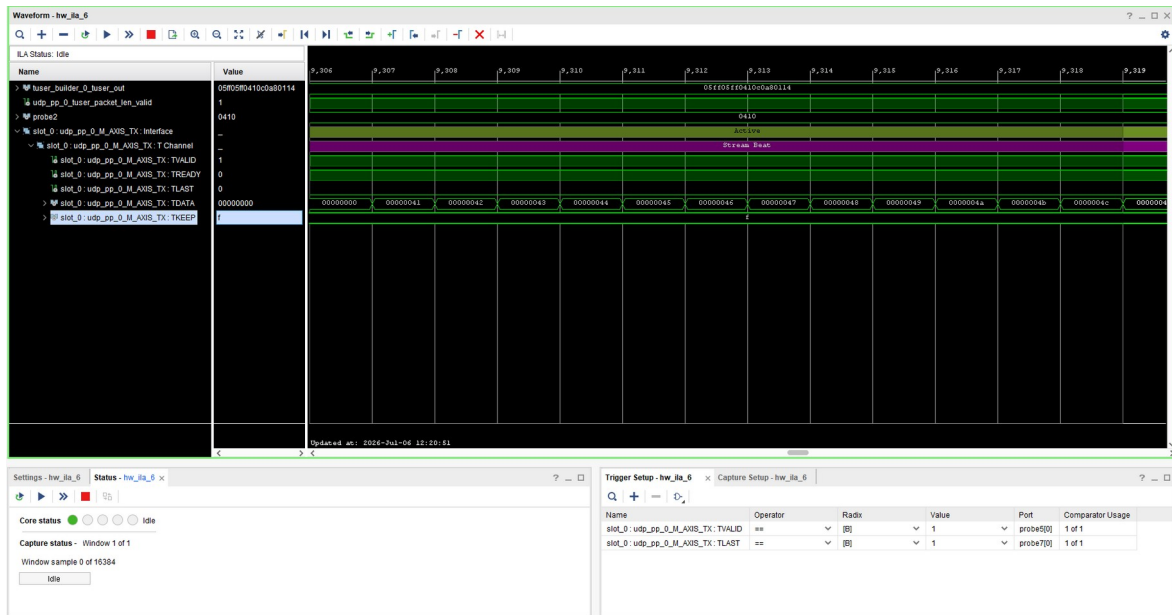


Figure 7.18: Packet header (zoom), payload

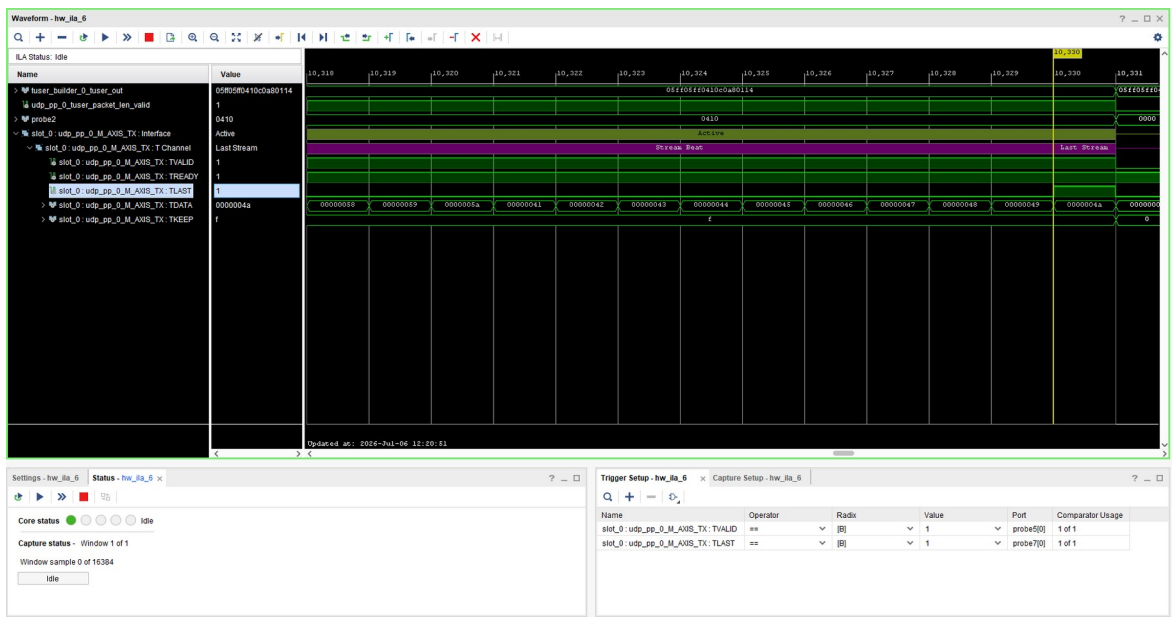


Figure 7.19: Packet header (zoom), end of payload+tlast rising

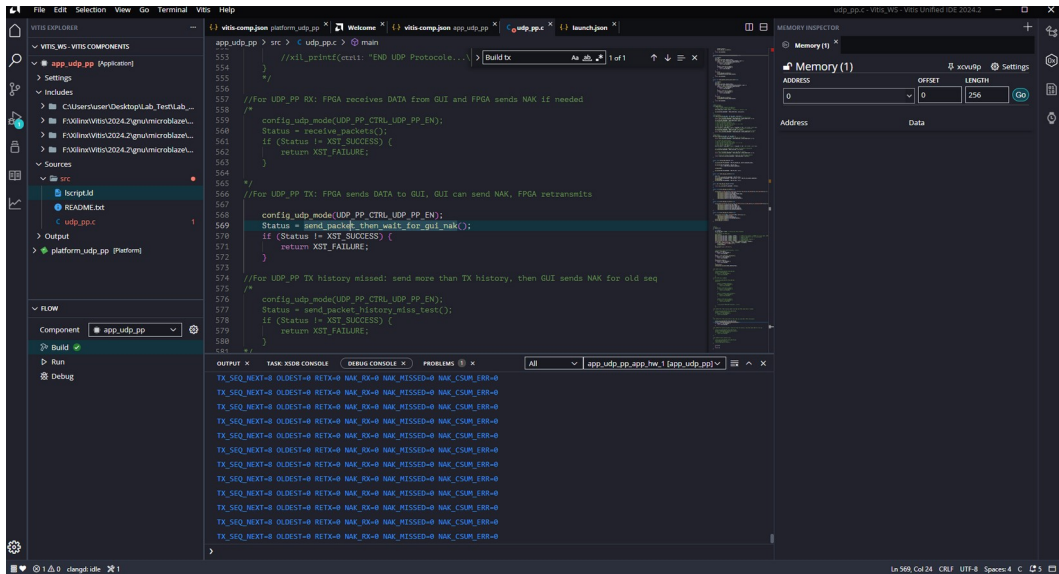


Figure 7.20: Vitis console, TX_SEQ_NEXT = 8

7.7.2 NAK Handling and Selective Retransmission (Test Case 2)

A NAK for sequence 1 was issued from the host. The 8 byte control frame is visible on the wire, two orders of magnitude smaller than a data frame, and the FPGA answers with a single retransmission of frame 1. The Tester consequently counts nine received frames, the recovered frame appearing after sequence 7. The receive side ILA captures the NAK entering the engine with a TUSER length field of 0x0008, and the counters confirm exactly one processed request and one retransmission: NAK_RX=1, RETX=1.

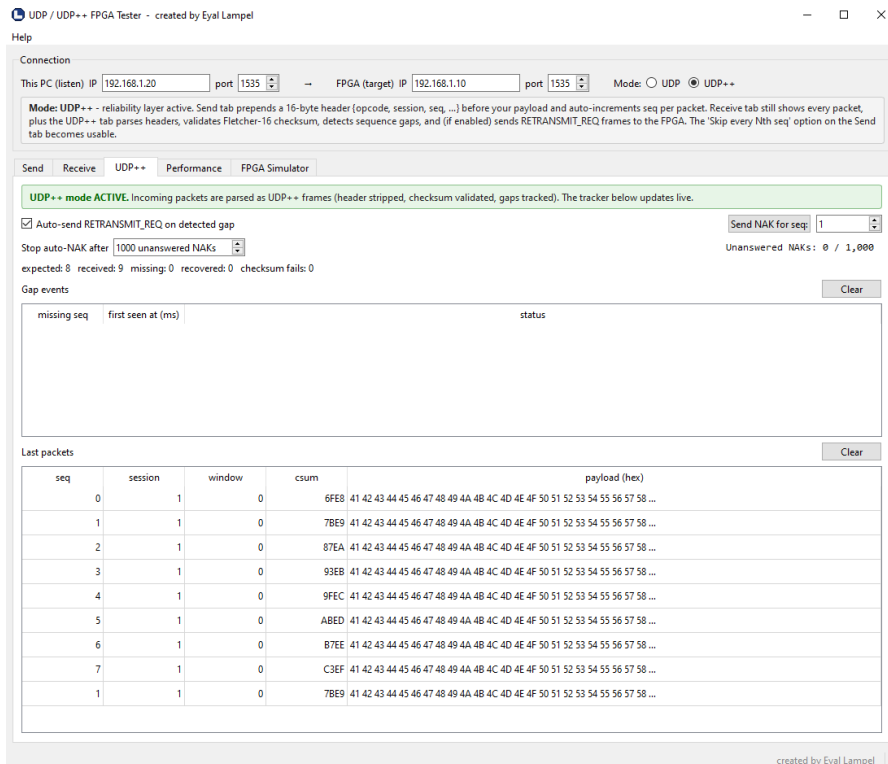


Figure 7.21: Tester, NAK for seq 1, retransmission received

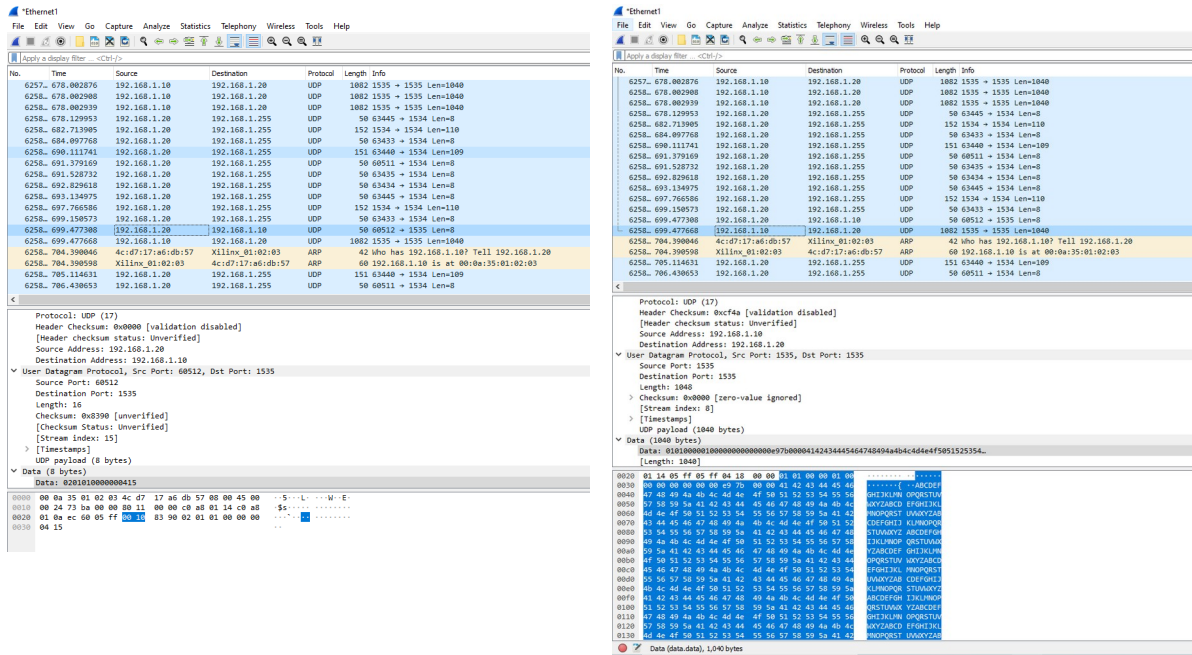


Figure 7.22: Wireshark, NAK frame (left) and retransmitted data (right)

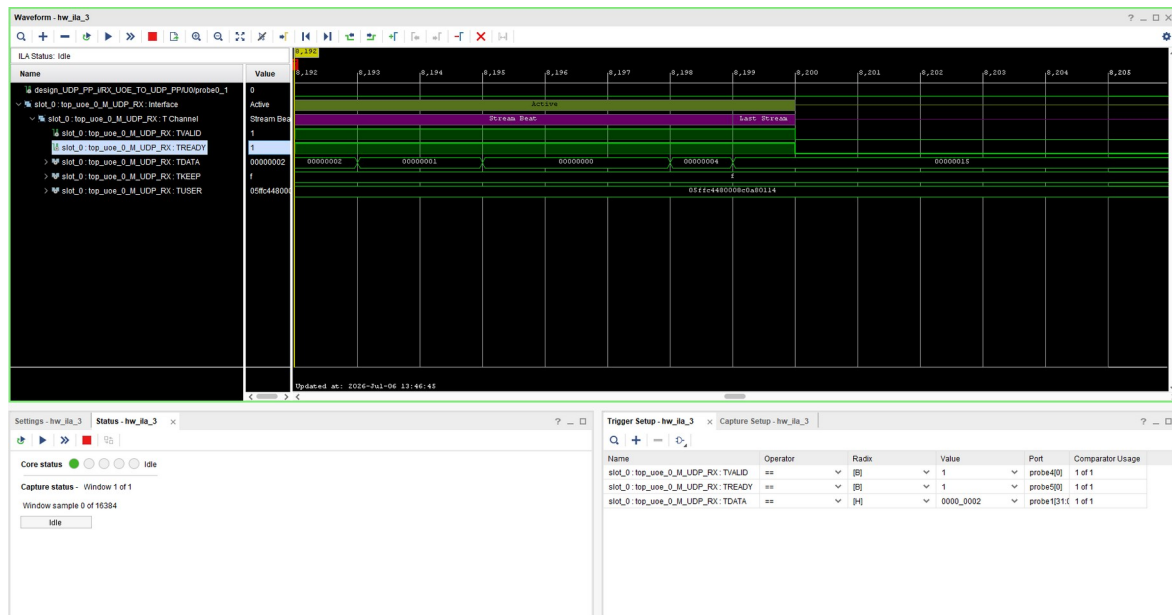


Figure 7.23: RX_UOE_TO_UDP_PP, 8 byte NAK ingress

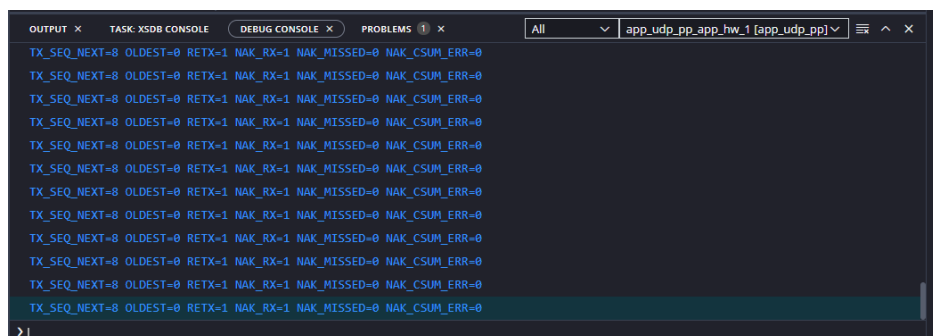


Figure 7.24: Vitis console, NAK_RX = 1, RETX = 1

7.7.3 TX History Window: Hit versus Miss (Test Case 3)

Forty frames were transmitted and received in full. A NAK for sequence 5, below the window bound OLDEST=24, remained unanswered: no retransmission appears on the wire, the Tester's unanswered NAK counter increments, and the hardware logs the request in NAK_MISSED while the link continues undisturbed (Figures 7.25–7.30). A NAK for sequence 36, inside the window, was served from the history buffer: the retransmitted frame appears on the wire and the received count rises to 41 (Figures 7.29–7.31). The final counters, NAK_RX=3, NAK_MISSED=2, RETX=1, OLDEST=24, quantify the bounded memory design of Section 7.4: requests inside the window are honored. Requests outside it degrade gracefully without disturbing the stream.

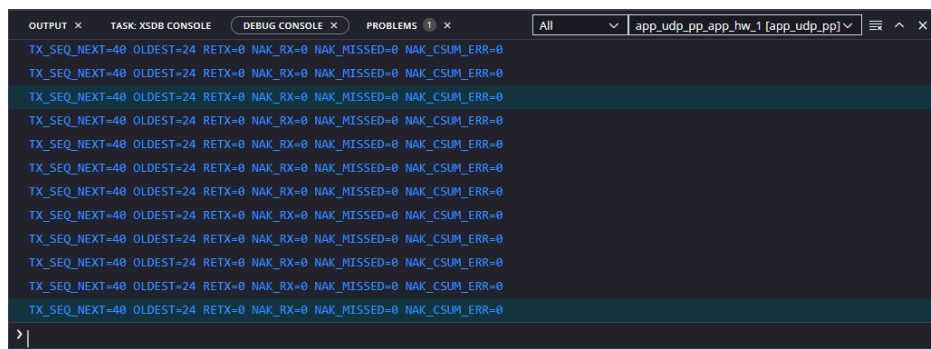


Figure 7.25: Vitis console, window state before NAK injection

Figure 7.26: Tester, in order progression

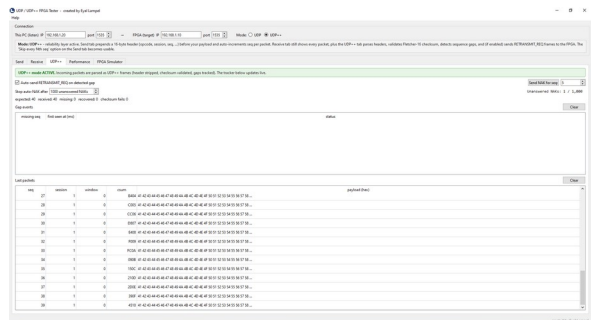
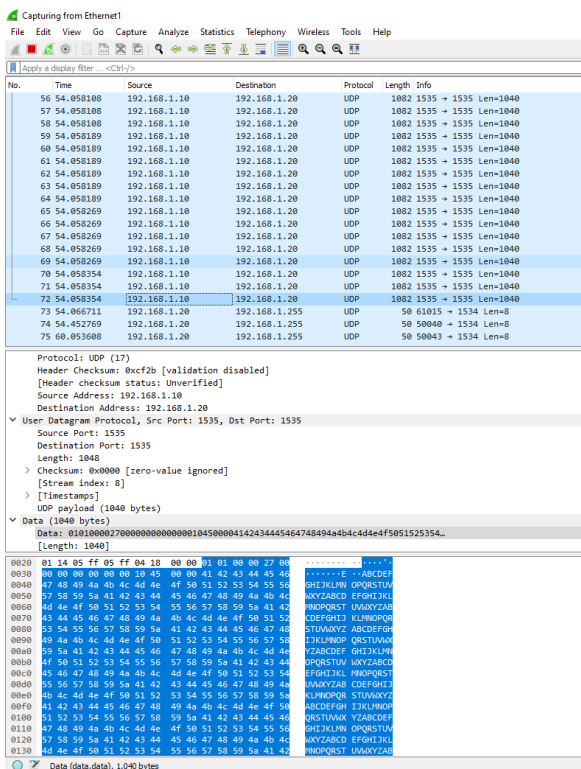


Figure 7.28: Last packet sent (left), NAK for seq 5 with no retransmission (right)

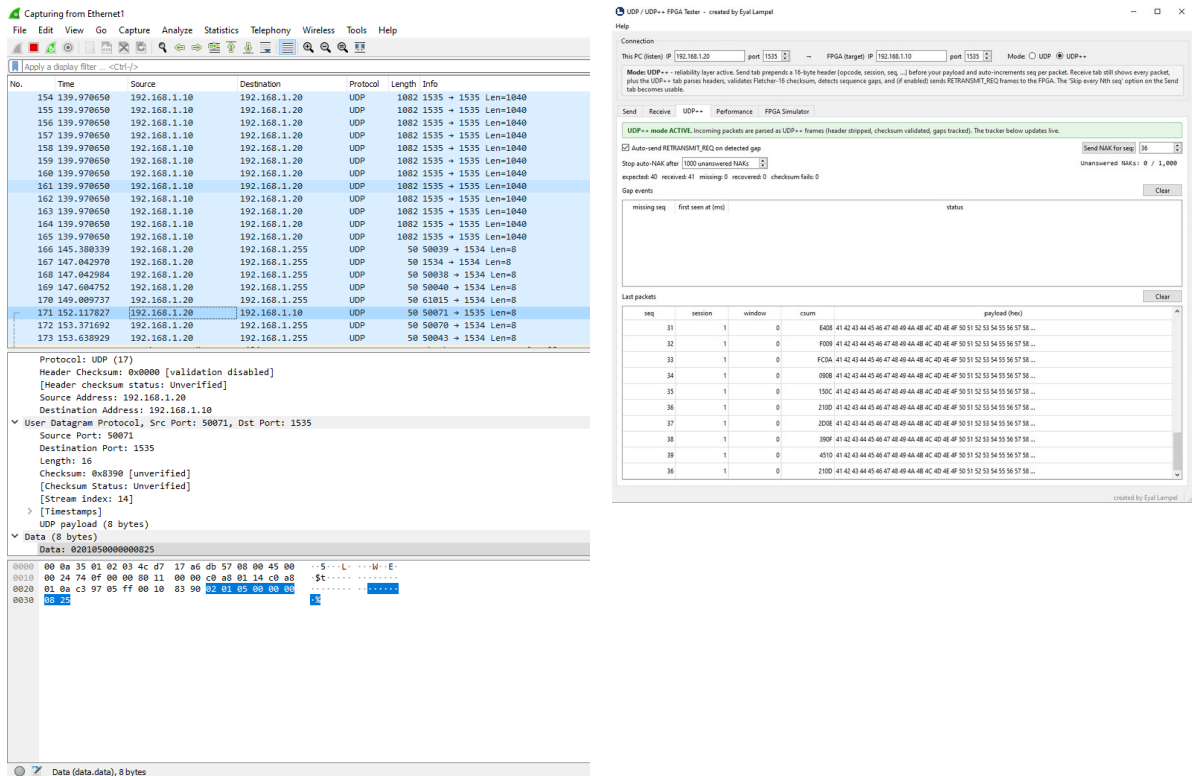


Figure 7.29: Seq 5 NAK on the wire (left), NAK for seq 36 and retransmission for 36 (right)

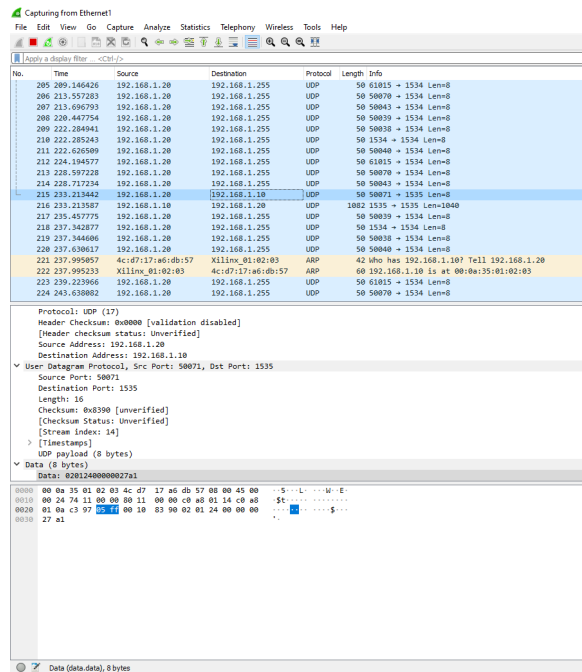


Figure 7.30: Seq 36 NAK on the wire

7.7.4 Out of Order Tolerance (Test Case 4)

During the same 40 frame FPGA to host run, the host observed packet reordering, with frame 13 arriving immediately after frame 5. The Tester temporarily marked sequences 6-12 as missing and later marked them as recovered when they arrived. No host generated NAK reached the FPGA and no retransmission was executed, as confirmed by `NAK_RX=0` and `RETX=0`. This result demonstrates the Tester's tolerance of reordering in this run. It does not demonstrate that the FPGA receive engine suppresses NAKs, because the implemented hardware requests a NAK when a future sequence first reveals a gap while buffering the out of order frame.

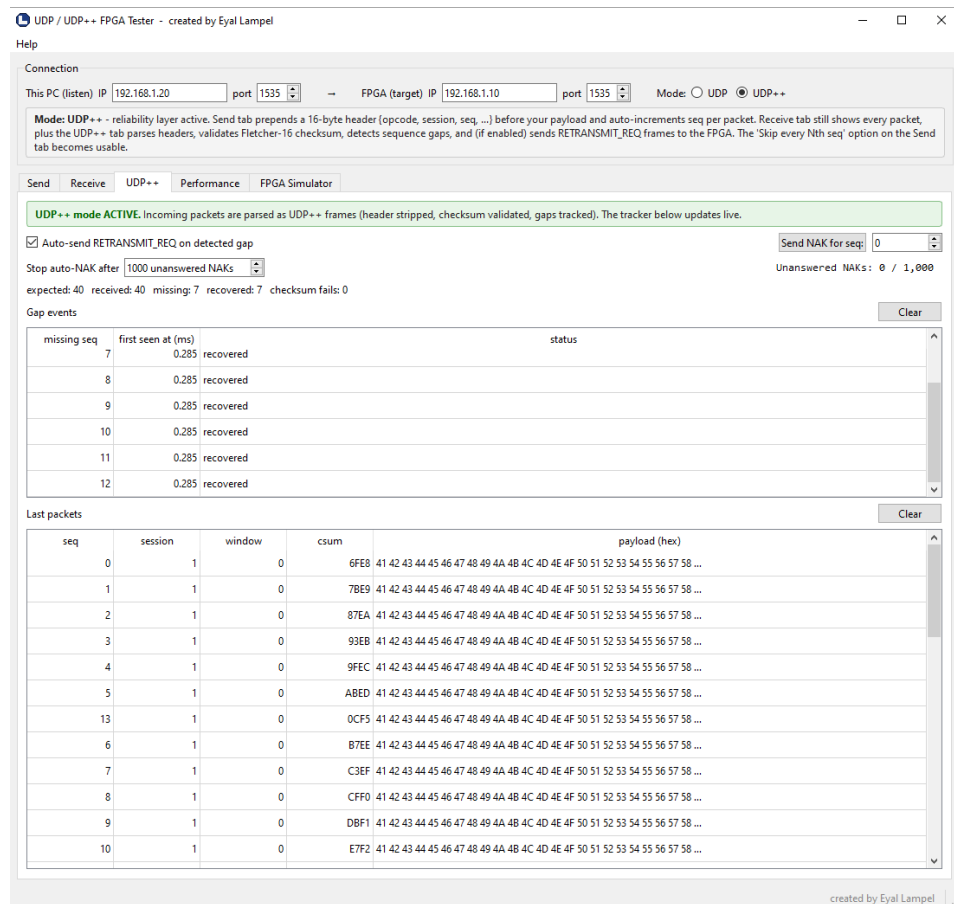


Figure 7.32: Tester gap events, sequences 7 to 12 recovered by reordering

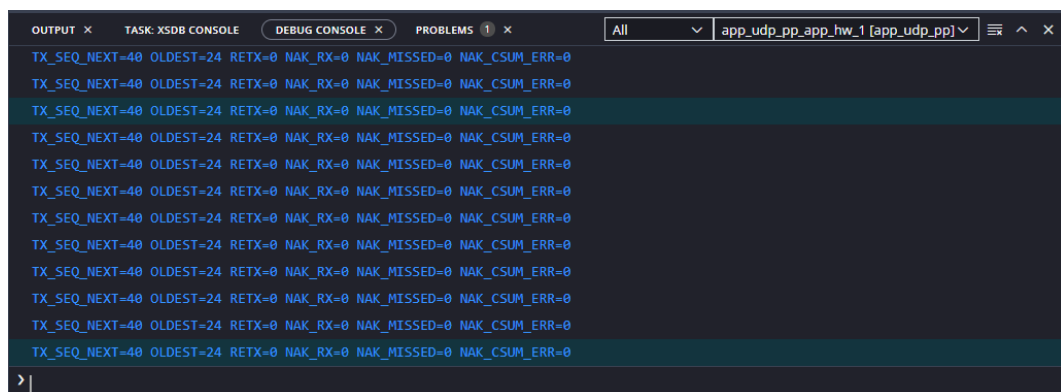


Figure 7.33: Vitis console, recovery by reordering, no retransmission

7.7.5 RX Path and Header Stripping (Test Case 5)

The reverse direction was verified with a single UDP++ frame carrying a 512 byte incrementing payload sent from the host on port 5002 (Tester Send tab and wire capture below). At the UDP++ ingress the ILA shows the complete frame, 16 byte header followed by the incrementing payload, while at the engine's DMA side output the header has been stripped and only clean payload bytes remain, with the s2mm interrupt closing the write into memory. The Vitis console and memory views confirm byte exact delivery: RX_PKT increments by one, the byte count increments by 512, no checksum or session errors are recorded, and the buffer contents match the transmitted pattern.

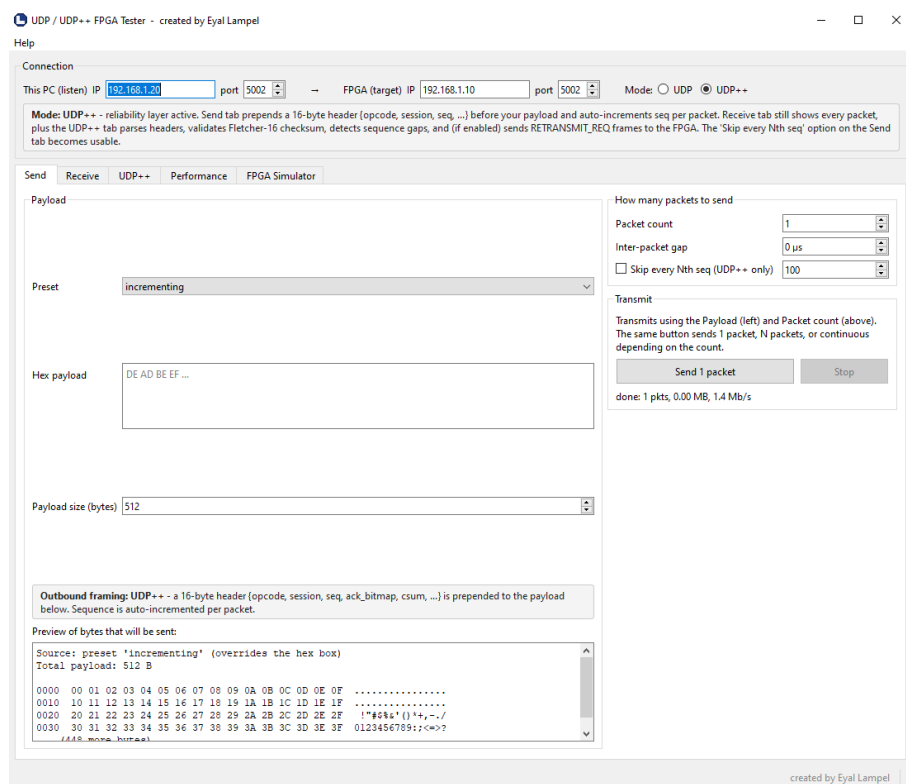


Figure 7.34: Tester Send tab, single 512 B UDP++ frame

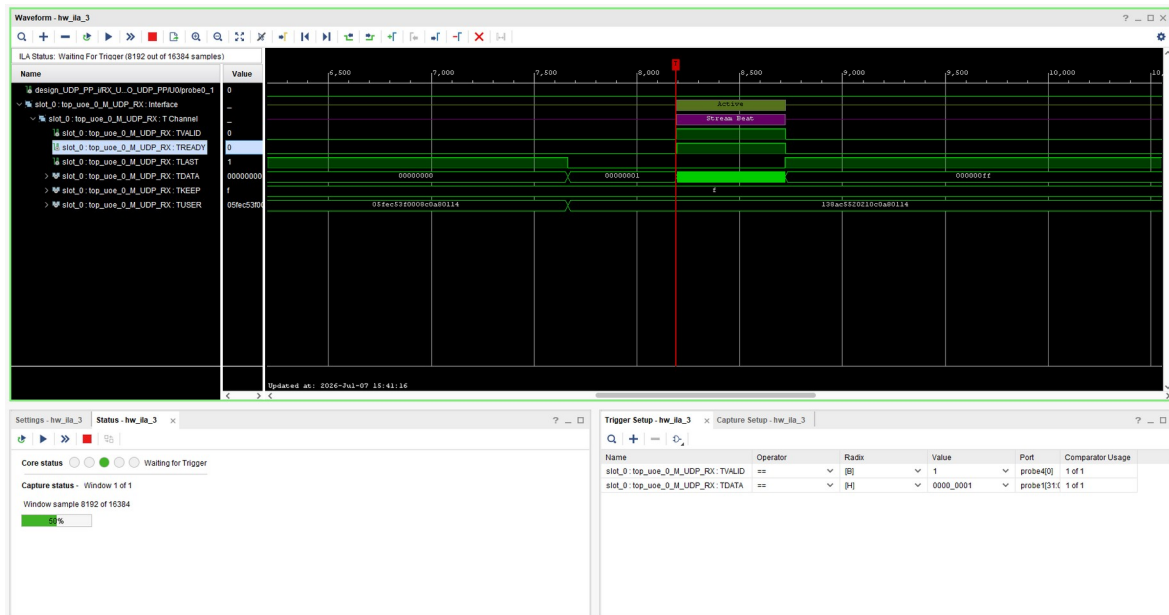


Figure 7.35: RX_UOE_TO_UDP_PP, frame arrival

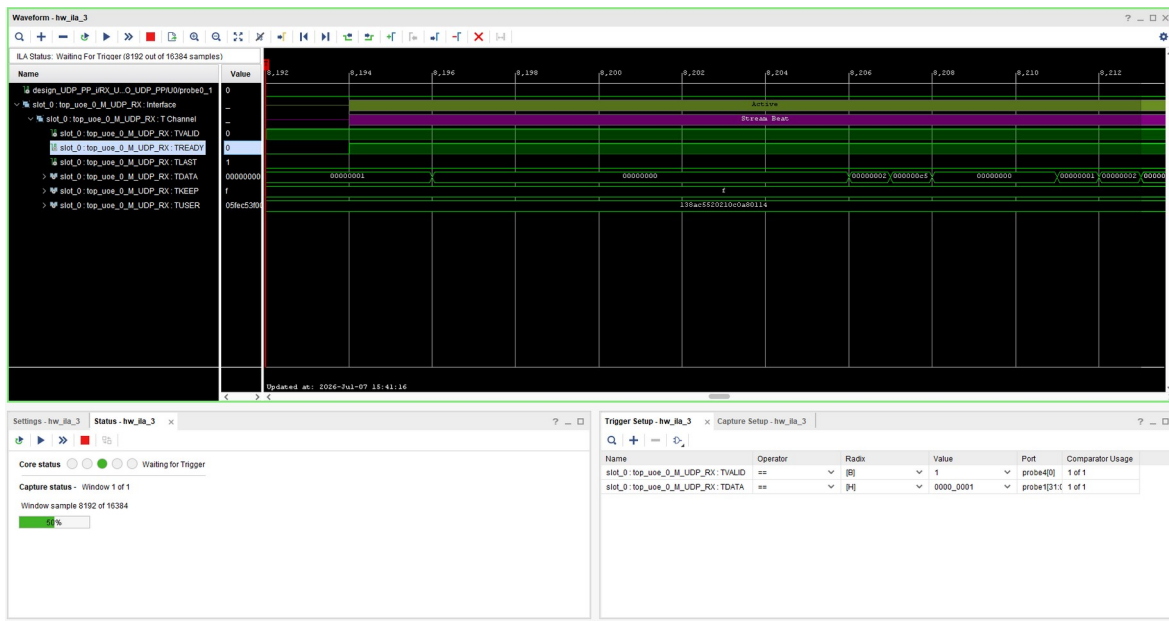


Figure 7.36: RX_UOE_TO_UDP_PP, header fields

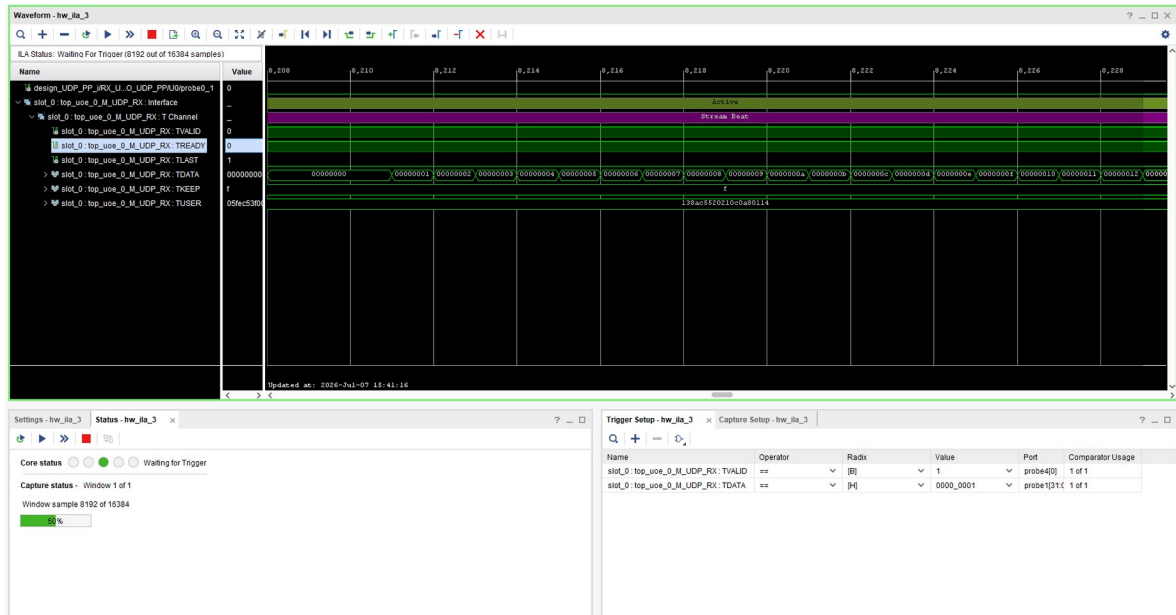


Figure 7.37: RX_UOE_TO_UDP_PP, payload after the header

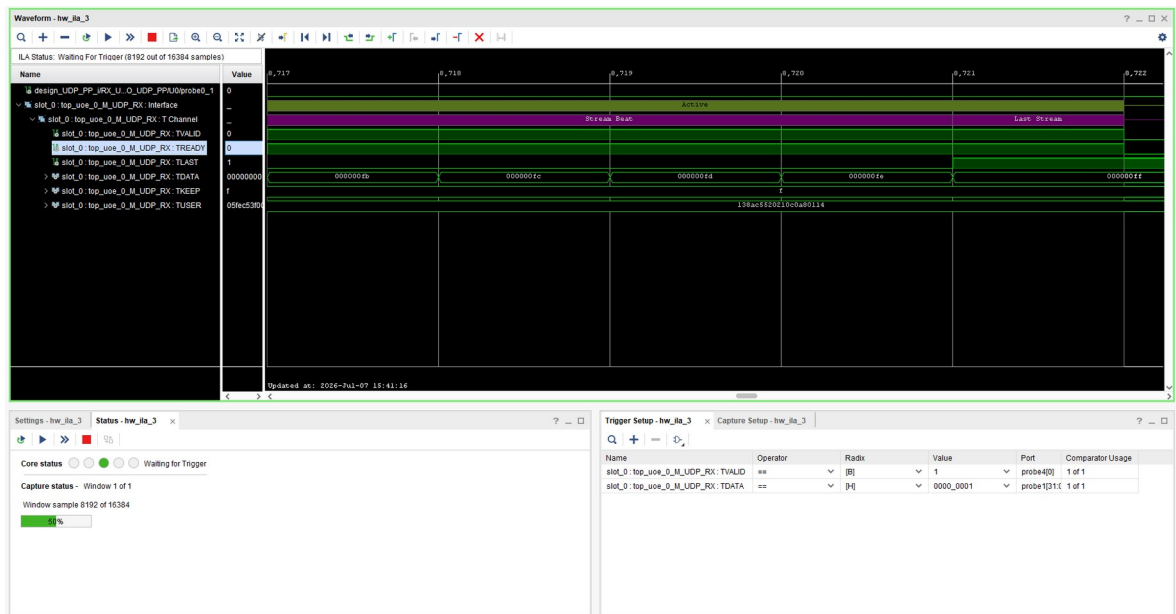


Figure 7.38: RX_UOE_TO_UDP_PP, end of frame with tlast rising

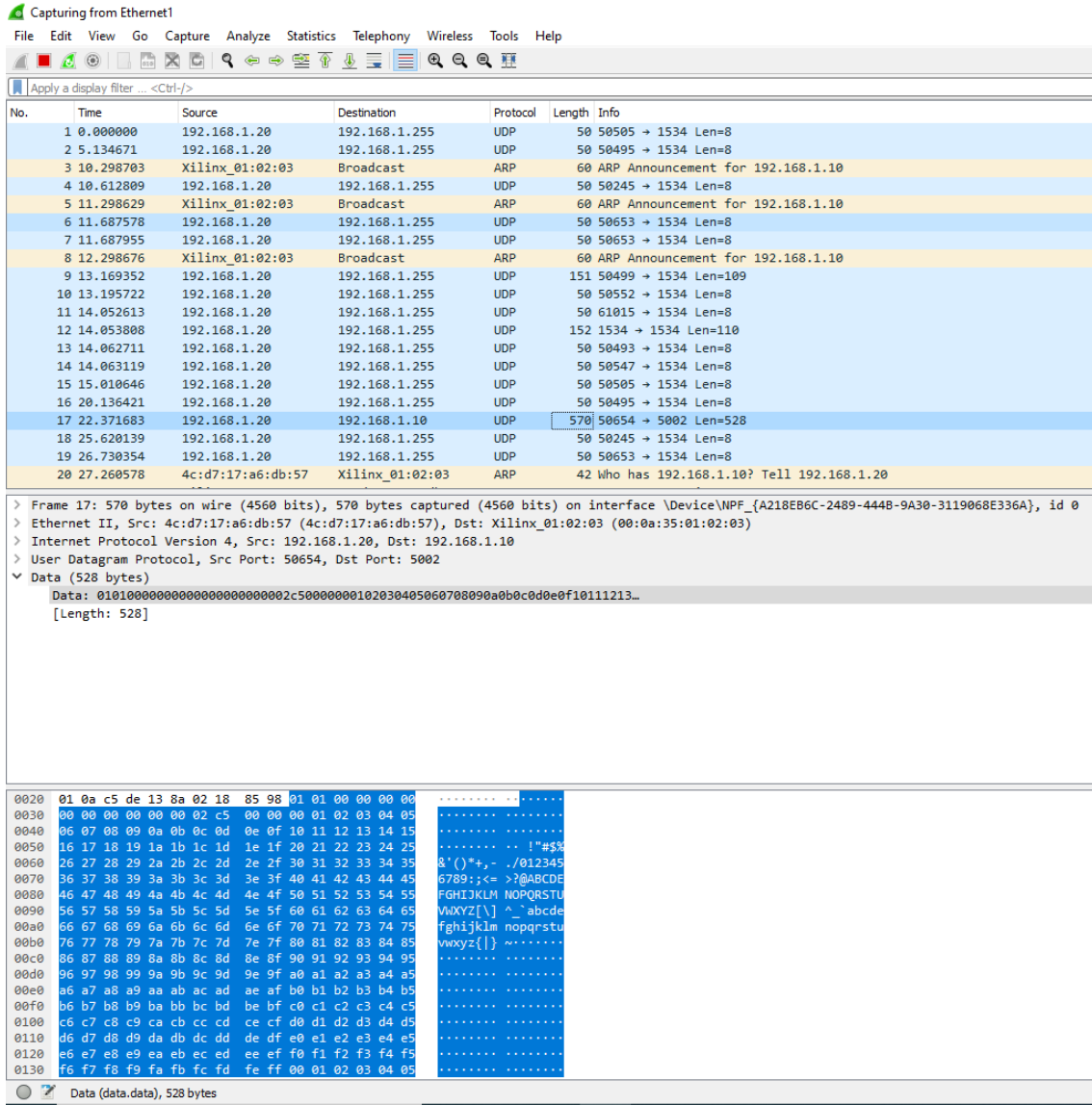


Figure 7.39: Frame in Wireshark

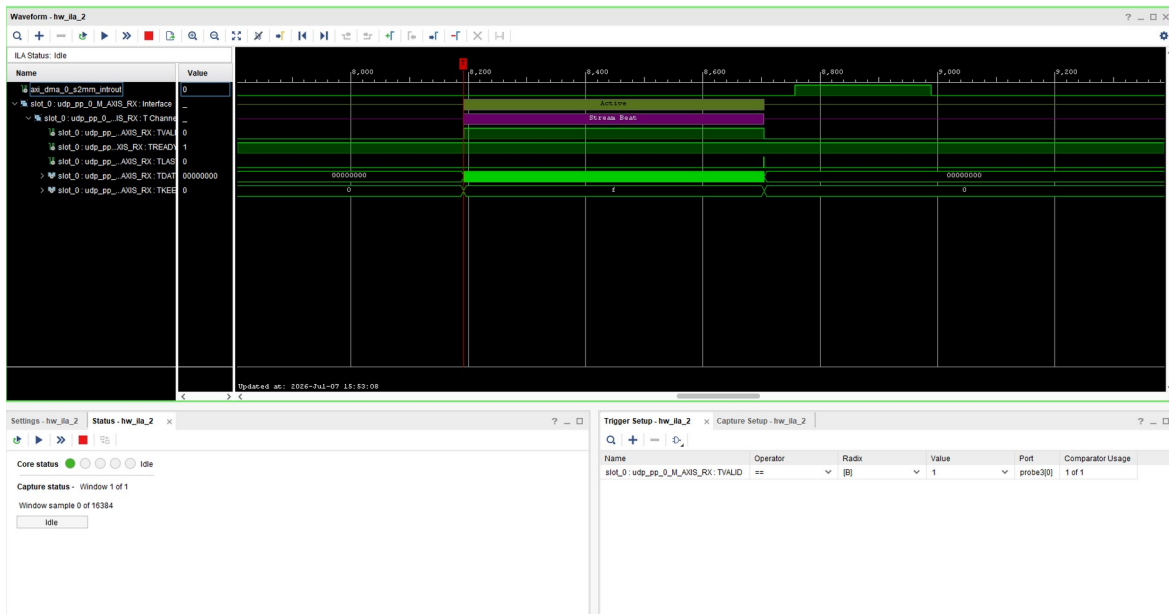


Figure 7.40: RX_UDP_PP_TO_DMA, only payload

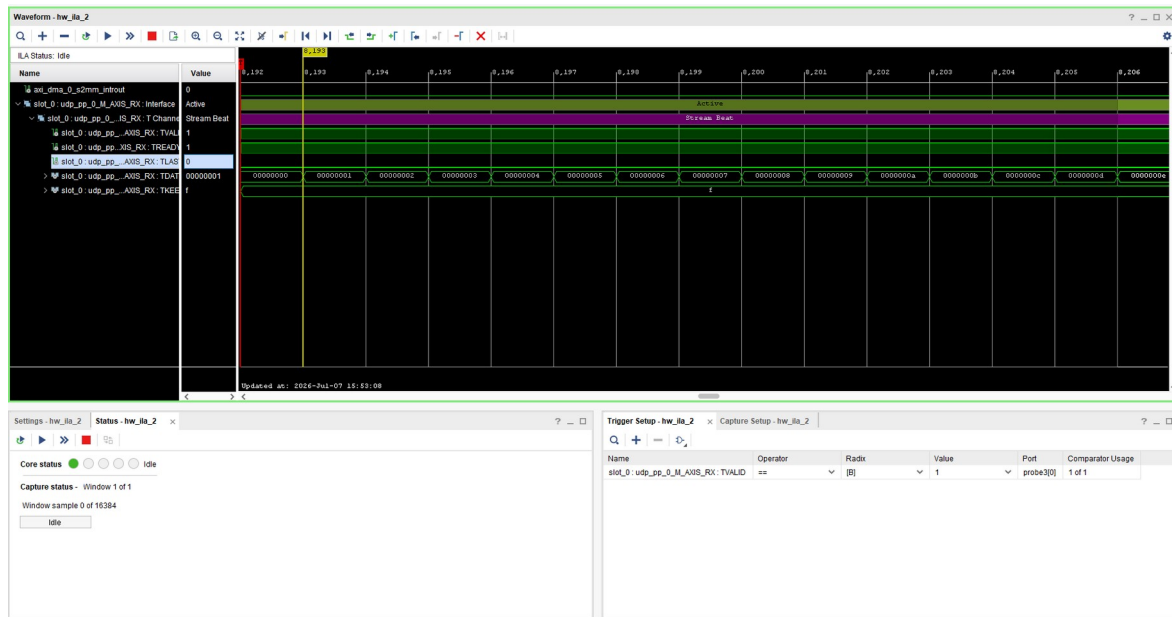


Figure 7.41: Payload zoom in

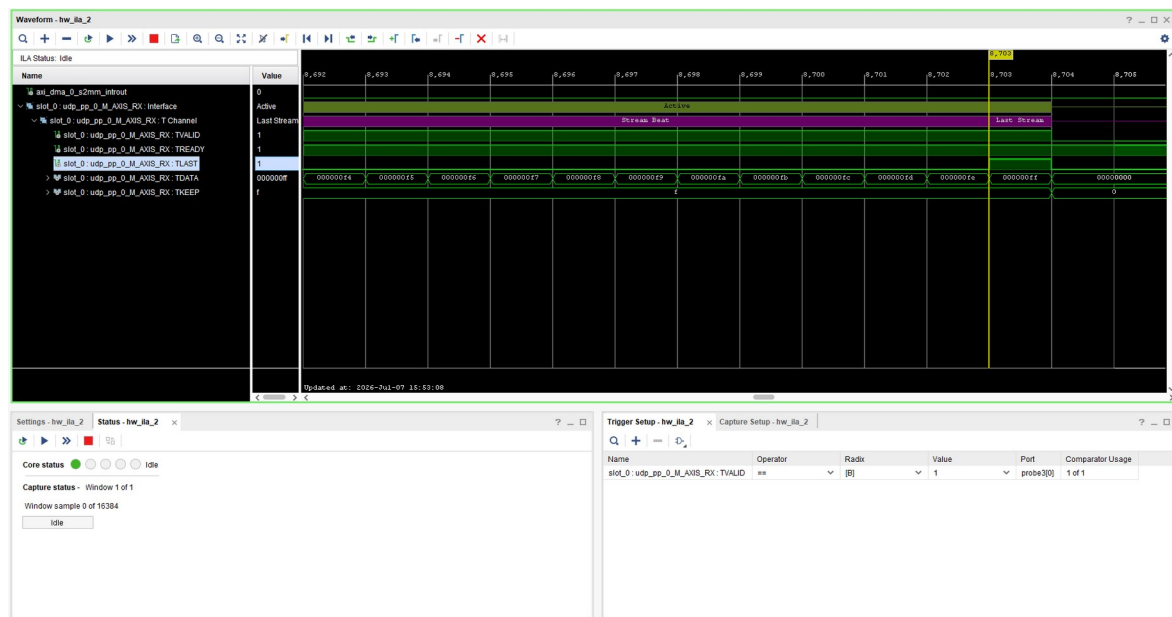


Figure 7.42: RX_UDP_PP_TO_DMA, TLAST rising

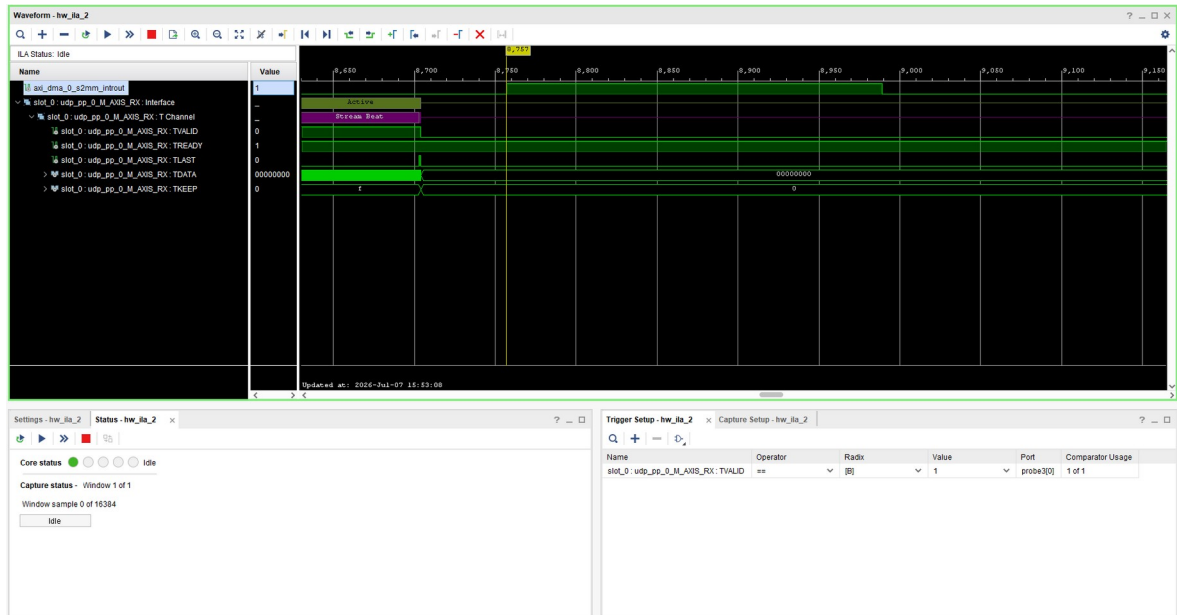


Figure 7.43: RX path, s2mm_introut after delivery

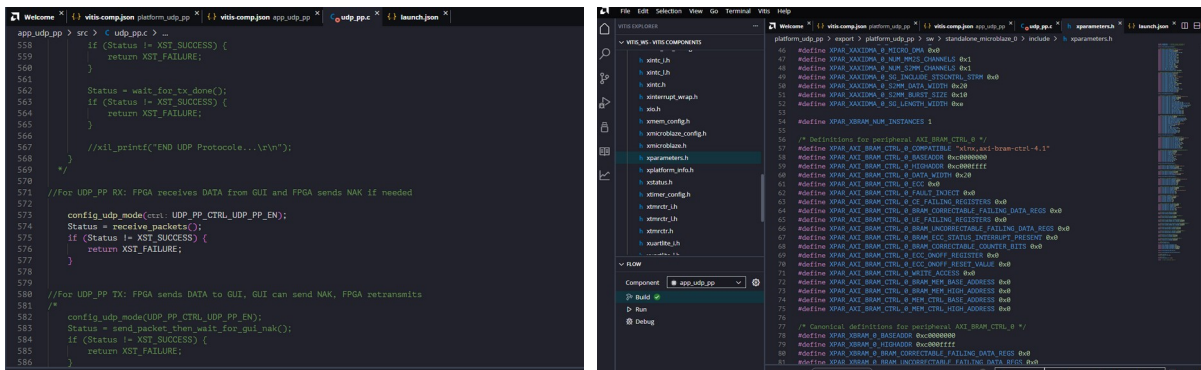


Figure 7.44: Vitis console (left) and xparameters showing memory start address (right)

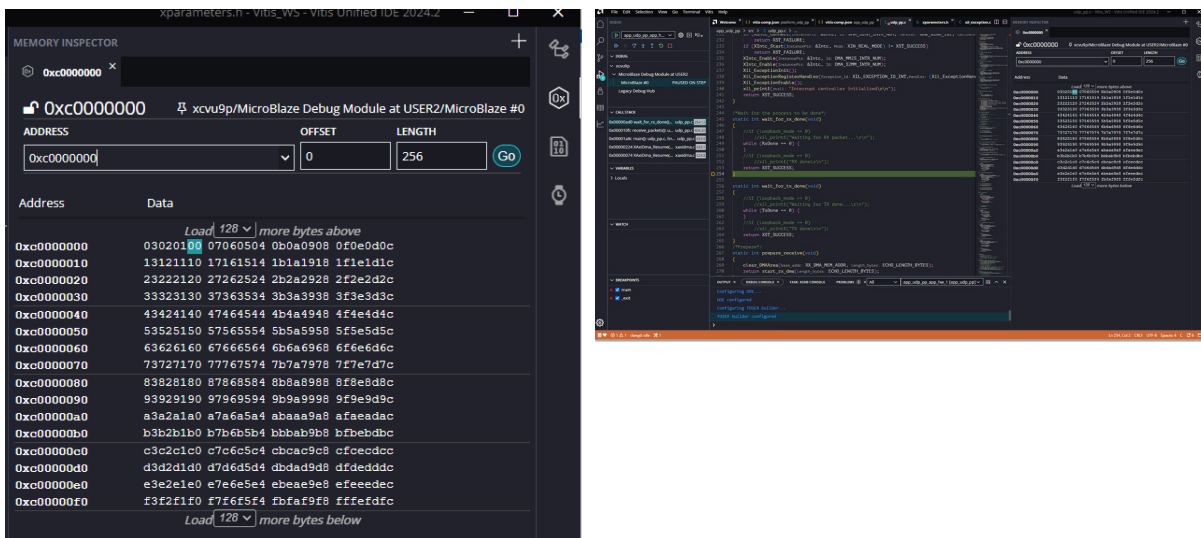


Figure 7.45: Vitis memory views, memory state in debug mode after delivery

7.7.6 RX Side Loss Detection and NAK Generation (Test Case 6)

The final functional test closes the reliability loop in the direction not yet exercised: the FPGA as the loss detector. The host transmitted a numbered sequence in which frame 3 was deliberately withheld (seq 0, 1, 2 sent, then 4 and 5), emulating receive side loss. The hardware gap tracker detected the missing sequence as soon as seq 4 arrived. It buffered the out of order frame and immediately issued a NAK request for sequence 3, which the transmit engine emitted as an 8 byte RETRANSMIT_REQ with opcode 0x02 and session 0x01, without processor involvement. Because seq 5 also arrived while sequence 3 was still missing, another NAK request for sequence 3 was generated. The NAKs are witnessed leaving the engine on the ILA, appearing on the wire in Wireshark, and arriving at the host in the Tester's receive log. The receive engine retained the future frames in its buffer and released payloads toward the DMA strictly in sequence order.

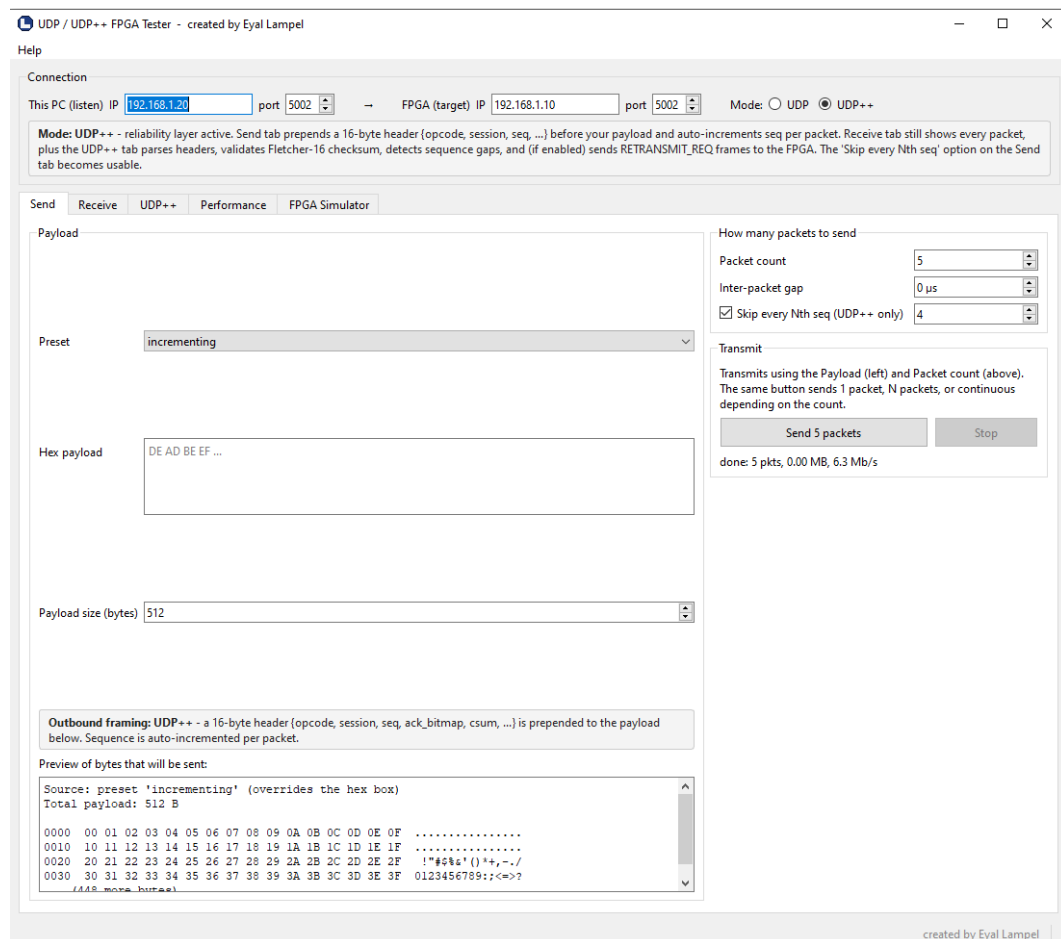
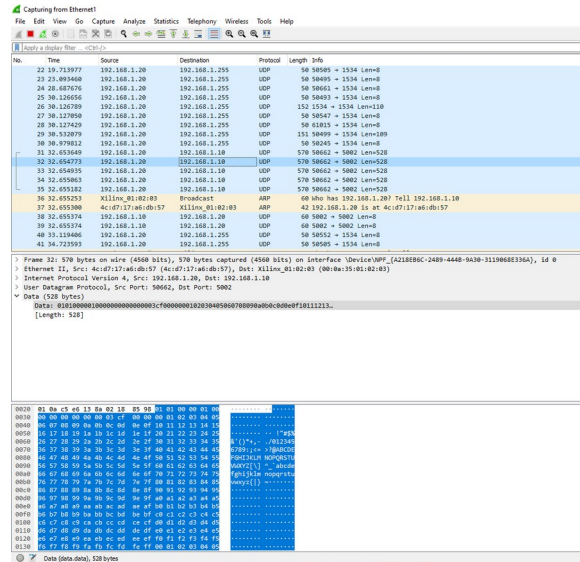
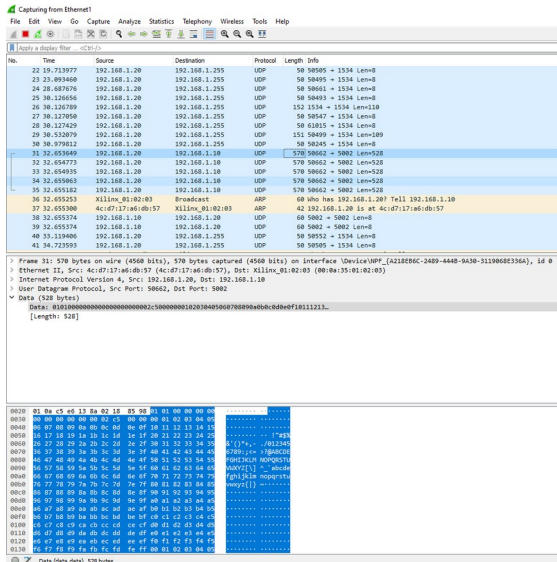
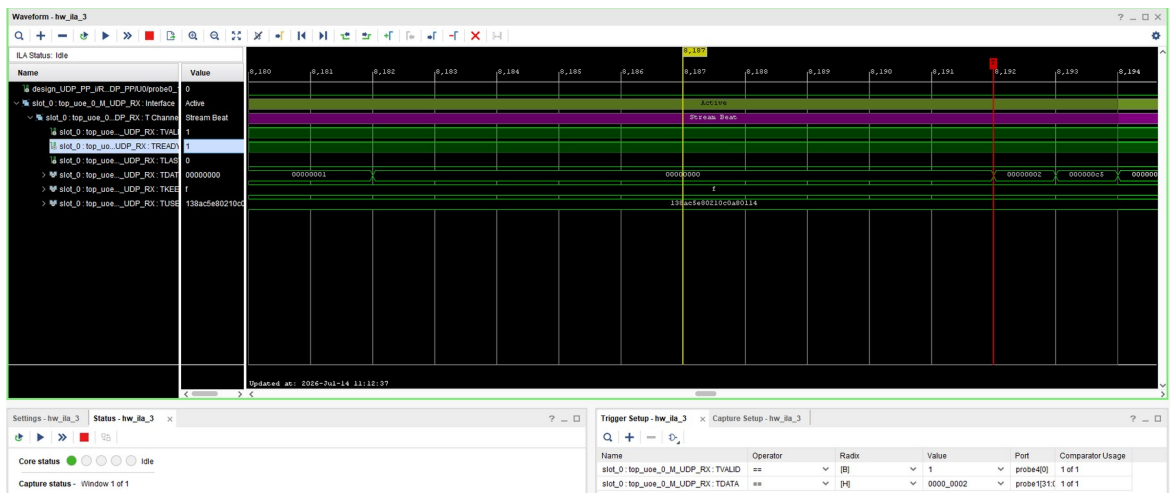
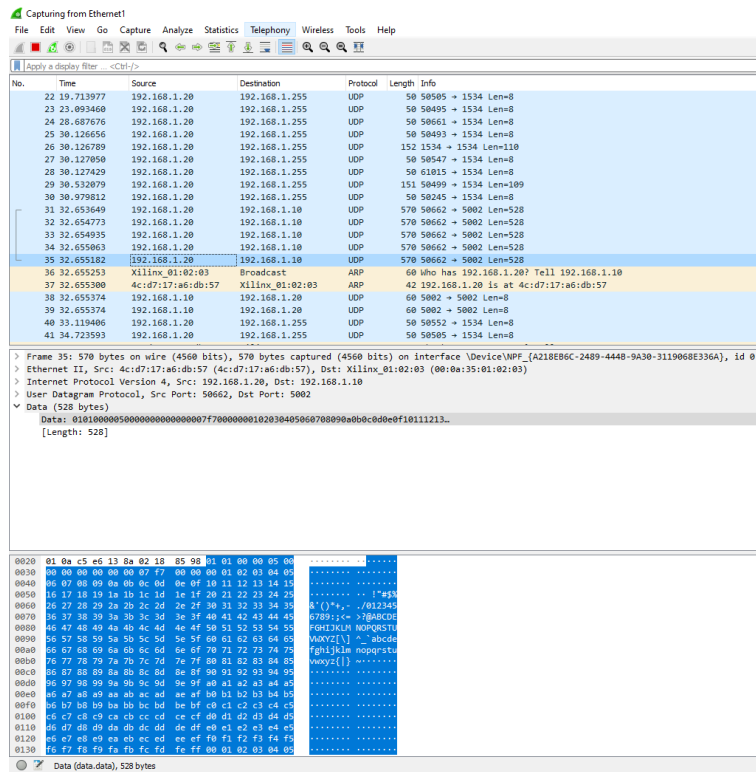


Figure 7.46: Tester Send tab, skip every Nth seq checkbox set to withhold packet 3





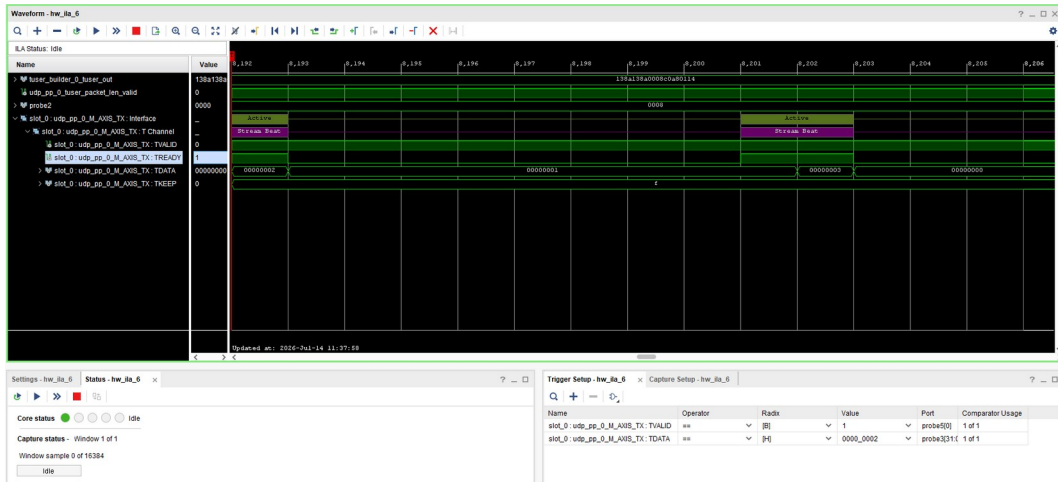


Figure 7.51: TX_UDP_PP_TO_UOE, NAK for seq 3, checksum word

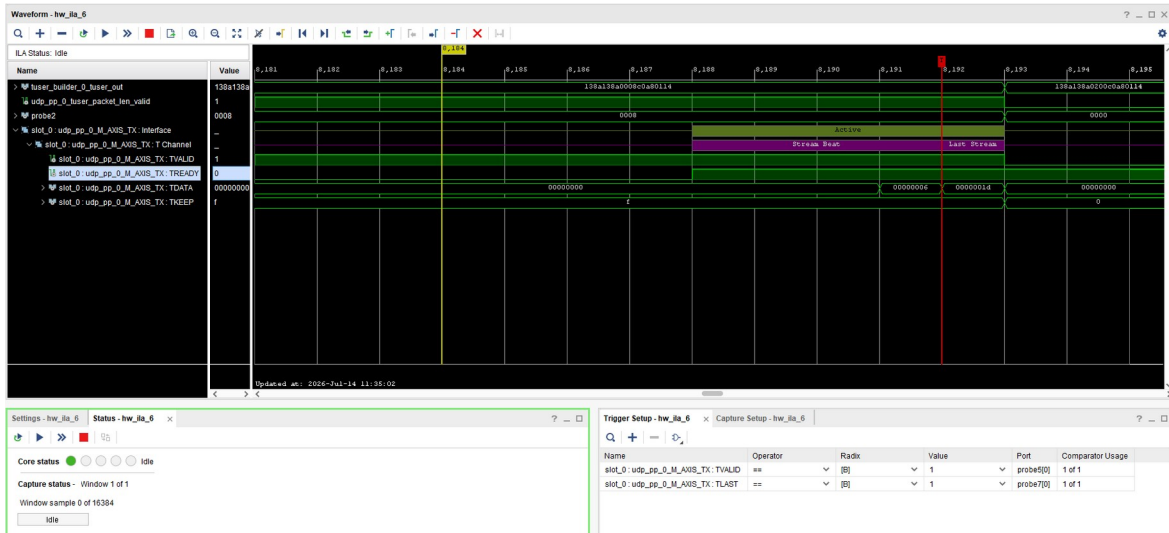


Figure 7.52: TX_UDP_PP_TO_UOE, NAK for seq 3

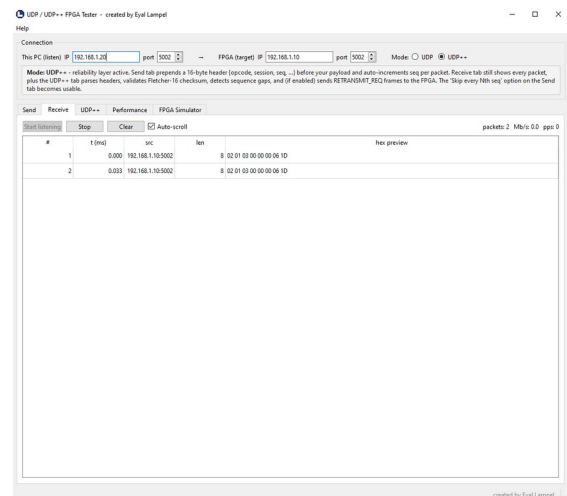
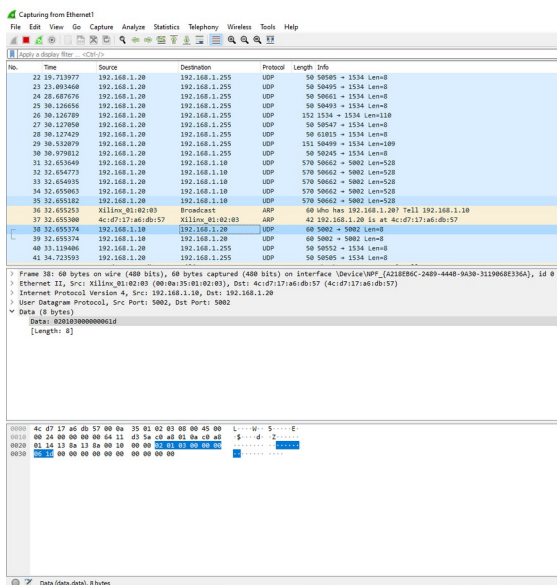


Figure 7.53: NAK for seq 3, Wireshark (left), Tester (right)

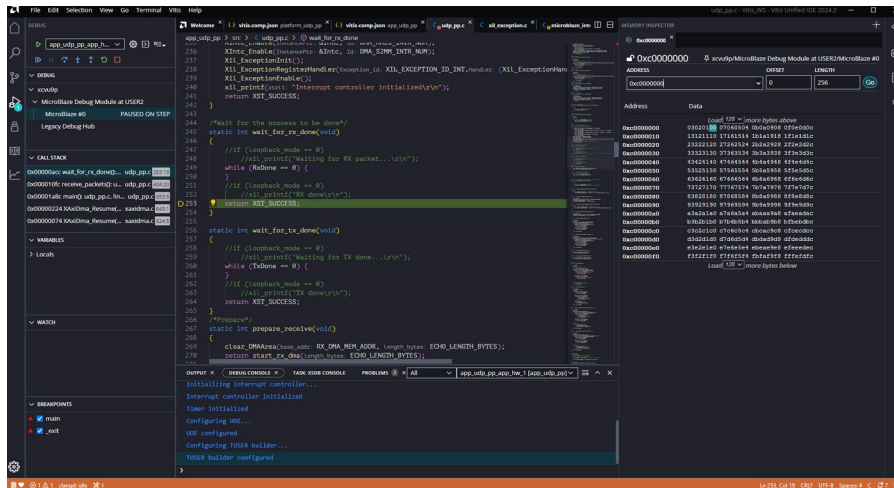


Figure 7.54: Vitis memory view, delivered payloads intact

7.7.7 Resource, Timing, and Power Cost

Figure 7.55 summarizes the post implementation utilization on the VCU118. Relative to the UDP baseline, overall utilization remains exceptionally low, with the total design consuming only 18,976 LUTs (1.61%) and 27,513 flip flops (1.16%). The highest utilization percentage is BRAM at 255.5 tiles (11.83%), reflecting the combined use of the TX history buffer, receive and datapath buffering, baseline system memories, and the deliberately deepened ILA capture memories. All timing constraints are met at the single 125 MHz clock with WNS = +1.418 ns and WHS = +0.010 ns, and total on chip power is 3.111 W.

Summary

Resource	Utilization	Available	Utilization %
LUT	18976	1182240	1.61
LUTRAM	2531	591840	0.43
FF	27513	2364480	1.16
BRAM	255.50	2160	11.83
IO	13	832	1.56
BUFG	9	1800	0.50
MMCM	1	30	3.33
PLL	2	60	3.33

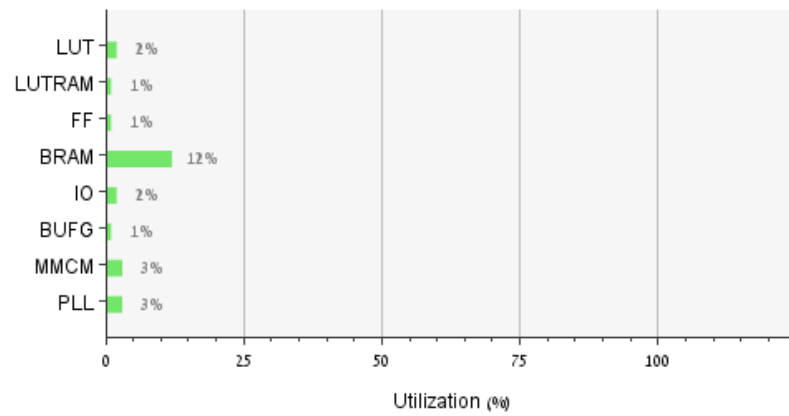


Figure 7.55: Utilization summary with UDP++

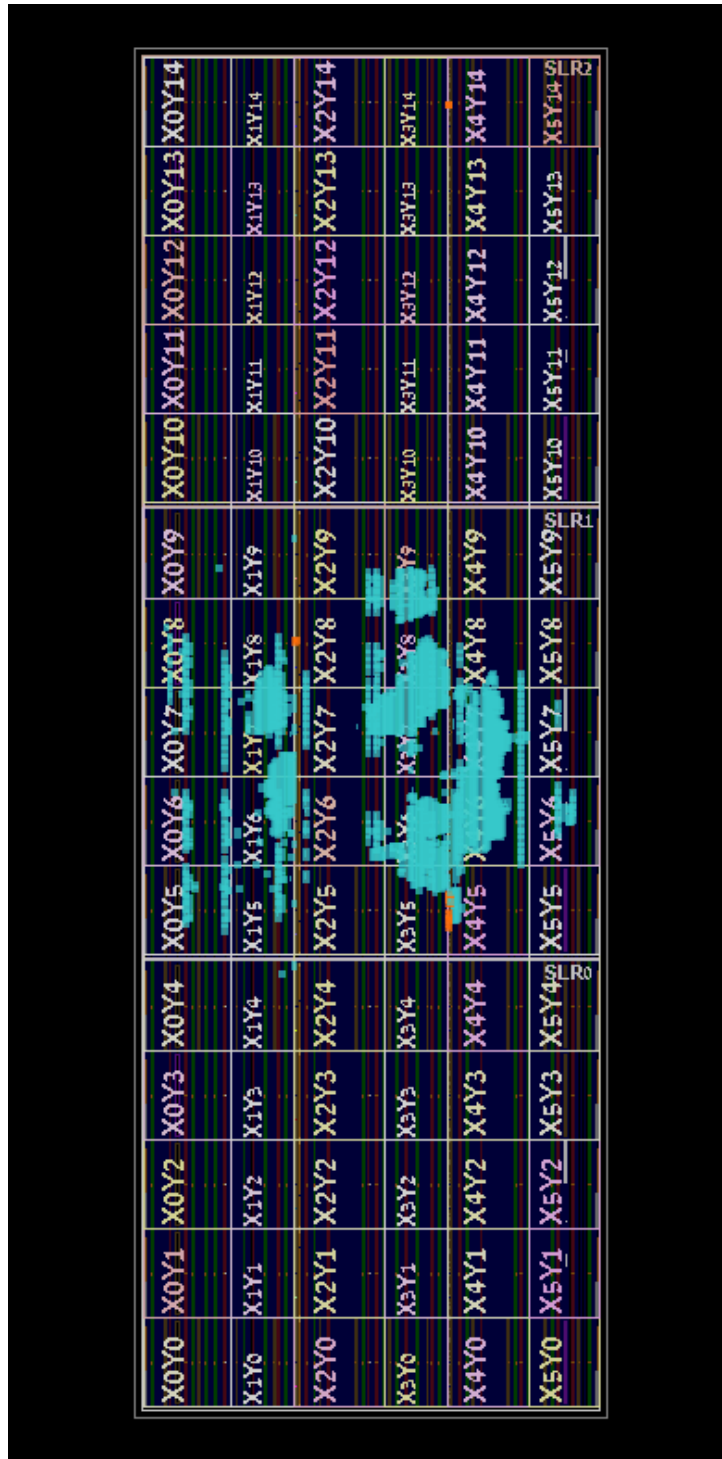


Figure 7.56: Implemented device view, UDP++ design

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.418 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 0.005 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 71535	Total Number of Endpoints: 71497	Total Number of Endpoints: 32270

All user specified timing constraints are met.

Figure 7.57: Timing summary with UDP++, all constraints met

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 3.111 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 26.7°C
 Thermal Margin: 73.3°C (126.0 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 0.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

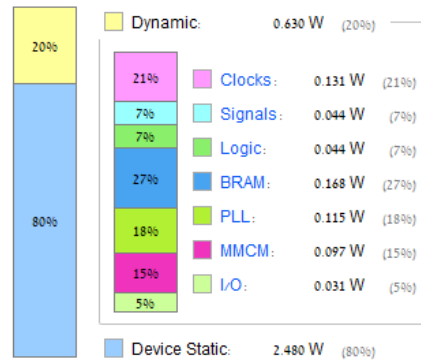


Figure 7.58: Power report with UDP++

```

Vitis Messages platform_udp_pp app_udp_pp:build X
65 C:/Users/user/Desktop/UDP_PP_LAB/UDP_PP_LAB/Vitis_WS/app_udp_pp/src/udp_pp.c:291:12: warning: 'send_packet' defined but not used [-Wunused-function]
66 291 | static int send_packet(void)
67 |     |
68 C:/Users/user/Desktop/UDP_PP_LAB/UDP_PP_LAB/Vitis_WS/app_udp_pp/src/udp_pp.c:271:12: warning: 'prepare_echo_transmit' defined but not used [-Wunused-function]
69 271 | static int prepare_echo_transmit(void)
70 |     |
71 [2/2] cmd.exe /C "cd . && F:\Xilinx\Vitis\2024.2\gnu\microblaze\nt\bin\mb-gcc.exe -O2 -mlittle-endian -mxi-soft-mul -mcpu=v11.0 -DSDT -MMD -MP -specs=C:
72 text data bss dec hex filename
73 18148 654 5558 24360 5f28 app_udp_pp.elf
74
75 Build Finished successfully
76 -----
77 [7/7/2026, 3:35:09 PM]: Build for app_udp_pp::build with id '0bcde90e-075a-4040-86f2-554019c702da' ended.
78
79

```

Figure 7.59: Vitis build log, UDP++ application footprint

7.7.8 Latency and Throughput

Latency and throughput of the complete UDP++ layer were measured with the same instrument, the same 512 byte payload, and the same per packet RTT definition used for the UDP baseline (Sections 4.2 and 6.10), giving a strictly like for like comparison.

Latency. With the reliability layer active in UDP++ loopback mode, all 1,000 probes returned (zero loss) with $p_{50} = 506.9 \mu s$ and $p_{99} = 2,593.2 \mu s$. Against the plain UDP control run's $413.5 \mu s$, the layer adds $93.4 \mu s$ at the median, This end to end increase includes UDP++ checksum validation and generation, receive-side header stripping and buffering, transmit side storage of the payload in the history buffer before transmission, header injection, and the existing DMA and software loopback turnaround. In the measured run the p_{99} tail was shorter ($2,593.2 \mu s$ versus $3,437.1 \mu s$ in the plain UDP control run). This difference should be treated as run to run variation, because the end to end RTT measurement does not isolate the source of the latency tail or the jitter contribution of the FPGA hardware. The Wireshark capture shows the probe stream carrying UDP packets with 528 byte UDP payloads, the 512 byte payload plus the 16 byte UDP++ header, confirming the layer was active end to end throughout the measurement. A

parallel probe in plain UDP mode through the same build shows the familiar baseline latency profile, the plain UDP path is unaffected by the inserted layer and its latency is preserved.

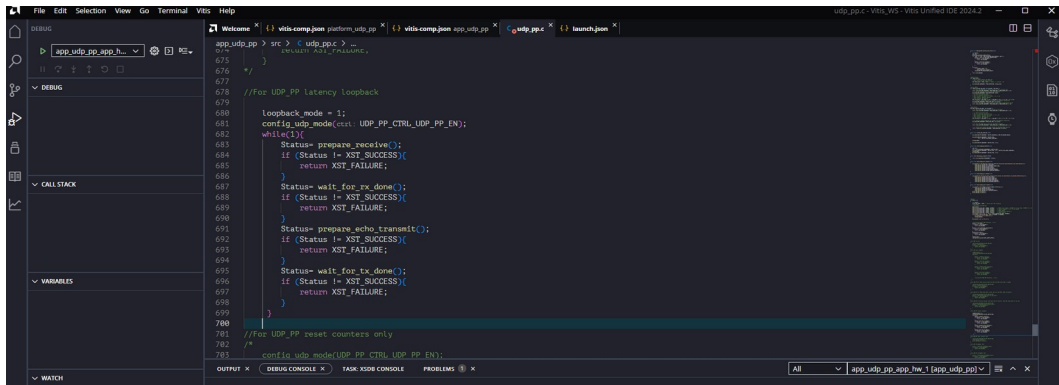


Figure 7.60: Measurement context, *udp_pp.c* loopback branch

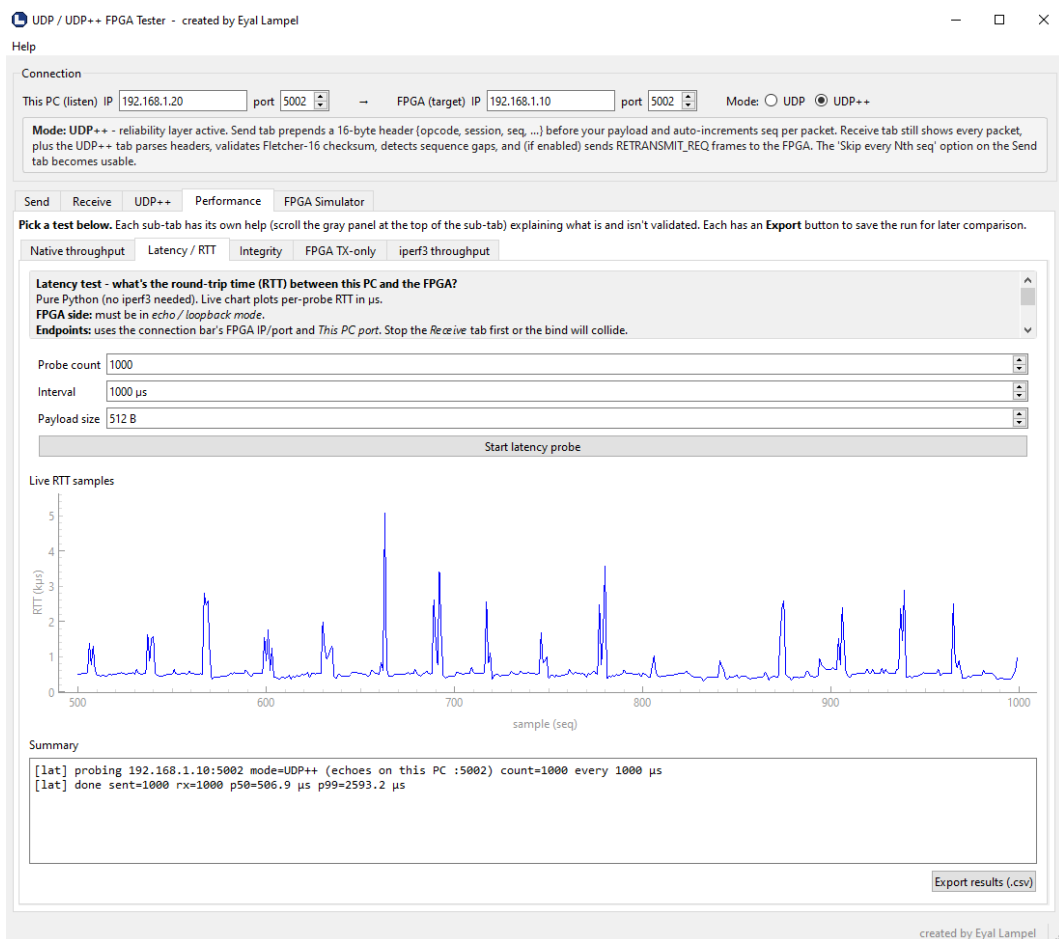


Figure 7.61: UDP++ latency, per packet RTT (512 B)

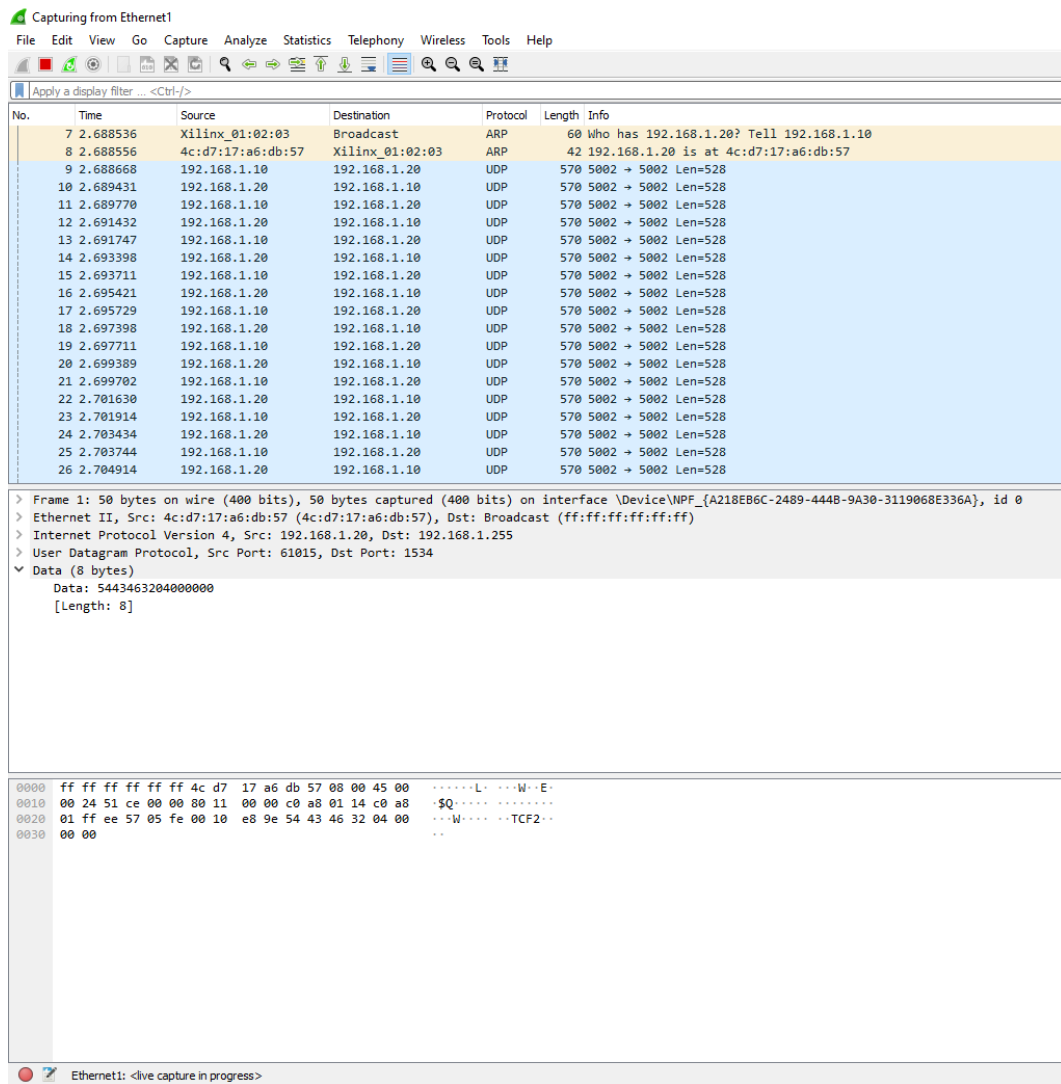


Figure 7.62: UDP++ latency run, 528 B datagrams on the wire

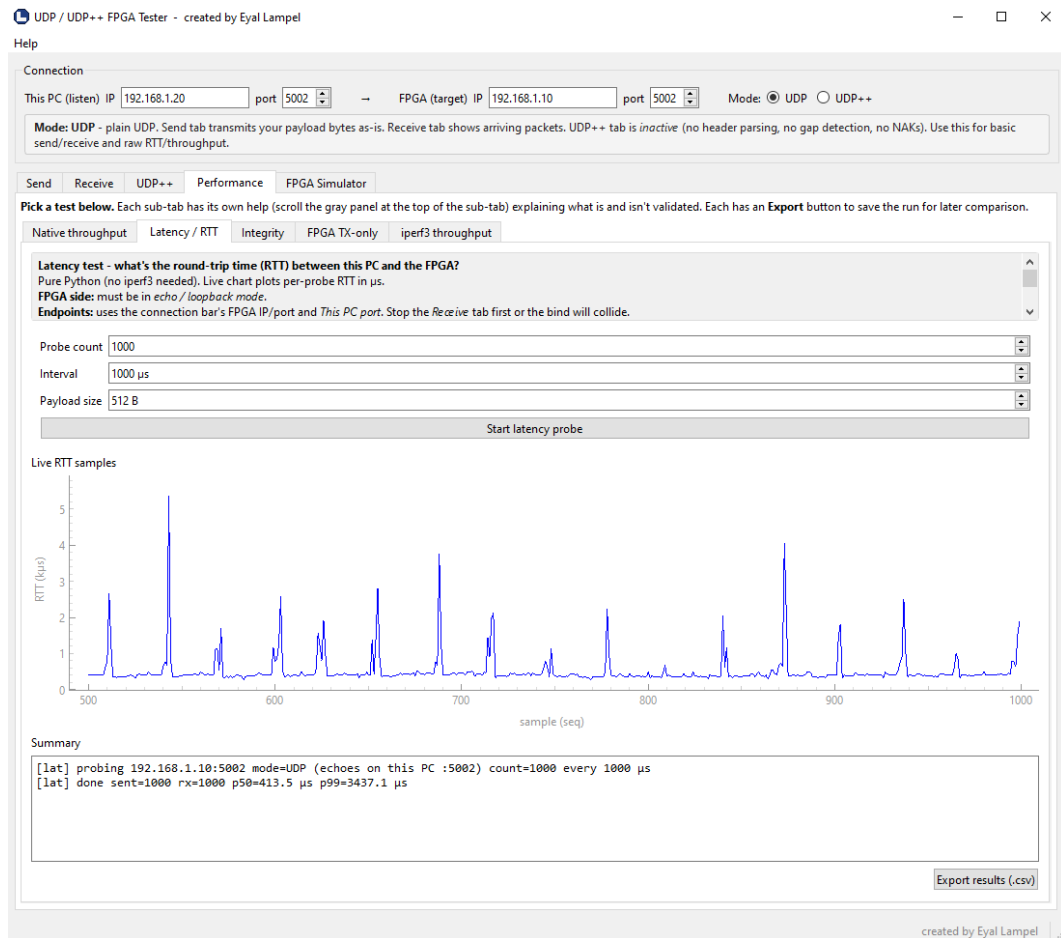


Figure 7.63: Plain UDP latency through the UDP++ build, per packet RTT (512 B), baseline latency preserved

Throughput. Under the native throughput test at 512 B, the UDP++ path sustained TX 700 / RX 692 Mbps with the same flat $\sim 1\%$ raw loss observed on the baseline. Against the baseline's 809 Mbps received at the same payload, the reliability layer costs $\sim 14\%$: 3.1 percentage points are pure header overhead (16 B on 528 B), and the remainder reflects the combined cost of checksum processing, receive side buffering and header stripping, transmit side history buffer storage, header injection, and DMA turnaround within the same 8 bit, 125 MHz datapath. The decisive difference is what the residual loss now means: on the baseline it is unrecovered. Under UDP++, detected gaps can be recovered by NAK driven selective retransmission. Test Case 2 demonstrated end to end retransmission and recovery, while Test Case 6 demonstrated receive side gap detection and autonomous NAK generation.

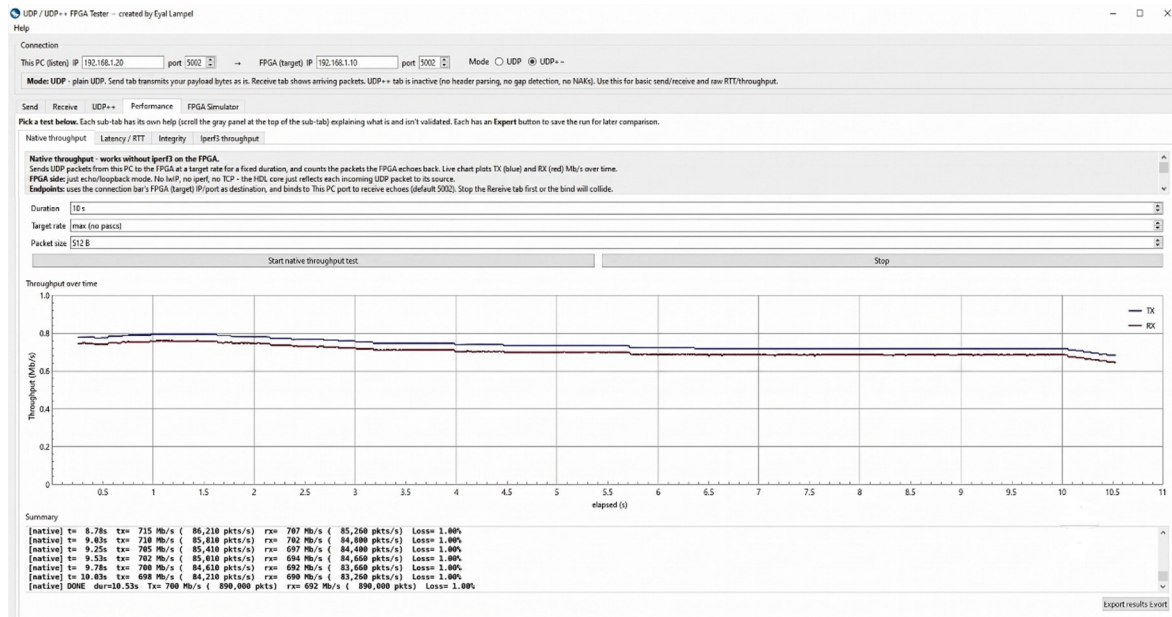


Figure 7.64: UDP++ native throughput (512 B)

8. Summary of Results

To consolidate the measured results, the two baseline servers, TCP and UDP, are first compared head to head. The two servers answer the same question, how fast can a Gigabit link be served from an FPGA platform, with opposite architectures, and their measured results bound the design space that UDP++ must justify itself within.

Axis	TCP baseline (PS)	UDP baseline (PL)
Implementation	ARM Cortex A9 + lwIP 2.2.0 on hard silicon MAC (ENET0), 0% PL	Hardware protocol offload in PL (UOE). MicroBlaze configures the system, manages DMA completion, and copies the payload in loopback mode
Sustained throughput	~940 Mbps (iperf, 300 s continuous)	~850 Mbps average under stress (1,024 B), 809 Mbps received at 512 B
Loss	0 observed, reliability provided by TCP ARQ	~1% under continuous stress (characterized, receive side), 1000/1000 received in latency runs
Latency (p50 RTT, 512 B)	Not instrumented (stack scheduled, OS dependent)	413.1 μ s, dominated by host, deterministic μ s scale FPGA turnaround
Determinism	Software scheduled, jitter from stack and OS	Protocol datapath hardware timed at 125 MHz; end to end echo also includes DMA and MicroBlaze turnaround
Resource cost	0% PL, 3.36 MB application requiring external DDR3	1.36% LUT, 2.43% BRAM, 20,352 B application in on chip memory

Table 8.1: Interim comparison, measured TCP and UDP baselines.

Why the hardware UDP path is slower than the software TCP path. The comparison's apparent paradox, a dedicated hardware engine trailing a software stack by ~90 Mbps, has a precise

architectural explanation. The TCP server runs on the hard silicon Gigabit MAC with wide, direct DDR access. Its data path is the Zynq's optimized hard IP, and lwIP merely keeps a 64 KB window full, letting large in flight bursts amortize every per packet cost. The UDP platform, by contrast, moves every byte through an 8 bit AXI Stream fabric path clocked at 125 MHz, a hard architectural ceiling of exactly 1,000 Mbps gross, before headers, interframe gaps, and turnaround. As an echo server it must also reverse direction for every packet: the S2MM transfer completes and raises an interrupt, the MicroBlaze copies the payload from the receive buffer to the transmit buffer, starts a Simple mode MM2S transfer, waits for its completion interrupt, and then rearms reception. These per packet software and DMA operations, together with the byte serial ceiling, contribute to the measured ~850 Mbps, roughly 85% of the raw 1,000 Mbps path capacity, with zero payload corruption.

The 512 byte native throughput run below makes the accounting visible: transmit pacing of ~817 Mbps against ~809 Mbps received, a flat ~1% loss with no bursts, consistent with a receive side service rate limitation rather than payload corruption or link errors. The gap to the ≥ 900 Mbps specification target is therefore a bounded, understood property of the chosen stream width, with the remediation path (32 bit widening to a 4,000 Mbps ceiling) already identified in Chapter 10. The interim comparison also shows that raw throughput was not the only objective of the PL platform: it provides a hardware timed protocol datapath, reduced processor involvement in packet transfer, and, decisively for this project, a substrate into which a hardware reliability layer can be inserted inline, which the implemented PS based path does not provide.

At a 512 B payload, the UDP baseline achieved approximately 850 Mbps during steady state operation and 809 Mbps as the whole run received average, as shown in Figure 6.17.

The complete measured picture of the three transports is consolidated below. All figures are from this chapter's acceptance campaigns on the delivered hardware. Latency entries use the per packet RTT definition of Section 4.2 at 512 B payload.

Metric	TCP (PS)	UDP (PL)	UDP++ (PL)
Data path platform	Hard MAC + ARM/lwIP	Hardware offload (UOE)	UOE + inline reliability core
Throughput	~940 Mbps (300 s)	~850 Mbps @ 1,024 B, 809 Mbps @ 512 B	692 Mbps @ 512 B
Effective loss	0 (ARQ)	~1% unrecovered	~1% raw loss. successful NAK based recovery demonstrated in the tested in window scenarios
RTT p50 / p99 (512 B)	not instrumented	413.1 μ s / 3,391 μ s	506.9 μ s / 2,593 μ s
PL logic (LUT)	0%	16,067 (1.36%)	18,976 (1.61%)
PL memory (BRAM)	0%	52.5 (2.43%)	255.5 (11.83%)
Software footprint	3.36 MB (external DDR3)	20,352 B (on chip)	24,360 B (on chip)
Timing closure	n/a (no PL)	WNS +1.131 ns	WNS +1.418 ns
Reliability mechanism	TCP ARQ in software stack	none	receiver driven NAK + selective replay in hardware

Table 8.2: Three way tradeoff summary, TCP, UDP, and UDP++ as measured.

The cost of UDP++ can be stated precisely. Relative to the UDP baseline, the complete UDP++ build uses 2,909 additional LUTs (+18.1% over the baseline and +0.25% of the device), increases BRAM utilization from 52.5 to 255.5 tiles (approximately 4.9 $\times$, including the reliability buffers and deepened ILA capture memories), reduces received throughput at a 512

byte payload from 809 to 692 Mbps (approximately 14.5%), and increases median RTT from 413.5 to 506.9 μ s (+93.4 μ s). In return, it adds receiver driven NAK generation and in window selective replay in hardware while retaining sub millisecond median RTT. TCP provides reliable delivery through software stack ARQ, whereas baseline UDP leaves the measured $\sim$ 1% raw loss unrecovered. UDP++ demonstrated successful NAK based recovery in the tested in window retransmission scenarios, with gap detection, NAK generation, and replay control implemented in hardware. The shorter p99 observed in the UDP++ run should be treated as run to run variation rather than an improvement attributable to the reliability layer.

Whether this cost is justified depends on the application, and the measurements define the boundary clearly. For loss intolerant, latency bounded embedded links, such as radar and sensor data distribution, avionics streams, and inter FPGA fabrics, UDP++ provides hardware based loss detection and selective recovery with a median round trip time of approximately 0.5 ms. The complete design uses 1.61% of the device LUTs, while the UDP++ addition accounts for approximately 0.25% of the device LUT capacity. The measured 11.83% BRAM utilization includes the deepened debug ILAs and can be reduced by removing this instrumentation and adjusting the history and receive buffer depths to the application requirements. For bulk host facing transfer, the tested TCP implementation provides the highest throughput, while for loss tolerant streaming the baseline UDP path is faster and uses fewer resources. UDP++ occupies the space between them: the acceptance campaign demonstrated hardware header injection, gap detection, NAK generation, history window hit and miss handling, and selective replay. More specifically, it demonstrated successful in window replay for host issued NAKs, correct rejection of out of window requests, host side tolerance of reordered packets, and FPGA side gap detection and NAK generation under induced receive side loss. The campaign does not establish delivery guarantees under repeated loss of the same frame, loss of the NAK itself, or sustained loss that exceeds the retained history window. Characterizing these cases under controlled impairment is the continuation defined in Section 10.4.

9. Problems and Solutions

Bring up of two independent hardware platforms surfaced a series of integration problems spanning the electrical, memory, clocking, and logic integration domains. Each is documented below in Symptom → Diagnosis → Resolution form. They also document the debug methodology that the instrumentation architecture (Section 6.8) was built to serve: every problem was reduced to an observable signal or report, diagnosed at its root, and closed with a verifiable fix.

9.1 PHY Initialization Failures (Electrical Domain)

9.1.1 I/O Bank Voltage Mismatch: "PHY setup failure"

Symptom. On the Zynq TCP platform, application startup terminated with a "PHY setup failure" report. No Ethernet link was established despite correct cabling and a verified host configuration.

Diagnosis. The failure occurs before any protocol activity, during MDIO based PHY initialization, pointing to the electrical layer rather than the stack. Cross referencing the board schematic against the PS MIO configuration identified the root cause: the Ethernet pins (MIO 16–27) reside in Bank 1, and the on board PHY communicates over RGMII, whose signaling standard mandates 1.8 V. The Vivado default assigns 3.3 V to the bank. At that level the MDIO/RGMII signaling thresholds are violated and the PHY never responds.

Resolution. Bank 1 I/O voltage was set to LVCMOS 1.8 V in the PS MIO Configuration (Bank 0 retained at 3.3 V for the QSPI flash). PHY initialization completed on the next boot, with the terminal confirming "Link speed for phy address 0: 1000". The setting was documented as a mandatory board adaptation parameter (Section 5.3).

9.1.2 PHY Latched in Reset at Power Up

Symptom. Intermittently at power on, the host NIC reported no carrier (red X link indicator) and the terminal reported auto negotiation errors, with no configuration change between working and failing boots.

Diagnosis. Intermittency at power up with a correct configuration indicates a reset sequencing race rather than a settings error: the PHY device could remain latched in its reset state ("frozen")

if its reset line was not deterministically released. The board routes the PHY reset to a physical push button on package pin V13, a signal the initial design did not constrain or drive.

Resolution. Two level fix. In hardware, the reset port was bound explicitly in the XDC (PACKAGE_PIN V13, IOSTANDARD LVCMOS18), placing the release under design control. In lab procedure, a documented recovery step was added: pressing SW1 restrobes the PHY reset and forces a clean 1000 Mbps handshake. The failure mode did not recur under the constrained design.

9.2 Memory and Linker Constraints (Software Domain)

9.2.1 TCP Application Footprint vs. On Chip Memory

Symptom. Not a runtime failure but a design blocking constraint: the TCP server could not be hosted in on chip memory.

Diagnosis. The Vitis build log quantified the issue precisely, application footprint of 3,361,542 B (dec column) against ~600 KB of total Zynq BRAM, a $5.6\times$ deficit before allocating the 64 KB TCP windows and 1024 entry pbuf/segment pools that Gigabit operation requires (Section 5.5). The diagnosis method is worth recording: reading the linker output before attempting hardware bring up surfaced the constraint at design time, before it could appear as a runtime crash.

Resolution. Code and network buffers were mapped to external DDR3, with the DDR controller bound to the exact on board device (MT41K256M16 RE-125). The decision was retrospectively validated by the 940 Mbps sustained result (Section 5.6): DDR access latency proved immaterial for a windowed, throughput oriented protocol.

9.3 Clocking and Synchronization (Timing Domain)

9.3.1 SGMII Reference Clock Mismatch: 625 MHz Board Preset vs. 125 MHz

Design

Symptom. On the VCU118 platform, the PCS/PMA core failed to achieve lock. No PHY layer readiness was indicated and no traffic could flow, despite a logically validated design.

Diagnosis. The phy_enable ILA (Section 6.8) localized the failure to the physical layer: lock signals persistently low while all upstream logic was healthy, isolating the problem to the PCS/PMA clocking. Configuration review revealed the root cause: associating the core with the

VCU118 board preset silently selects a 625 MHz SGMII reference clock, while the design's entire clock tree, deliberately unified at 125 MHz (Section 6.4), supplies 125 MHz. With a 5× frequency mismatch at its reference input, the core's clock recovery cannot lock.

Resolution. The board association for the reference clock was overridden: DIFFCLK set to Custom and the LVDS reference clock explicitly configured to 125 MHz in Core Functionality. Lock was achieved on the next bitstream, confirmed by the phy_enable ILA showing TX/RX lock and PHY_LAYER_RDY at '1'. The episode is documented as the rationale for manually pinning clock parameters rather than trusting board presets in a custom clocking architecture.

9.3.2 Auto Negotiation Fragility on the TCP Platform

Symptom. Occasional link establishment failures or degraded speed negotiation on the Zynq platform, correlated with cable seating and transient line conditions rather than with any configuration change.

Diagnosis. With the electrical and reset issues of Section 9.1 resolved, residual failures clustered in the auto negotiation phase itself, a probabilistic exchange vulnerable to momentary line noise. In a fixed two node bench whose both endpoints are known Gigabit capable, auto detection adds a failure mode while providing no benefit.

Resolution. The negotiated speed was pinned in the BSP: `temac_phy_link_speed` changed from `CONFIG_LINKSPEED_AUTODETECT` to `CONFIG_LINKSPEED1000` (Section 5.5), complemented procedurally by verifying full connector engagement. Link establishment became immediate and deterministic at 1000 Mbps across all subsequent test sessions.

9.4 Integration Defects (Logic Domain)

9.4.1 Unconnected DMA Clock Inputs After Connection Automation

Symptom. Validate Design failed following DMA integration, reporting the `m_axi_mm2s_aclk` and `m_axi_s2mm_aclk` inputs unconnected, despite Run Connection Automation having been executed for the block.

Diagnosis. Direct inspection of the validation report and the block design confirmed the automation's coverage gap: it had wired the AXI4 Lite control path but left the two memory side clock inputs floating. A companion warning flagged `S_AXIS_S2MM` as dangling. Topology review classified this one as expected, that interface connects to the UOE only at a later

integration stage, illustrating the necessary discipline of separating genuine defects from staged integration artifacts in validation output.

Resolution. Both clock inputs were tied manually to the central 125 MHz source, synchronizing the DMA's read and write channels with the rest of the fabric. Revalidation passed with only the anticipated S_AXIS_S2MM notice, which cleared upon UOE connection.

9.4.2 Reset Polarity Inconsistency Across the UOE Boundary

Symptom. Risk class defect identified at integration: the UOE's internal logic requires active high reset ($G_ACTIVE_RST = 1$), while the surrounding AXI infrastructure operates active low (aresetn). Left implicit, the mismatch produces a core held permanently in, or never entering, reset, a failure mode that manifests only in hardware and is costly to trace.

Diagnosis. Identified during IP packaging by reviewing the core's generics against the system reset scheme, before the mismatch could manifest in hardware.

Resolution. Enforced at three levels: the reset interfaces (RST_UOE, RST_RX, RST_TX) were explicitly parameterized $POLARITY = ACTIVE_HIGH$ inside the IP metadata, preventing Vivado from inferring inversions. A dedicated Processor System Reset block supplies the active high peripheral_reset to the UOE, and Utility Vector Logic NOT gates handle the remaining polarity crossings (PCS/PMA reset, external PHY1_RESET_B). The locked gated reset scheme (dcm_locked from the Clocking Wizard into all three reset blocks, Section 6.4) completes the deterministic release sequence.

9.4.3 Mid Transmission Metadata Corruption Risk: TVALID/TLAST

Coordination

Symptom. Risk class defect in the transmit path: the UOE stamps each outgoing packet with routing metadata from the 80 bit TUSER vector, sourced from processor writable registers. A register update landing mid packet would corrupt the in flight header, an intermittent, traffic dependent fault of the worst diagnostic class.

Diagnosis. Identified analytically from the AXI Stream protocol's structure: the payload channel is hardware timed at 125 MHz while register writes are asynchronous software events. Absent an explicit synchronization contract, the race is real. A secondary obstacle compounded it, the handshake signals needed to detect packet boundaries (TVALID, TLAST, TREADY) are point to point by protocol definition and thus invisible to a third party block.

Resolution. Two cooperating custom blocks (Section 6.7): the Splitter, inserted on the DMA → UOE path, passes the stream through unmodified while exporting replicated handshake copies. The tuser_builder consumes these to latch its register contents at start of packet, freezing the metadata for the packet’s duration. The race is thereby eliminated at the structural level. Correct latching was verified in hardware via the TX_UDP ILA’s 80 bit TUSER probe under both deterministic and stress traffic (Sections 6.9, 6.10).

9.5 External Constraint: Laboratory Access Disruption

One project level problem was environmental rather than technical. Following the outbreak of the war, physical access to the Elta facilities was restricted to essential personnel, suspending access to the laboratories and the FPGA hardware for an extended period. Compounded with the examination period unaccounted for in the original schedule, this produced the delay documented in the progress report. The mitigation applied the same engineering logic as the technical fixes: work was repartitioned so that everything separable from silicon, architecture finalization, Vivado/Vitis environment configuration, HDL development, and simulation level verification, proceeded remotely, with hardware dependent verification batched for restored access. Schedule impact analysis appears in Section 10.2.

10. Conclusions and Recommendations

10.1 Achievement of Project Goals

The project's goals, as defined in the PDR and preliminary design [5], are assessed individually against the delivered results:

Goal 1, Implement and evaluate TCP and UDP servers on FPGA, establishing performance and functionality baselines. Achieved, and exceeded in architectural scope.

The preliminary design envisioned both baselines as lwIP applications on a single Zynq platform. The delivered system realizes them as two architecturally distinct implementations, a PS based software stack and a full PL hardware offload, turning the baseline pair itself into a controlled comparison of the two fundamental design strategies (Section 3.3). Both servers are functional, verified at signal level, and quantitatively characterized.

Goal 2, Achieve ≥ 900 Mbps throughput on a 1 GbE link. Achieved on TCP. Approached on UDP (94% of target). The TCP server sustained ~ 940 Mbps over a 300 second continuous run, effective saturation of a Gigabit link's practical goodput ceiling (Section 5.6). The UDP data path measured ~ 850 Mbps average (TX 860 / RX 851 Mbps) under continuous stress (Section 6.10).

The UDP figure warrants an honest engineering assessment. First, its context: ~ 850 Mbps was achieved through a deliberately byte serial data path, 8 bit stream width at 125 MHz, an architectural ceiling of 1,000 Mbps raw before interpacket gaps and DMA turnaround, with UDP processing performed in hardware, while the MicroBlaze controls the DMA and copies the payload between the receive and transmit buffers in loopback mode. For a MicroBlaze based PL implementation with a single byte per cycle pipeline, this operating point confirms the offload thesis: throughput is governed by the hardware path. Second, the source of the gap is understood: the stable $\sim 1\%$ loss with TX consistently exceeding RX by ~ 9 Mbps is the signature of a receive side service rate limit at burst peaks (Section 6.10). Third, the closure paths are identified and quantified in principle:

- **Stream widening**, migrating the DMA $\leftrightarrow$ UOE path from 8 bit to 32 bit width at 125 MHz raises the architectural ceiling to 4,000 Mbps, converting the current margin less design into one with $4\times$ headroom. The TEMAC boundary then becomes the sole byte rate constraint.

- **Deeper UOE pipelining and transmit side buffering**, decoupling echo turnaround from reception pacing, targeting the diagnosed receive side limit directly.
- **DMA transaction batching**, amortizing per packet register programming overhead, the residual control plane cost in the current Simple mode driver.

Goal 3, Comparative analysis under controlled network conditions. Achieved.

The acceptance campaigns of Chapters 5 through 7 measured TCP, UDP, and UDP++ on a common set of axes, throughput, loss, and resource cost, with per packet RTT measured on the two PL transports, on the delivered hardware over the controlled point to point bench (Chapter 8 and Section 7.7), and the consolidated analysis of Chapter 8 quantifies the cost of hardware reliability against both baselines. Two boundaries of this comparison are stated explicitly: TCP latency was not instrumented (Table 8.1), and since the two baselines run on different platforms, the TCP versus UDP comparison is between complete platform architecture combinations rather than between isolated protocols, a direct consequence of the dual platform decision of Section 3.3. Extension of the campaign to impaired network conditions (controlled loss, jitter, and RTT via netem) remains the natural continuation.

Goal 4, Investigate FPGA design methodologies for networking applications. Achieved.

The project produced transferable methodology beyond its numerical results: reproducible configuration practices, including Tcl enforced IP packaging (Section 6.5). Observability as an architectural requirement (the five core ILA array, Section 6.8), and a documented Symptom → Diagnosis → Resolution debug discipline (Chapter 9).

Goal 5, Deliver working prototypes, measurement framework, and documentation. Achieved.

The delivery comprises two verified hardware prototypes, the UDP++ reliability layer implemented and accepted on the PL platform (Sections 7, 7.7), the complete measurement and verification environment, and this report.

10.2 Comparison: Plan vs. Execution

Scope. Executed scope matches the preliminary definition, with the documented platform migrations (Sections 5.2 and 6.2), engineering necessities that preserved, and in the UDP case strengthened, the project goals. No planned deliverable was descoped. The UDP++ layer was implemented, integrated, and accepted on the PL platform as planned (Sections 7, 7.7).

Schedule. Execution deviated from the original Gantt for two reasons analyzed in the progress report and Section 9.5: an underweighted examination period and, dominantly, the suspension

of physical access to the Elta facilities after the outbreak of the war, which severed access to the FPGA hardware for an extended period. The deviation was actively managed: work was repartitioned along the hardware dependency boundary, architecture, HDL, environment configuration, and simulation advanced remotely. Silicon dependent verification was batched for restored access. Section 9.5 justifies the deviation as a force majeure constraint outside the foreseeable risk set, with bounded impact and both critical path deliverables, the two verified baselines, completed.

Budget. The project consumed no procurement budget beyond the preliminary estimate: both hardware platforms (the custom Zynq board and the VCU118), the toolchain licenses, and the laboratory infrastructure were supplied through the Elta environment as planned. Effort hours constitute the dominant actual cost, elevated relative to plan in proportion to the schedule extension.

Table 10.1 details the final cost accounting against the preliminary estimate. While hardware costs were fully absorbed by the sponsor, the extended remote simulation phase increased total student effort hours from the planned 1,800 to 2,200.

Item	Planned Cost (ILS)	Actual Cost (ILS)	Notes
Supervisor Labor	2,000	2,000	40 hours
Students' Labor	142,000	173,800	Increased from 1,800 to 2,200 hrs due to lab access delays
Hardware Boards	1,150	0	Migrated to VCU118 & Custom Zynq, fully supplied by Elta
Accessories (JTAG, cables)	110	110	
Overhead (10%)	14,526	17,591	
Total Estimated Cost	159,786	193,501	Reflected entirely in effort hours, no procurement deviation

Table 10.1: Final Budget Accounting (Plan vs. Actual)

10.3 Advantages and Disadvantages of the Delivered Architecture

Advantages:

- **Hardware protocol data path (PL platform).** UDP/IP packet processing is performed in hardware at 125 MHz, while the MicroBlaze manages DMA completion and copies payload data between the RX and TX buffers in loopback mode. The PL resident AXI Stream path enables insertion of the inline hardware reliability layer.
- **Near line rate software baseline at zero fabric cost (PS platform).** 940 Mbps from hard silicon and a tuned open stack, with 0% PL utilization, a rapidly reproducible reference point and a fully free fabric.
- **Resource economy with engineered headroom.** ~1.36% LUTs / ~2.43% BRAM on the VCU118 (Section 6.11), retiring the preliminary design's resource exhaustion risk and leaving the platform open for UDP++ and beyond.
- **Race free software/hardware contract.** The tuser_builder latching mechanism uses the exported TX handshake signals to freeze the routing metadata for the duration of each packet, eliminating the metadata corruption race by construction (Sections 6.7, 9.4.3), a reusable pattern for any processor configured streaming design.
- **Built in observability.** The five core ILA array renders every data path boundary inspectable in hardware, making internal behavior directly observable during debug.

Disadvantages:

- **Development and verification cost.** The PL implementation required custom IP packaging, VHDL 2008 integration handling, clock and reset engineering, and the development and integration of multiple custom blocks, including tuser_builder, Splitter, and udp_pp, work that the PS based lwIP implementation did not require.
- **Debug opacity absent instrumentation.** Internal high speed fabric signals are not directly observable during hardware operation without on chip instrumentation. The ILA array therefore had to be designed into the system, consuming development time and substantial BRAM in the deep capture configuration.
- **Byte serial throughput ceiling.** The 8 bit stream width bounds the current UDP path below full line rate under stress, a known, quantified limitation with identified remedies (Section 10.1).

- **Platform heterogeneity.** Two boards and two toolflow variants increase maintenance surface relative to the single platform plan, the accepted cost of the architectural comparison the dual platform structure enabled.

The UDP++ results sharpen this ledger into a quantified verdict. The complete UDP++ build uses 1.61% of the device LUTs and 11.83% of its BRAM, while the UDP++ addition accounts for approximately 0.25% of the device LUT capacity relative to the UDP baseline. The BRAM total includes the reliability buffers, baseline system memories, and the deliberately deepened ILA capture memories. The architecture adds hardware based gap detection, NAK generation, and selective in window replay to the PL protocol path, with successful recovery demonstrated in the tested retransmission scenarios and a measured median RTT increase of 93.4 μ s (Section 8). For applications that cannot tolerate packet loss but still require a low-latency, hardware timed protocol path, UDP++ provides a middle ground by recovering detected in window losses in hardware while retaining a median RTT of approximately 0.5 ms. For bulk transfers requiring reliable delivery, the TCP baseline remains the appropriate choice. For loss tolerant streaming, the UDP baseline remains the appropriate choice. The project therefore provides three alternatives across its two delivered platforms, each suited to different application requirements.

10.4 Recommendations and Future Work

The recommendations are ordered by their proximity to the project's defined trajectory:

1. **Hardening and scaling of the delivered UDP++ layer**, With the reliability layer implemented, integrated, and accepted (Sections 7, 7.7), the next step is productization: removing or reducing the deep debug ILA capture memories, sizing the TX history and receive buffers to the application requirements, and exposing the buffer depths and maximum payload size as synthesis time generics while retaining the session identifier as a run time AXI Lite parameter.
2. **Extension of the comparative campaign to impaired networks**, reproducing the three way measurements of Chapters 5 through 7 under controlled loss, jitter, and RTT (netem). The clean bench results bound the expectation: The campaign should determine whether UDP++ improves effective delivery as loss increases and quantify its RTT and jitter under impairment. Gap detection, NAK generation, and replay remain in hardware, although end to end measurements still include host, NIC, and operating system effects.

3. **UDP data path throughput closure**, stream widening to 32 bits, transmit side buffering, and DMA batching per Section 10.1, targeting ≥ 900 Mbps on the UDP path independently of the reliability work.
4. **Reproducibility hardening**, consolidating the build into a fully scripted TCL first flow (project creation through bitstream and platform generation), extending the Tcl enforced packaging discipline already applied to the UOE.
5. **Platform scaling**, leveraging the UltraScale+ fabric toward the 10 GbE stretch goal noted in the preliminary design [5]. The widened stream architecture of item 3 is an intermediate step, while a 10 GbE implementation would require a wider and/or faster datapath together with a suitable 10 GbE MAC and PHY interface.

Closing assessment. The project set out to quantify the speed reliability tradeoff on real FPGA hardware and to build the foundation for resolving it. It delivered two verified Gigabit class implementations embodying the two competing architectural philosophies, a software stack running on hard silicon, and a PL based hardware protocol offload with MicroBlaze controlled DMA loopback, characterized them in detail, and closed every integration problem encountered on the way with documented root cause resolutions. For continuation at Elta, the offload platform was engineered from the outset as a substrate for further work: every data path boundary is observable, the software contract is race free, and more than 98% of the LUT capacity remains available, while the instrumented build uses 11.83% of the BRAM capacity. The UDP++ layer, and the protocol research it enables, inherits a foundation that has already been proven at speed. With the UDP++ layer implemented, verified at signal, wire, and application level, and measured against both baselines, that foundation has been built upon and validated: the speed reliability tradeoff is now quantified across all three transports on real hardware.

11. References

- [1] "LwIP TCP Perf Server using VITIS 2024," TCP Performance Server implementation documentation, project p-2026-142 technical archive, 2026.
- [2] "UDP Ethernet Performance Server using VITIS 2024.2 for VCU118 Board," UDP Echo Server implementation documentation, project p-2026-142 technical archive, 2026.
- [3] B. Zhang, B. Yan, M. Liu, B. Hu, A. Xu and K. Wang, "Development of an Industrial Real time Ethernet Protocol Stack Based on SoC," 2022 International Symposium on Networks, Computers and Communications (ISNCC), Shenzhen, China, 2022, pp. 1–6.
- [4] School of Electrical and Computer Engineering, Ben Gurion University of the Negev, "Fourth Year Engineering Project Procedure (נוהל פרויקט הנדסי שנה ד'), Academic Year 2026," Ed. 2020-1.
- [5] O. Elijah and Y. Tayri, "Reliable High-Speed Networking on FPGA Platforms (UDP, TCP, and UDP++ Protocols), PDR and Preliminary Design Report," project pre-2026-142, Ben Gurion University of the Negev, Nov. 2025.
- [6] AMD/Xilinx, "VCU118 Evaluation Board User Guide," UG1224.
- [7] Avnet, "PicoZed Based Embedded Computing Systems, Carrier Design Guide."
- [8] Avnet, "ISM Networking FMC Module User Guide," AES-FMC-ISMNET2-G.
- [9] AMD/Xilinx, "Gigabit Ethernet MAC (TEMAC) LogiCORE IP Product Guide," PG047.
- [10] Texas Instruments, "DP83867E/IS/CS Robust, High Immunity, Small Form Factor 10/100/1000 Ethernet Physical Layer Transceiver," Datasheet (Rev. G).

12. Appendices

12.1 Appendix A: Source Code

The complete, commented source tree accompanies this report in the project archive submitted through the projects site. The principal files are:

- **UDP_Main.c**, the complete MicroBlaze bare metal application: MAC/IP identity configuration, tuser_builder AXI4 Lite programming, AXI DMA Simple mode driver, DMA interrupt handlers, and the polling based echo loop that copies the received payload from the RX buffer to the TX buffer.
- **Splitter.vhd**, the AXI Stream pass through module used in the UDP baseline to replicate TVALID, TLAST, and TREADY for the tuser_builder. In the UDP++ build, these handshake signals are supplied directly by the udp_pp core. (Section 6.7).
- **tuser_builder VHDL sources**, the five register AXI4 Lite peripheral in which the routing registers assemble the 80 bit TUSER vector and the fifth register provides PHY readiness status, with the selected TUSER value latched at the start of each packet.
- **VCU118_rev2.0.xdc / board XDC**, pin, I/O standard, and PHY reset constraints for both platforms (including the V13 PHY reset binding, Section 9.1.2).
- **IP packaging Tcl sequence**, the VHDL 2008 enforcement commands applied to the UOE core (Section 6.5), forming the seed of the recommended TCL first flow (Section 10.4).
- **udp_pp.c**, the MicroBlaze control and test application for UDP++ configuration, DMA operation, session setup, counter reset, and hardware status monitoring.
- **udp_pp VHDL sources**, including the top level, AXI4 Lite register file, TX and RX engines, checksum unit, NAK checker, and retransmission controller.

12.1.1 UDP_Main.c: Structured Code Walkthrough

The application is deliberately small, a single translation unit built on the standard Xilinx standalone drivers (xaxidma, xintc, xil_printf, xparameters, and the AXI Timer driver), because UDP/IP header processing is performed in hardware, while the MicroBlaze polls DMA completion flags, copies the received payload from the RX buffer to the TX buffer, and starts the DMA transfers in loopback mode. The complete, commented source accompanies the

project archive submitted through the projects site. The walkthrough below follows its structure phase by phase.

Phase 1, Platform, identity, and PHY initialization. The file opens with the driver includes and the address/parameter definitions taken from `xparameters.h`, the code side mirror of the Address Editor map. Two dedicated helper groups follow. The MDIO pair, `mdio_init` and `mdio_write(u8 phy_addr, u8 reg_addr, u16 data)`, implements the DP83867 PHY register configuration described in Section 6.8: management channel writes to the PHY's IEEE and extended registers at its strapped address [10]. The `config_tuser_builder(u32 Length_bytes)` function programs the `tuser_builder` routing registers (Section 6.7): destination IP address, payload length, source UDP port, and destination UDP port. These fields form the 80 bit TUSER vector that the hardware latches at the start of each packet. The fifth AXI4 Lite register provides PHY readiness status and is not part of the TUSER vector.

Phase 2, DMA and timer initialization. The AXI DMA instance is located and initialized through `XAxiDma_LookupConfig / XAxiDma_CfgInitialize` against its base address, in Simple mode per the hardware configuration of Section 6.6. Per transfer completion interrupts are enabled on both channels with `XAxiDma_IntrEnable` (MM2S and S2MM channel selectors). The AXI Timer (`XTmrCtr`) is initialized alongside, it supplies the free running timestamps used by the latency instrumentation of Phase 4.

Phase 3, Interrupt controller and exception wiring. The AXI INTC is initialized (`XIntc_Initialize`), the two DMA interrupt lines, concatenated in hardware into the INTC (Section 6.6), are bound to their service routines with `XIntc_Connect`, enabled individually, and the controller started. The MicroBlaze exception layer is then armed (`Xil_ExceptionInit`, dispatcher registration, `Xil_ExceptionEnable`). From this point the application is fully event driven.

Phase 4, The steady state echo engine. Reception is prearmed by posting an S2MM transfer with `XAxiDma_SimpleTransfer` toward the BRAM receive region. The main loop then idles while hardware works. The per packet cycle proceeds in interrupt context:

1. RX completion, `dma_s2mm_isr`. A frame has traversed PHY → TEMAC → UOE and its stripped payload now resides in BRAM (Section 6.9, Stations 1–2). The handler reads and acknowledges the channel's interrupt status, captures the received length, and marks

the buffer ready for transmission. Error bits, if set, route to a channel reset path rather than silent continuation.

2. Header/metadata refresh, `update_packet_header(u32 seq)`. Before transmission, the code refreshes the outgoing packet's application level fields: an incrementing sequence number and an `XTmrCtr_GetValue` timestamp are written into the payload header region. These two fields are what make the host side latency and loss accounting of Sections 6.10 and 6.11 possible, each returned packet is individually identifiable and time referenced.
3. **TX launch.** With the routing metadata latched per packet by the `tuser_builder` hardware (Section 6.7), the transmit side `XAxiDma_SimpleTransfer` streams the payload back through Splitter → UOE, where it is reencapsulated with exchanged source/destination fields (Section 6.9, Stations 3–4).
4. **TX completion and rearm.** The MM2S interrupt confirms the frame has left memory toward the MAC. The handler acknowledges it and reposts the S2MM receive transfer, returning the system to its armed state. The interrupt pair per packet, one S2MM, one MM2S, is the processor's entire per packet cost, consistent with the throughput analysis of Section 6.10.

Diagnostic `xil_printf` tracing over the JTAG UART accompanies each phase during bring up and is compiled out of performance runs. The buffer base addresses and maximum packet length are defined against the Address Editor map and the provisioned 64 KB buffer capacity. Changing buffer provisioning requires touching only these definitions and the linker script.

12.1.2 `udp_pp.c`: UDP++ Control Software Walkthrough

The UDP++ control application (`udp_pp.c`, 631 lines) runs on the same platform as `UDP_Main.c` and follows the same skeleton, MDIO PHY bring up, DMA, interrupt controller, and timer initialization, extended with the UDP++ register interface. Its structure: global definitions and the register map. UOE/`tuser_builder` configuration and DMA/ISR plumbing inherited from the baseline. The scenario functions `send_packet_then_wait_for_gui_nak` and `send_packet_history_miss_test` driving Test Cases 1–3. `config_udp_mode` for session setup, counter reset, and layer enable, and the counter readout used as evidence in Section 7.7. `main` performs PHY initialization over MDIO and selects the active test scenario block.

12.1.3 tuser_builder_v1_0.vhd: Dynamic TUSER Length Injection

The tuser_builder top level was extended for variable length frames: the entity adds the inputs udp_packet_len_i[15:0] and udp_packet_len_valid_i. The architecture overrides TUSER bits [47:32] when the length is valid, tracks packet boundaries with an in_packet register, and latches the selected TUSER word per frame. The AXI4 Lite slave submodule is identical to the UDP baseline.

12.1.4 udp_pp VHDL Source Tree: Architectural Walkthrough

The complete hardware source of the UDP++ reliability core comprises seven VHDL units, packaged as the udp_pp v1.0 IP under Vivado 2023.2, prior to the toolchain update of Section 4.3, and carried unchanged into the 2024.2 project. All units share the single 125 MHz clock and active low reset of the platform clocking architecture (Section 6.4). The walkthrough below follows the hierarchy top down and cross references the design description of Section 7. The machine readable sources accompany the report in the project archive.

udp_pp_v1_0.vhd, structural top level (452 lines). The top module is a structural wrapper: it declares the generics (C_S00_AXI_DATA_WIDTH = 32, C_S00_AXI_ADDR_WIDTH = 6) and the architecture constants TX_HISTORY_DEPTH_C = 16, MAX_PAYLOAD_BYTES_C = 1024, and RX_BUFFER_DEPTH_C = 16. Instantiates the five subunits (u_axi_slave, u_tx, u_rx, u_nak_checker, u_retx_ctrl), and routes the two symmetric AXI Stream paths (DMA → TX → UOE and UOE → RX → DMA). The TX output handshake is replicated toward the extended tuser_builder together with the per frame length (tuser_packet_len / tuser_packet_len_valid), realizing the dynamic TUSER mechanism of Section 7.5. Two internal NAK flows are wired here: the gap detection request from the receiver to the transmitter (rx_nak_req/rx_nak_seq → nak_tx_req_i), and the incoming NAK validation chain (RX field extraction → NAK checker → retransmission controller → TX). A single synchronous process accumulates the Table 7.1 status counters, merging, for example, session mismatch pulses from both the data receiver and the NAK checker, and drives the read back inputs of the register file. udp_pp_v1_0_S00_AXI.vhd, AXI4 Lite register file (449 lines). The control plane bridge between the MicroBlaze software (Appendix 12.1.2) and the hardware. The standard Xilinx slave template (ADDR_LSB = 2, OPT_MEM_ADDR_BITS = 3, sixteen slave registers) is extended with the UDP++ register set. On the write path, slv_reg0 (CTRL) and slv_reg1 (SESSION_ID) are decoded into single clock control pulses: a write with WDATA(2) = '1' emits reset_counters_pulse_o and self clears the bit, and any SESSION_ID write emits session_wr_pulse_o, which resets the sequence/history state in both datapath engines. On the read path, the address decode mirrors Table 7.1 exactly (offsets 0x00–0x34), multiplexing the

live status inputs, TX window bounds, RX packet/byte counters, checksum/session error counters, and the NAK/RETX counters, onto S_AXI_RDATA. These registers constitute the primary evidence source quoted throughout Section 7.7.

`udp_pp_tx.vhd`, transmit engine and TX history buffer (519 lines). A 23 state FSM implements the three transmit paths of Section 7. Byte exact frame layouts are defined by two pure functions: `header_byte` composes the 16 byte DATA header (opcode 0x01, session, window fields, 32 bit sequence, Fletcher 16 checksum) and `nak_packet_byte` the 8 byte control frame (opcode 0x02). The DATA path (TX_CLEAR_CSUM → TX_INIT_HEADER_CSUM → TX_RECV_PAYLOAD → TX_WAIT_CSUM → TX_WRITE_HEADER → TX_SEND_PRIME → TX_SEND_PACKET) streams the DMA payload into the BRAM history array `tx_history_mem_s` while feeding the checksum engine, then writes the finalized header into the same slot and transmits with a registered read prefetch. On the last beat it pulses `tx_data_sent`, advances `tx_seq_next`, and maintains the sliding window (`tx_history_count` / `tx_history_oldest_seq`), the 16 frame recoverability property of Section 7.4. The RETX path replays a slot addressed by the retransmission controller and pulses `retx_done`. The NAK path assembles, checksums, and transmits the 8 byte request generated by the receiver. Pending latches hold retransmission and NAK requests until the engine idles. CTRL[1]/CTRL[0] select UDP++, transparent UDP pass through, or disabled mode, and a SESSION_ID write returns the engine to a clean sequence state.

`udp_pp_checksum.vhd`, Fletcher 16 checksum engine (41 lines). A compact synchronous implementation of the Fletcher 16 algorithm used by every frame class: two 8 bit accumulators are updated modulo 255 for each enabled input byte (`sum1 += byte`, `sum2 += sum1`), cleared per frame via `clear_i`, and composed onto `csum_o` with `sum2` as the high byte, matching the checksum low/checksum high positions of the header layout. The unit is instantiated three times, giving the transmitter, the receiver, and the NAK checker independent single cycle per byte integrity engines. The one clock settle states (TX_WAIT_CSUM, RX_WAIT_DATA_CSUM, NAK_WAIT_CSUM) in the surrounding FSMs account for its registered output.

`udp_pp_rx.vhd`, receive engine, header stripping, and reorder window (631 lines). The receiver contains two cooperating state machines over a multislot packet buffer (`rx_payload_mem_s`, 16 slots × 1,024 bytes with per slot sequence, length, and valid flags), allowing one packet to be received while another is delivered. The parser FSM classifies each incoming frame by opcode: DATA frames (0x01) have their 16 byte header extracted and stripped, are admitted only if `seq_in_window` holds and the target slot (`seq mod 16`) is free, and are then validated in

RX_CHECK_DATA, session filtering (mismatch pulse, deliberately without a NAK), Fletcher 16 verification, and the gap logic of Section 7.3: an in order packet advances the expected sequence across all contiguously stored future slots and requests only a still missing one. A future packet is stored while a NAK is issued for the first missing sequence. A corrupted expected packet is NAKed. Stale duplicates are dropped quietly, the reordering tolerance demonstrated in Section 7.7.4. NAK frames (0x02) have their fields extracted and handed to the NAK checker. The DMA sender FSM (SND_IDLE/PRIME/ACTIVE) releases payloads strictly in sequence order (deliver_rx_seq) from a registered data byte, guaranteeing in order, header free delivery to memory, while a push/pop counter tracks buffer occupancy for back pressure.

udp_pp_nak_checker.vhd, NAK validation (160 lines). The gatekeeper between a received control frame and the retransmission machinery. On nak_frame_valid it latches the reported fields, drives bytes 0–5 of the frame through its own Fletcher 16 instance, and issues a verdict in NAK_CHECK: an opcode other than 0x02 or a checksum mismatch raises nak_csum_err_pulse, a foreign session raises nak_session_mismatch, and only a fully valid request asserts nak_valid_o with the missing sequence number. This preprocessor filtering prevents spurious retransmissions before any processor involvement, and its error pulses feed the NAK_CSUM_ERR and session mismatch counters of Table 7.1.

udp_pp_retx_ctrl.vhd, retransmission controller (51 lines). The final link of the NAK chain converts a validated request into a concrete replay command by testing the requested sequence against the live history window [tx_history_oldest_seq, tx_seq_next): a sequence below the window (already overwritten) or at/above it (not yet transmitted) raises nak_missed_pulse, incrementing the NAK_MISSED counter while the link continues uninterrupted (history miss, Section 7.4), whereas an in window sequence asserts retx_req with retx_slot = seq mod 16 toward the transmit engine (history hit), which replays the frame and increments RETX_COUNT.

The seven units realize the complete receiver driven reliability loop of Section 7 entirely in programmable logic: header injection and history retention on transmit, windowed validation, stripping, and reordering on receive, independent NAK verification, and window checked selective retransmission, with the processor confined, as designed, to configuration and monitoring through the AXI4 Lite register file described above.

12.2 Appendix B: Management Tasks Table

Weekly activities were logged in the management system throughout both semesters, per procedure requirements [4]. The board view below illustrates the live task tracking, execution status, and estimated timelines for all project phases.

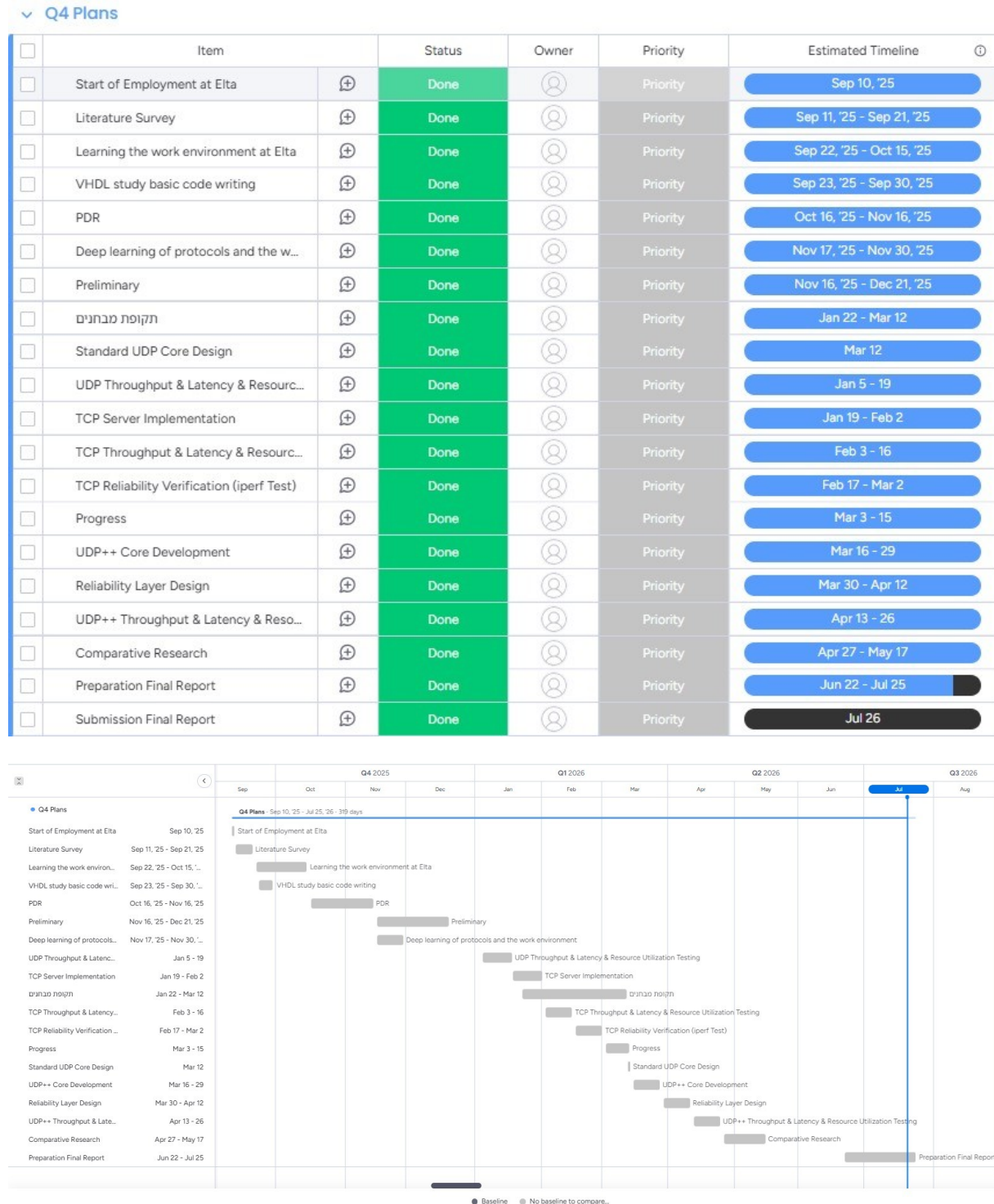


Table 12.1: Gantt Execution

12.3 Appendix C: Linker Script and Memory Map Details

This appendix consolidates the memory architecture artifacts referenced in Sections 6.3 and 9.2: the final lscript.ld (all sections mapped to lmb_bram, 128 KB region), the section size report establishing the 23,248 B application footprint, and the Vivado Address Editor map, Network 0 (AXI data/control peripherals: DMA, Timer, UOE, BRAM controller, UART, tuser_builder) and Network 1 (LMB instruction space), verified overlap free before bitstream generation.

UDP++ update: the reliability layer adds a single AXI4 Lite slave to Network 0, udp_pp_0/S00_AXI at base address 0x44A2_0000 (64 KB, high address 0x44A2_FFFF), adjacent to the Ethernet control block (tri_mode_ethernet_mac_0 at 0x44A0_0000 and tuser_builder_0 at 0x44A1_0000), so that the entire UDP++ control plane occupies one contiguous region of the address space. Table 12.2 lists the consolidated, overlap free map as verified in the Address Editor. The Address Editor assignment is shown in Figure 7.3, and Figure 12.1 shows the rendered address map.

Slave segment	Base address	Range	Network
microblaze_0_local_memory (dlmb/ilmb, SLMB)	0x0000_0000	128 KB	0 / 1
top_uoe_0 / S_AXI	0x0002_0000	16 KB	0
axi_uartlite_0 / S_AXI	0x4060_0000	64 KB	0
microblaze_0_axi_intc / s_axi	0x4120_0000	64 KB	0
mdm_1 / S_AXI	0x4140_0000	4 KB	0
axi_timer_0 / S_AXI	0x41C0_0000	64 KB	0
axi_dma_0 / S_AXI_LITE	0x41E0_0000	64 KB	0
tri_mode_ethernet_mac_0 / s_axi	0x44A0_0000	64 KB	0
tuser_builder_0 / S00_AXI	0x44A1_0000	64 KB	0
udp_pp_0 / S00_AXI	0x44A2_0000	64 KB	0
axi_bram_ctrl_0 / S_AXI	0xC000_0000	64 KB	0

Table 12.2: Consolidated address map of design_UDP_PP, including the UDP++ layer.



Figure 12.1: Consolidated address map of design_UDP_PP.

12.4 Appendix D: Grade Recommendation Form

המלצת ציון לדו"ח מסכם

אם יש צורך, לכל סטודנטית בנפרד

מספר הפרויקט: P-2026-142

שם הפרויקט: Reliable High-Speed Networking on FPGA Platforms (UDP, TCP, and UDP++ Protocols)

שמות המנחים: פרופ' רון דבורא ואייל למפל

שם הסטודנט: יאיר טיירי

שם הסטודנטית: אושרת אליזה

קריטריון	1 - חלש	2 - בינוני	3 - טוב	4 - ט"מ	5 - מצוין
הגדרת המטרה – האם מטרת הפרויקט ברורה? המטרה צריכה לכלול גם את התרומה הצפויה מהשלמת הפרויקט					
ארכיטקטורת הפתרון – האם הובהר מדוע הארכיטקטורה המוצעת לפתרון ההנדסי מתאימה לפתרון הבעיה? האם הייתה התייחסות לאלטרנטיבות?					
הצגת התוצאות – האם הוסברו המשמעויות של התוצאות, או המסקנות הנובעות מהן, האם רק הייתה הצגה עובדתית של התוצאות?					
מבנה – מצגת – האם יש מבנה הגיוני להצגת העבודה? מבוע, רקע ו-SoTA, מטרות, שיטה, פתרון, תוצאות, מסקנות					
רמת קושי – כיצד הינך מעריך את רמת הקושי הצפויה בפרויקט בהשוואה לפרויקטים אחרים השנה או בשנים קודמות?					
איכות כתיבה / הצגה – נא להתייחס לאיכות הצגת העבודה: תחביר, מבנה וקשר הגיוני בין החלקים במהלך הצגת העבודה.					
שורה תחתונה – האם מטרות הפרויקט הושגו? יש להתייחס למטרות כפי שהוגדרו בתחילה.					
הבנה – האם נראה שהסטודנט מבין את העבודה שנעשתה? האם עומק הדיון מספק?					
השקעה – האם ניכר שהסטודנט השקיע בעבודה בפרויקט ו/או בהכנת ההצגה של העבודה? האם נראה שהעבודה נעשתה בחופזה, כדי לצאת מידי חובה?					
דירוג – כיצד הנך מעריך את איכות הפרויקט ביחס לכלל הפרויקטים שראית?					
המאפיינים העיקריים בפרויקט – יש לסמן את כמות המאפיינים שהושגו (ולו באופן חלקי). מאפיין אחד בעמודה 1 וכן הלאה.... המאפיינים: * מימוש/יצור מערכת * אנליזה מתמטית * תכנון אלגוריתמי * אינטגרציה עם מערכת קיימת * חקר ביצועים					

אם יש כוונה לפרסם/יפורסם מאמר, שם כתב העת ומועד משוער להגשה:

ציין אם יש כוונה לשקול המלצה כפרויקט מצטיין:

הערות נוספות: